L L M E N G I N E E R I N G

Week 05

Optimization

Thursday, 12 November 2026

Interactive version: https://llm.strueber.dev/#/week-05

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

Recap

1

Announcements

Quiz 2 on ILIAS

- Weeks 3, 4 and Bonus 2
- Due Friday 13 November, end of day

Fourth exercise: this lecture

- Due Wednesday 18 November, 22:00
- Email when it reaches your repository

▼ Dates for the coming week

Quiz 2 is on ILIAS and closes at the end of Friday, 13 November. It covers Week 3, Week 4 and Bonus 2.

The fourth coding exercise covers the content of this lecture and is due on Wednesday, 18 November, at 22:00. You will get an email when it is in your pair's repository.

The plan until February changes

- Cluster down 23 November to 4 December
- New Kids comes as a video, 3 December
- No lecture on 10 December, tutorial on Zoom
- Free week for your project: ideas, pitch, first experiments
- And a fun Christmas surprise event, more on both next week

▼ What changes until February

The UC3 cluster is switched off from 23 November to 4 December, because its computing centre is disconnected from the power grid for maintenance. The end date is expected, not certain. Most of our group is also at the NeurIPS conference from 3 to 15 December. So the plan for the rest of the term changes. The next slide gives the new order, and the weeks are numbered in it: New Kids on the Block becomes Week 8 and mixture of experts Week 13.

- Week 8, New Kids on the Block, comes as a recorded video on 3 December. It is a tour of recent techniques and a good source of final-project ideas, so it comes before the break.
- There is no lecture on 10 December. The tutorial that week still takes place, on Zoom, in the Thursday slot. Use the free week for your final project: find an idea, work on your pitch and run first experiments. Bonus 4 and the new Bonus 5, on vision-language and multimodal models, also fit into these two weeks.
- That week also brings a fun Christmas surprise event. Details on it and on the final projects follow next week.

New dates until February

Date	Lecture	Exercise due
19 Nov	Week 6: Parallelization	25 Nov
26 Nov	Week 7: Scaling laws	16 Dec
3 Dec	Week 8: New Kids (video)	none
10 Dec	No lecture: NeurIPS	none
17 Dec	Week 9: Data curation	6 Jan
7 Jan	Week 10: Inference	13 Jan
14 Jan	Week 11: Post-training I	20 Jan
21 Jan	Week 12: Post-training II	27 Jan
28 Jan	Week 13: Mixture of experts	3 Feb
4 Feb	Week 14: Guest lecture (planned)	none

- Pass any 6 of 11 exercises
- Quiz 6 moves to Friday 5 February

▼ The new dates

The table gives every lecture and exercise deadline until February.

- On 10 December there is no lecture, because most of our group is at NeurIPS.
- Data curation follows on 17 December, then inference, the two post-training weeks, and mixture of experts on 28 January. The last lecture, on 4 February, is planned as a guest lecture. The tutorial that week is the last chance to pitch your final project.
- The fifth exercise, on Week 6, is still due on 25 November. Its tests run on CPU, so the downtime does not stop you from passing it. Run the GPU measurements before 23 November if you can.
- The sixth exercise, on Week 7, runs until 16 December. Its fitting code runs on a laptop, and the scaling sweep needs the cluster from 5 December.
- Week 8 has no exercise this year. That leaves 11 exercises, and you still need to pass any 6 of them.
- Quiz 6 moves to Friday, 5 February, so it can include the last lectures.

What changed between these runs?

Test run, 53B: attention logits pass 1,000.
Main run, 1T: 15.5T tokens, no loss spike.

What keeps a long training run stable?

- Optimizer: how each update is shaped
- Schedule: learning rate and batch over time
- Monitoring: signals that warn early
- Controls: bound extremes where they form
- Precision: range and accuracy per tensor
- This week: build all five into your trainer

▼ **Five choices keep a long run stable**

The two runs come from the Kimi K2 report by Moonshot AI's Kimi Team. The first was a mid-scale test run of a mixture-of-experts model with 9B activated and 53B total parameters. In its Figure 2 the report writes that "attention logits rapidly exceed 1000, which could lead to potential numerical instabilities and even training divergence" (Kimi Team, 2025). It does not say that this run diverged. The second run is Kimi K2 itself, a model with one trillion total parameters. The report states that it was pre-trained on 15.5 trillion tokens "with zero loss spike" (Kimi Team, 2025).

The report does not test the choices one at a time, so it cannot show which one kept the loss smooth. Every lab report cited below shares a related limit: it records what one team did for one model, usually in a single run and often without the numbers behind a choice, and none compares its choices with another lab's under matched conditions. Unless a note names a controlled test, treat a reported choice as one data point for that model, to be validated before you adopt it.

The question of the lecture is broader: what does it take to keep a long training run stable, and how do you see trouble coming? Five choices interact. The **optimizer** shapes each parameter update and keeps state of its own, which costs memory. The **schedule** sets how the learning rate and the batch size change over the token budget. **Monitoring** lines up the loss, the gradient norm before clipping, the throughput and the training state on one step axis, so that a signal can warn before the loss diverges. **Controls** such as clipping and normalization bound extreme values on the tensor path where they form. **Precision** decides which ranges, accumulations and states need more bits than the rest of the computation.

This week you build all five into your trainer. You write AdamW and Muon from their update rules, and a warmup-stable-decay (WSD) schedule from which you can start a decay leg, a short learning-rate decay branched from a saved checkpoint. The trainer measures the pre-clip gradient norm, a spike score and the largest attention logit. It trains in bf16 or fp16 under autocast while the parameters stay in fp32, with a dynamic loss scaler for fp16. qk-norm, a normalization of queries and keys inside attention, is a configuration option of the model. You

then train two runs at a hot learning rate, one with qk-norm and one without, and compare their traces.

6

Where Week 4 left off Read on your own

- Attention: limited by compute or by bytes moved
- FlashAttention: fewer bytes, same result
- Extreme values: attention sinks, large logits, massive activations
- Now: keep values in range while training

▼ What Week 4 hands to this week

Weeks 2 to 4 built the model. This week is about training it reliably.

Week 4 asked what limits attention's speed. For a stated workload, attention can be compute-bound or bound by the bytes it moves through memory. Many modern attention mechanisms speed attention up by moving fewer bytes, often more than by doing less arithmetic. FlashAttention is the clearest case: it computes the same attention function as the standard implementation and never writes the full score matrix to memory.

Week 4 also met extreme values. Some trained heads put a large share of their attention on a few positions, the attention sinks, and the scores behind that pattern can be large. Sun et al. report a separate observation, massive activations: a very few hidden-state values, at fixed positions, far larger than all the others (Sun et al., 2024).

Training turns extreme values into a stability problem in two ways. Attention logits that keep growing push the softmax toward putting nearly all of a query's weight on one key. Extreme hidden-state values press against the finite range of later operations and of low-precision number formats.

Adam, AdamW & the Graveyard

Training steps against the loss gradient

- Goal: lower the loss over the parameters
- Step against the gradient: θ ← θ − η·g
- Stochastic gradient descent (SGD): gradient of one minibatch
- One learning rate for every coordinate

▼ **Stochastic gradient descent is the baseline**

Training a language model means finding parameters θ that make the loss $\mathcal{L}(\theta)$ small. The gradient $\nabla_\theta \mathcal{L}$ points in the direction in which the loss rises fastest, so a small step against it lowers the loss. Gradient descent repeats that step,

$$\theta_t = \theta_{t-1} - \eta\, g_t, \qquad g_t = \nabla_\theta \mathcal{L}_t(\theta_{t-1}), \tag{0}$$

where θ_t is the parameter vector after step t and the learning rate η sets how far each step goes.

The loss is an average over the whole training set, which is far too large to differentiate at every step. **Stochastic gradient descent** (SGD) uses the gradient of one minibatch instead, $g_t = \nabla_\theta \mathcal{L}_t(\theta_{t-1})$, evaluated at the parameters before the update. Over random minibatches its average is the full gradient, so each step points the right way on average, and the minibatch adds noise from step to step.

Every optimizer in this lecture keeps the minibatch gradient and changes what happens to it before the step. SGD has one weakness that the next slides address. Transformer gradients differ greatly from one coordinate to the next. Embedding rows, normalization gains and the entries of weight matrices receive gradients of very different sizes, and they do not share one useful step size. Plain SGD moves every coordinate by the same multiple of its gradient, so a learning rate that is safe for the coordinates with large gradients hardly moves the others. In practice, plain SGD barely trains a transformer.

Adam adapts each coordinate's step

(1) $$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$$

(2) $$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \theta_t = \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- m: exponential moving average (EMA) of gradients, averages out noise
- v: EMA of squared gradients, sets each coordinate's step
- Bias correction: both start at zero, t counts steps
- Adam: a diagonal preconditioner

▼ Adam is a bias-corrected diagonal preconditioner

Adam gives every coordinate its own step size (Kingma & Ba, 2015). It keeps two running averages per coordinate. The momentum m is an exponential moving average (EMA) of the gradients. Minibatch noise changes sign from step to step and partly cancels in the average, while the direction that recurs adds up. Across a narrow valley the back-and-forth part cancels and the part along the valley builds up. The second moment v is an EMA of the squared gradients. Its square root estimates how large a coordinate's gradients usually are, and dividing by it gives every coordinate a step of about the same size: coordinates with large gradients take smaller steps than SGD would give them, and coordinates with small gradients larger ones. Bias correction rescales both averages, because both start at zero.

In formulas (1) and (2), θ_t and g_t are as in formula (0). The first-moment EMA is m_t and the uncentered second-moment EMA v_t. The coefficients β_1 and β_2 are fixed hyperparameters that set how long each average remembers, η is the learning rate, and ϵ is a small positive constant that keeps the denominator away from zero. The step counter t runs $1, 2, 3, \ldots$, and β_1^t is β_1 raised to the power t. A hat marks bias correction, $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$. Every operation acts elementwise, and the scalars apply to every coordinate.

The power t comes from the zero start. Unrolled from $m_0 = 0$, formula (1) gives

$$m_t = (1-\beta_1) \sum_{i=1}^{t} \beta_1^{t-i}\, g_i,$$

and the weights on the gradients sum to $(1-\beta_1)(1+\beta_1+\cdots+\beta_1^{t-1}) = 1-\beta_1^t$. The missing weight β_1^t falls on the zero the buffer started from, so early averages are pulled toward zero. Dividing by $1-\beta_1^t$ makes the weights sum to one again, and the same holds for v_t with

β_2. As t grows, β^t goes to zero and the correction fades: with $\beta_1 = 0.9$ it is negligible after about 50 steps, while with $\beta_2 = 0.999$ it still matters after a thousand.

A preconditioner is a matrix that multiplies the update direction before the step. The update in formula (2) can be written as

$$\theta_t = \theta_{t-1} - \eta\, P_t\, \hat{m}_t,$$

where P_t is the diagonal matrix with entries $1/(\sqrt{\hat{v}_{t,j}} + \epsilon)$. Adam is therefore a **diagonal preconditioner**: it rescales each coordinate by that coordinate's own gradient history, and it never mixes coordinates.

Claim. At the first step, bias correction makes each coordinate's adaptive term about the sign of its gradient. Adam starts both buffers at zero, $m_0 = v_0 = 0$, so

$$\begin{aligned}
\hat{m}_1 &= \frac{(1-\beta_1)\, g_1}{1-\beta_1} = g_1 && \text{formula (1)}, m_0 = 0 \\
\hat{v}_1 &= \frac{(1-\beta_2)\, g_1^2}{1-\beta_2} = g_1^2 && \text{formula (1)}, v_0 = 0 \\
\frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \epsilon} &= \frac{g_1}{|g_1| + \epsilon} && \sqrt{g_1^2} = |g_1| \\
&\approx \operatorname{sign}(g_1) && \text{when } |g_1| \gg \epsilon.
\end{aligned}$$

Without the correction, and with ϵ neglected, the first step would carry the factor $(1-\beta_1)/\sqrt{1-\beta_2}$, which comes from the zero initialization and not from the gradient. With $\beta_1 = 0.9$ it is about 0.45 at the course's $\beta_2 = 0.95$, a smaller step, and about 3.2 at Adam's original $\beta_2 = 0.999$, a larger one.

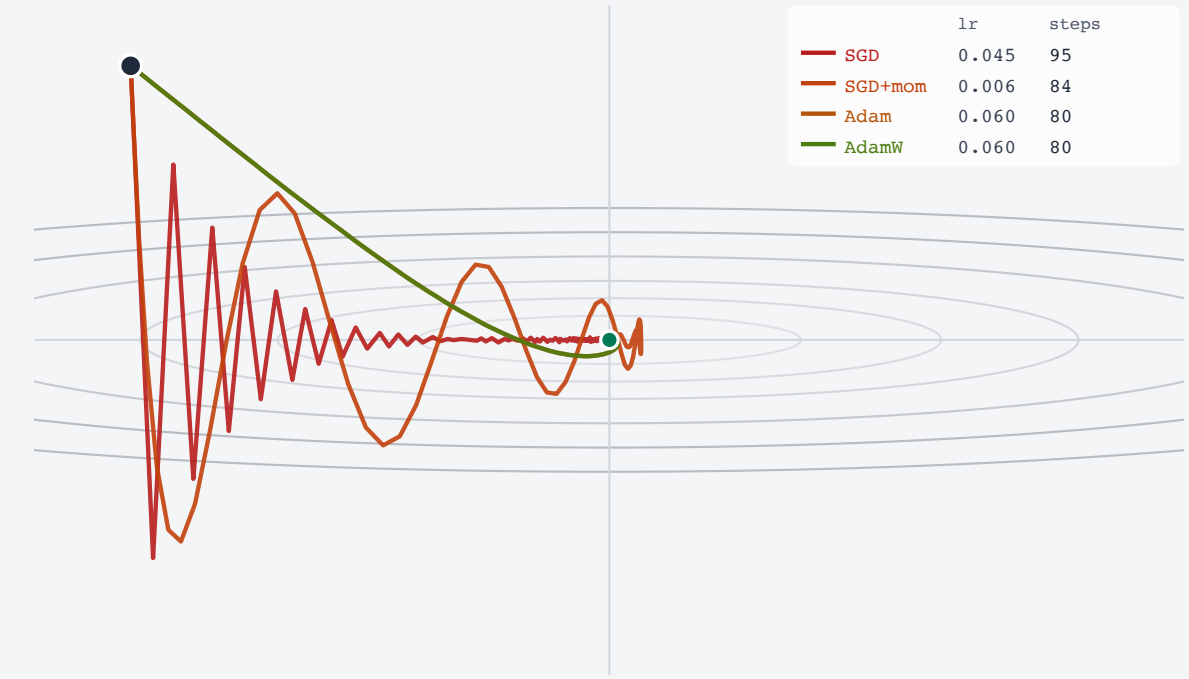
In code, initialize both buffers to zero, increment t before forming $1-\beta_1^t$ and $1-\beta_2^t$, and keep the placement of ϵ the same in code and in checkpoints.

Adam on an ill-conditioned valley Read on your own

- Condition number κ: the valley's elongation
- Fair race: tune each optimizer
- Decay slider: Adam+L2 against AdamW

ILLUSTRATIVE Optimizers: SGD SGD+mom Adam AdamW

LR mode: Tuned per optimizer each optimizer runs at its own LR; the slider scales all of them



LR multiplier: ×1.00

Condition number κ: 40

β₁ (momentum): 0.90

β₂ (second moment): 0.999

Weight decay: 0.00

Gradient noise

Replay Reset step 96 / 96

▼ Reading the optimizer race on a synthetic valley

The widget races SGD, SGD with momentum, Adam and AdamW down the two-dimensional quadratic valley $f(x, y) = \frac{1}{2}(x^2 + \kappa y^2)$, from the same start and with the same gradient noise for every optimizer. The **condition number** κ is the ratio of the valley's curvature along its steep axis to its curvature along its flat axis, so a larger κ means a more elongated, more ill-conditioned valley. Here κ stands in for the spread of gradient scales inside a transformer. With one scalar step size, a step small enough for the steep axis crawls along the flat one. Each run stops once $f < 0.001$, and the legend scores the race in steps to that target.

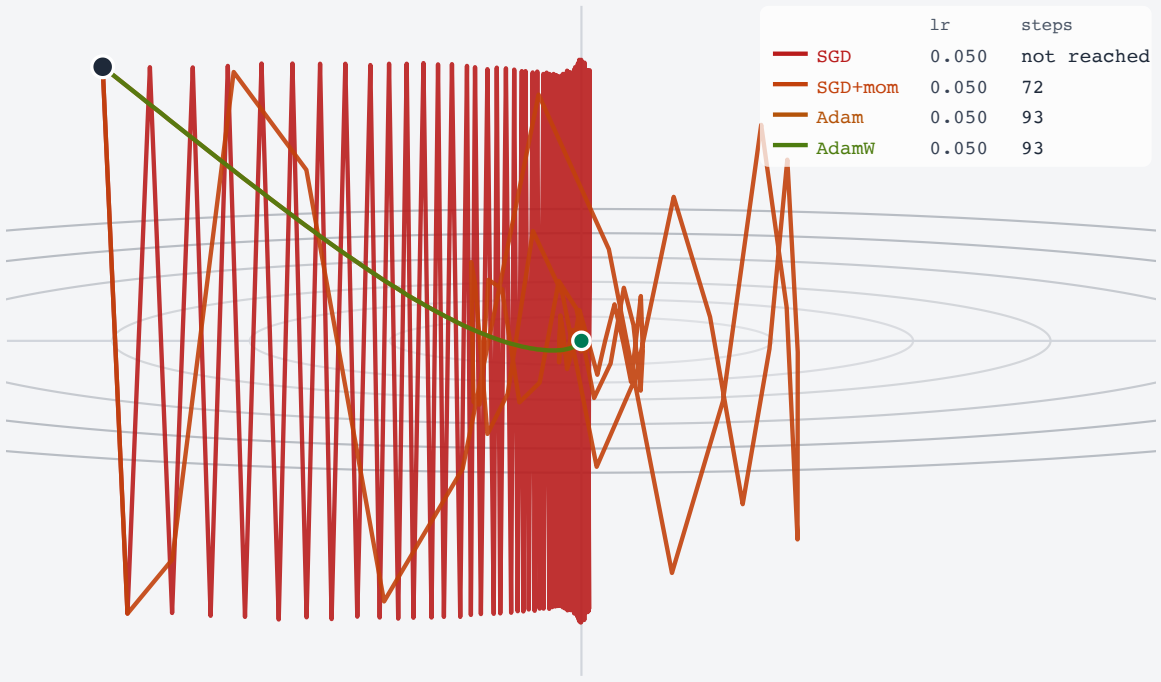
A fair optimizer comparison tunes each optimizer separately, so the widget opens in "Tuned per optimizer" mode: each optimizer runs at its own learning rate, and the slider scales all of them together. At the opening setting, $\kappa = 40$, Adam and AdamW reach the target in 80 steps, SGD with momentum in 84 and plain SGD in 95. The tuned rates show that a learning rate does not carry over from one optimizer to another. On this bowl Adam's tuned rate is not ten times below SGD's, as it often is in real networks. SGD's rate is capped by the curvature: on the steep axis each step multiplies the distance to the minimum by $1 - \eta\kappa$, so the iteration diverges once $\eta\kappa > 2$. Adam takes steps of about η in each coordinate, so it needs a large rate to cover the distance in time.

Click "Tuned per optimizer" again to switch it off, and every optimizer runs at the slider's shared rate, 0.05. This shows why a shared rate favors some optimizers. There $\eta\kappa = 2$, plain SGD is at its stability edge, its steps flip back and forth across the narrow axis, and it never reaches the target. SGD with momentum now finishes first, in 72 steps, and Adam needs 93. Set above zero, the weight-decay slider adds a dashed Adam+L2 trace beside the solid AdamW trace. AdamW shrinks the weights directly, Adam+L2 adds the decay to the gradient, and the two paths separate because the adaptive step scales L2 decay differently on each coordinate.

The valley and its noise are synthetic. Its ordering and its step counts are not a measured ranking of optimizers for language models.

ILLUSTRATIVE Optimizers: SGD SGD+mom Adam AdamW

LR mode: **Tuned per optimizer** shared LR: every optimizer uses the slider value



Learning rate: 0.050

Condition number κ: 40

β_1 (momentum): 0.90

β_2 (second moment): 0.999

Weight decay: 0.00

Gradient noise

Replay Reset step 500 / 500

One shared learning rate, 0.05, for all four optimizers. Plain SGD is at its stability edge, $\eta\kappa = 2$: its steps flip back and forth across the narrow axis, and it never reaches the target. SGD with momentum now finishes first, in 72 steps, and Adam and AdamW need 93 each. At one shared rate the race favors some optimizers over others.

AdamW decouples weight decay from the adaptive step

(3) $$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \, \theta_{t-1} \right)$$

- An L2 penalty in the gradient gets divided by $\sqrt{\hat{v}_t}$
- Same λ, but each coordinate shrinks by a different amount
- AdamW, formula (3): decay outside the adaptive step
- Decay ηλ falls with the learning rate (LR)

▼ **AdamW keeps the decay out of the adaptive denominator**

Where you put weight decay changes what it does. For plain SGD, an L2 penalty and weight decay give the same step. Adding $\lambda\theta_{t-1}$ to the gradient yields

$$\theta_t = \theta_{t-1} - \eta \, (g_t + \lambda\theta_{t-1}) = (1 - \eta\lambda) \, \theta_{t-1} - \eta \, g_t,$$

which is weight decay by the factor $1 - \eta\lambda$.

Adam breaks that equivalence. In the coupled form, coordinate j uses the penalized gradient $\tilde{g}_{t,j} = g_{t,j} + \lambda\theta_{t-1,j}$. Both moment buffers are formed from it,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \, \tilde{g}_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) \, \tilde{g}_t^2,$$

so the penalty passes through the adaptive denominator $\sqrt{\hat{v}_t} + \epsilon$ like the rest of the gradient. Coordinates with different gradient histories therefore shrink by different amounts even when they share the same λ.

AdamW decouples the decay: it acts on the weight directly, outside the adaptive step (Loshchilov & Hutter, 2019). In formula (3) the bias-corrected moments come from the task gradient g_t alone.

Claim. Under AdamW, every coordinate of a parameter group shrinks by the same factor $1 - \eta\lambda$ per step. Distributing η over the two terms of formula (3) and collecting θ_{t-1} gives

$$\theta_t = (1 - \eta\lambda) \, \theta_{t-1} - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

The decay factor does not contain $\hat{v}_t$. An implementation can multiply the old parameter by $1 - \eta\lambda$ and then apply the adaptive update, with the same placement of ϵ as in formula (2). A

parameter group with a different decay policy carries its own λ.

The decay per step is $\eta\lambda$, so weight decay is tied to the learning-rate schedule: as the learning rate (LR) decays, the decay shrinks with it. Which decay setting trains better remains an empirical question. With a decay of $\eta\lambda$ per step, a weight forgets its old value over roughly $1/(\eta\lambda)$ steps. Wang and Aitchison find that this timescale, measured as a fraction of the run, is what carries over across model and dataset sizes, so a large change of the learning rate should come with a change of λ that keeps $\eta\lambda$ about the same (Wang & Aitchison, 2025). Kosson et al. argue that weight decay, more than the per-width scaling of µP, is what lets learning rates transfer across model widths in practice (Kosson et al., 2026).

A practical AdamW starting point Read on your own

Setting	Course starting point	Why it is here
beta1	0.9	paper default
beta2	**0.95**	changed from the paper default 0.999
epsilon	**1e-8**	chosen with published evidence (OLMo 2)
weight decay	0.1	common disclosed choice
gradient clip	1.0	common disclosed choice
peak LR	**varies with scale**	especially scale-sensitive

- β2 0.95: a 20-step horizon
- Peak LR: re-tune it whenever the model's scale changes

▼ β₂, ε and the peak rate need the most care

Several large pretraining runs that disclose their optimizer cluster around the settings in the table. DeepSeek V1 to V3 (DeepSeek-AI, 2024a; DeepSeek-AI, 2024b), Llama and OLMo 2 report such settings. Qwen reported such settings in 2023, but its 2026 Qwen3.8-Flash-Next trains its weight matrices with Muon (Qwen Team, 2026). Only one entry has a controlled test behind it. OLMo 2 reports that changing ϵ from 10^{-5} to 10^{-8} lowered and stabilized its early gradient norms (OLMo Team, 2025). A smaller ϵ keeps the denominator small enough for useful early updates and still prevents division by zero. MAI-Thinking-1 is a counterexample for one model, with $\beta_1 = 0.95$, $\beta_2 = 0.925$ and weight decay set per parameter group (Microsoft AI Team, 2026).

The memory horizon of an EMA with coefficient β is about $1/(1-\beta)$ steps. The horizon is a heuristic: the weight of older gradients fades gradually, with no hard cutoff. For $\beta_2 = 0.999$ it gives $1/(1-0.999) = 1{,}000$ steps, and for $\beta_2 = 0.95$ it gives $1/(1-0.95) = 20$ steps. OLMo 2, Llama and DeepSeek report 0.95. The change shortens how long any squared gradient stays in v, from about 1,000 steps to about 20. At the start of a run both moment histories are short and noisy. Bias correction removes the known factor from the zero initialization, but it cannot create a long history.

For the other entries, keep the placement of ϵ as in formula (2), apply weight decay only to the groups you intend, and log the gradient norm before clipping so that you can choose a clip threshold. The peak learning rate is the entry most sensitive to scale and to the parametrization. OLMo 2 reports 3e-4 at 7B and 9e-4 at 13B, and DeepSeek V3 reports 2.2e-4

at 671B (OLMo Team, 2025; DeepSeek-AI, 2024b). When the model's scale changes, re-tune it, or transfer it with evidence from an empirical scaling ladder.

Optimizer state is real memory Read on your own

- m, v in fp32: 8 bytes per parameter
- A 7B model needs 56 GB of optimizer state
- Week 6 shards these buffers

▼ Worked example: a 7B model's training memory

Week 2 split training memory into parameters, gradients, activations and optimizer state. AdamW's state is its two moment buffers. In fp32 the first moment m costs 4 B per parameter and the second moment v another 4 B, so the moments alone cost 8 B per parameter, on top of the weights and gradients. For 7 billion parameters, in decimal units, that is

$$7 \times 10^9 \times 8 \text{ bytes} = 56 \times 10^9 \text{ bytes} = 56 \text{ GB}$$

of optimizer state.

The full account depends on how an implementation stores each buffer. Take one common mixed-precision setup for a dense 7B model: a bf16 copy of the weights for computation (2 B per parameter), bf16 gradients (2 B), an fp32 copy of the weights that the optimizer updates, the master weights (4 B), and fp32 m and v (4 B each). Then

$$\begin{aligned} \text{per parameter:} \quad & 2 + 2 + 4 + 4 + 4 = 16 \text{ bytes}, \\ \text{for the model:} \quad & 7 \times 10^9 \times 16 \text{ bytes} = 112 \text{ GB}. \end{aligned}$$

This is the logical total before any sharding, without activations or temporary buffers, and it changes when an implementation stores the gradients or weights differently.

The master weights and the moments are fp32 even when the computation runs in bf16, because bf16 cannot hold small changes. Its values near 1 are about 0.008 apart, so an AdamW step of about the learning rate, 10^{-4} to 10^{-3}, does not move a normalization gain of 1.0 at all. With Adam's original $\beta_2 = 0.999$, one step also moves v by only 0.1% of the gap between g_t^2 and v_{t-1}. Added in bf16, such changes round away. The Mixed Precision tab works through this rounding, and it names a lab that keeps m and v in bf16 anyway.

The course trainer keeps its parameters in fp32 and runs the forward pass under autocast, which casts the operands of each matrix multiplication to bf16 or fp16 as it goes. Its own account therefore has no separate stored bf16 weight copy: the fp32 parameters are the weights.

In this lecture, optimizer state means the optimizer's own buffers, here m and v. Like them, the master weights are read and written only by the optimizer step, so ZeRO, the sharding scheme

Week 6 builds, counts them as optimizer state: $4 + 4 + 4 = 12$ bytes of the mixed-precision account's 16. The two counts differ only in where they book the master copy.

This week's second optimizer, Muon, applies to the hidden weight matrices and keeps one momentum buffer for each, while the remaining parameters stay with AdamW. The share of parameters that Muon can take depends on the architecture, so keep the hybrid account symbolic. With P_{Muon} the number of parameters in Muon's matrices and P_{AdamW} the number in the remaining AdamW groups, the fp32 optimizer state is

$$4P_{\text{Muon}} + 8P_{\text{AdamW}} \text{ bytes :}$$

one 4 B momentum buffer for the matrix group, and two 4 B moments for the rest. Week 6 shards these buffers across devices, so that each device holds only a part of them.

AdamW has been hard to replace Read on your own

- Schmidt et al.: no consistent winner over Adam (Schmidt et al., 2021)
- AlgoPerf: winners built on Shampoo, Schedule-Free AdamW (Kasimbeg et al., 2025)
- Wen et al.: speedups shrink with scale (Wen et al., 2026)
- Smol Training Playbook: tried newer optimizers, then shipped AdamW (Ben Allal et al., 2025)

▼ **Benchmarks rarely beat a tuned AdamW**

Many optimizers have been proposed as replacements for AdamW, and the benchmarks that compare them have found it hard to beat. Schmidt et al. benchmarked fifteen optimizers on eight deep-learning problems, in over 50,000 runs, and found that none consistently beats a well-tuned Adam (Schmidt et al., 2021). The AlgoPerf competition put training algorithms on a common budget (Dahl et al., 2023). In its results, the winner with external tuning used Distributed Shampoo, and the winner that tuned itself was based on Schedule-Free AdamW (Kasimbeg et al., 2025). Schedule-Free is AdamW with a different scheme for the step size (Defazio et al., 2024). Wen et al. studied ten optimizers on models from 0.1B to 1.2B parameters. Under matched hyperparameter tuning, the speedup of the matrix-based optimizers over a well-tuned AdamW fell from 1.4× at 0.1B to 1.1× at 1.2B (Wen et al., 2026). The Smol Training Playbook team tried the newer methods and shipped AdamW (Ben Allal et al., 2025).

Schmidt et al. and Wen et al. compare optimizers under controlled conditions, within named tasks, scales, budgets and tuning protocols. Schmidt et al. show why tuning the baseline matters, and Wen et al. that a ranking or a speedup can change with scale and with the checkpoint you compare at. The Smol team's choice is an engineering decision for its model.

Muon

14

Muon starts from matrix-shaped updates

- Hidden weights: 2D matrices, rows and columns
- Flattening throws that structure away
- **Muon** treats momentum as a matrix
- Keller Jordan's blog, December 2024 (Jordan et al., 2024)

▼ **Muon treats the update as a matrix**

AdamW adapts each coordinate on its own. It treats a weight matrix as a long list of numbers, and shuffling the entries before the step would not change its update. Most of a transformer's parameters, however, are in the hidden layers' 2D weight matrices: the attention projections and the MLP matrices. A matrix $W \in \mathbb{R}^{m \times n}$ maps an input vector x of length n to an output vector $y = Wx$ of length m, so its rows belong to output features and its columns to input features. Flattening the update into a vector throws that row and column structure away. **Muon** starts from the other view. It keeps the momentum of a hidden weight as a matrix and decides how far the update moves along each of the matrix's directions.

Muon changes what happens after the momentum is formed. It first accumulates the gradients into a momentum buffer, as SGD with momentum does, and then reshapes that buffer to a 2D matrix and changes its singular values.

Keller Jordan introduced Muon in a blog post in December 2024 (Jordan et al., 2024). It came out of speedrun experiments: a small model trained under a fixed budget, with the training code public. That setting makes it quick to compare optimizers on equal terms, which is why the speedrun works as a proving ground for them.

15

How many singular values hold 90%?

Momentum M ← 0.95·M + G, 24 × 24, 50 steps.
G = 0.7 · same rank-3 signal + 0.3 · fresh noise
90% of the sum of all 24: about 3, 7, 16 or 24?

Answer A1

One global step size starves the quiet directions

- Real momentum updates: effectively low-rank (Jordan et al., 2024)
- One scale: loud directions dominate
- Wanted: an equal step per direction

▼ Why a concentrated update needs more than one scale

The same concentration appears in real networks. Momentum, $M_t = \beta M_{t-1} + G_t$ with $\beta = 0.95$, is a running sum of past gradients with geometrically decaying weights, an EMA up to a constant factor. A hidden layer receives correlated gradients, because the same input and output patterns push its weights step after step. So the same accumulation concentrates its update. Jordan et al. report that the updates of SGD with momentum and of Adam for the 2D parameters of transformers typically have a very high condition number, the ratio of the largest singular value to the smallest. The updates are almost low-rank matrices (Jordan et al., 2024).

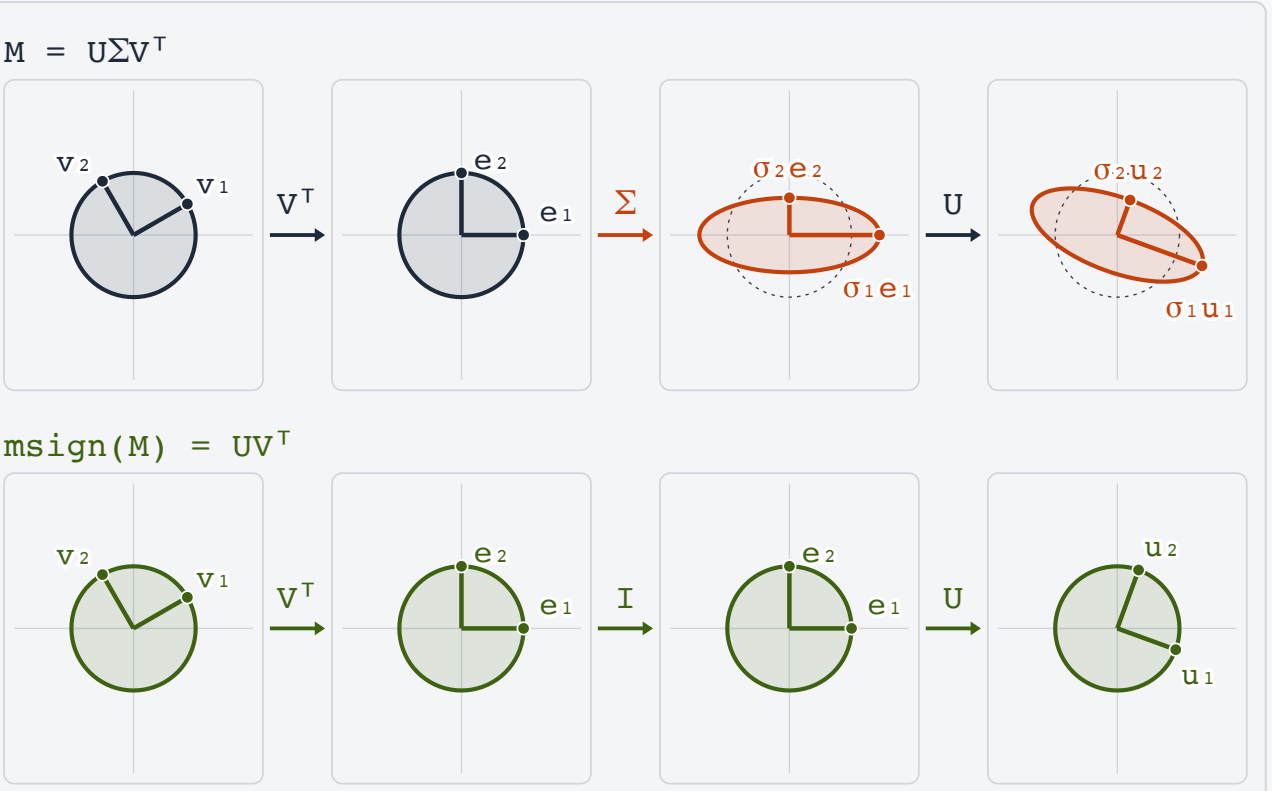
Any optimizer that scales the whole update by one number, such as normalizing the update matrix by its norm, keeps the ratios between its singular values. The few loud directions take almost all of the step, and the many quiet directions barely move. What we want is a step of equal size in every direction that the update contains.

Muon equalizes the update's singular directions

(4) $$M = U\Sigma V^\top \qquad \operatorname{msign}(M) = UV^\top$$

(4a) $$M_t = \beta M_{t-1} + G_t \qquad W_t = W_{t-1} - \eta \operatorname{msign}(M_t)$$

- Singular value decomposition (SVD): rotations and gains
- msign keeps U and V, sets every σ to 1
- Orthogonalization: an equal step per direction
- Muon never computes the SVD itself



Read each row from left to right, then compare the rows panel by panel. They differ only in the middle step, where Σ stretches the circle and I leaves it alone. The dashed circle is the unit circle.

▼ The SVD and the matrix sign

Muon's step has three parts, formula (4a): add the new gradient G_t to the momentum M_t, replace the momentum matrix by its matrix sign, and step against that with learning rate η. The full update also multiplies by a scale factor that depends on the matrix's shape, explained with Muon's parameter groups on the slide "Muon targets matrices; AdamW handles the rest". Muon keeps one buffer, the momentum, and no second moment: the matrix sign takes over the role that Adam's $\sqrt{\hat{v}_t}$ plays.

The matrix sign is defined through the singular value decomposition (SVD). Let the momentum matrix be $M \in \mathbb{R}^{m\times n}$ with rank r. Its compact SVD is

$$M = U\Sigma V^\top = \sum_{i=1}^{r} \sigma_i \, u_i v_i^\top,$$

where U is $m \times r$, Σ is $r \times r$ and V is $n \times r$. The columns of U are orthonormal, and so are the columns of V. The diagonal of Σ holds the positive singular values $\sigma_1 \geq \cdots \geq \sigma_r$. In the sum, the input direction v_i maps to the output direction u_i with gain σ_i.

The matrix sign replaces each nonzero gain by one and keeps both singular directions. Formula (4) defines it as $\operatorname{msign}(M) = UV^\top$, again an $m \times n$ matrix (Jordan et al., 2024). So M and $UV^\top$ act on the same row and column subspaces, but all singular values of $UV^\top$ equal one. A direction with zero singular value is not in the sum, and the matrix sign does not invent it. Every direction in the update's span then takes a step of the same size. Gradient normalization sets one scale per matrix, the matrix sign one per direction.

The figure shows both maps in two dimensions. Read the top row from left to right: $V^\top$ rotates the unit circle, Σ stretches it along the axes into an ellipse with semi-axes σ_1 and σ_2, and U rotates the ellipse. That ellipse is what M makes of the unit circle. The bottom row applies $UV^\top$ instead: the same two rotations, with the identity I in place of Σ, so the circle stays a circle and both directions keep length one. The two rows differ only in the middle step. For a full-rank M, $UV^\top$ is the matrix with orthonormal rows or columns closest to M in the Frobenius norm, so the matrix sign keeps M's directions and drops its gains. Replacing Σ by the identity is called **orthogonalizing** the update, and it gives the optimizer its name: Muon stands for MomentUm Orthogonalized by Newton–Schulz (Jordan et al., 2024).

The SVD defines the target, but Muon never computes one. An SVD of every hidden matrix at every step would cost too much, so Muon reaches $UV^\top$ approximately, with matrix multiplications only, and never forms U, Σ or V on the way.

Claim. If the momentum matrix is wide with full row rank ($m < n$, so $r = m$), the rows of $UV^\top$ are orthonormal:

$$\begin{aligned} (UV^\top)(UV^\top)^\top &= UV^\top VU^\top && \text{transpose of a product} \\ &= UU^\top && V^\top V = I_r \\ &= I_m && U \text{ square, as } r = m. \end{aligned}$$

A tall matrix of full column rank ($m > n$) gets orthonormal columns the same way, $(UV^\top)^\top(UV^\top) = I_n$. A rank-deficient matrix satisfies these identities only on its rank-r span. Orthonormal rows or columns are what "orthogonalizing" refers to.

Different optimizers bound different kinds of steps Read on your own

- msign is steepest descent under the spectral norm
- A spectral bound limits the largest change in amplification
- Frobenius keeps the relative sizes of the singular values
- Adam without EMA is sign descent, under the ℓ∞ norm (Bernstein & Newhouse, 2024)

Optimizers are choices of norm

▼ Steepest descent under three norms

An optimizer step has to decide what counts as a large update, and a norm on the step makes that decision. The matrix sign is the steepest-descent step under the spectral norm. Bernstein and Newhouse show that, with their exponential moving averages switched off, Adam is sign descent, which is steepest descent under the vector ℓ_∞ norm, and Shampoo, a preconditioned optimizer that keeps a left and a right matrix preconditioner, is steepest descent under the spectral norm (Bernstein & Newhouse, 2024). Optimizers are choices of norm.

The local problem

Let G be the gradient of the loss with respect to a weight matrix, and Δ the step, both $m \times n$.

- **A1.** To first order, a step Δ changes the loss by the Frobenius inner product $\langle G, \Delta \rangle_F = \sum_{j,k} G_{jk} \Delta_{jk}$.

Steepest descent chooses the step that lowers this most among all steps of size at most η in a chosen norm:

$$\Delta^* = \arg\min_{\|\Delta\| \le \eta} \langle G, \Delta \rangle_F. \tag{4b}$$

The norm changes the direction of the best step, not just its length.

Claim (Frobenius). Under A1, the solution of (4b) with $\|\Delta\|_F \le \eta$ is $\Delta^* = -\eta\, G / \|G\|_F$:

$$\begin{aligned} \langle G, \Delta \rangle_F &\ge -\|G\|_F \, \|\Delta\|_F && \text{Cauchy–Schwarz} \\ &\ge -\eta \, \|G\|_F && \|\Delta\|_F \le \eta, \end{aligned}$$

with equality when Δ points opposite to G and has norm η. This is normalized gradient descent. Its singular values stay proportional to those of G, and all directions share one budget

for the sum of their squared sizes.

Claim (coordinates). Under A1, the solution of (4b) under the coordinate ℓ_∞ bound $\max_{j,k}|\Delta_{jk}| \le \eta$ is $\Delta^* = -\eta\,\mathrm{sign}(G)$. Each coordinate can be chosen on its own, and each term $G_{jk}\Delta_{jk}$ is smallest at $\Delta_{jk} = -\eta\,\mathrm{sign}(G_{jk})$. Every coordinate with a nonzero gradient moves to the edge of the box. Adam reduces to this rule only with the EMA switched off ($\beta_1 = \beta_2 = 0$) and with ϵ small next to $|g_t|$, since then $\hat{m}_t/\sqrt{\hat{v}_t} = g_t/|g_t|$. Ordinary Adam, with its stored moments, is outside that equivalence.

Claim (spectral). Under A1, the solution of (4b) under the spectral norm is $\Delta^* = -\eta\,\mathrm{msign}(G)$. The spectral norm $\|\Delta\|_2$ is the largest singular value of Δ, the most that Δ can stretch a unit vector. So for unit vectors u and v, the bilinear form satisfies $|u^\top \Delta v| \le \|\Delta\|_2$. With the compact SVD $G = \sum_i \sigma_i u_i v_i^\top$,

$$
\begin{aligned}
\langle G, \Delta\rangle_F = \sum_i \sigma_i\, u_i^\top \Delta v_i \qquad & \langle uv^\top, \Delta\rangle_F = u^\top \Delta v \\
\ge -\sum_i \sigma_i\, \|\Delta\|_2 \qquad & |u_i^\top \Delta v_i| \le \|\Delta\|_2 \\
\ge -\eta \sum_i \sigma_i \qquad & \|\Delta\|_2 \le \eta.
\end{aligned}
$$

The step $\Delta^* = -\eta UV^\top$ reaches this bound. All its singular values equal η, so its spectral norm is η, and $u_i^\top \Delta^* v_i = -\eta$ for every i. Every active singular value of the step reaches the budget.

What each bound controls

A linear layer computes $y = Wx$. If the weight changes by Δ, the output changes by Δx, and $\|\Delta x\|_2 \le \|\Delta\|_2\,\|x\|_2$. A bound on the spectral norm therefore limits directly the largest change in how much the layer amplifies any input. Because $\|\Delta\|_2 \le \|\Delta\|_F$, a Frobenius bound limits that amplification change too.

What the derivation does not show

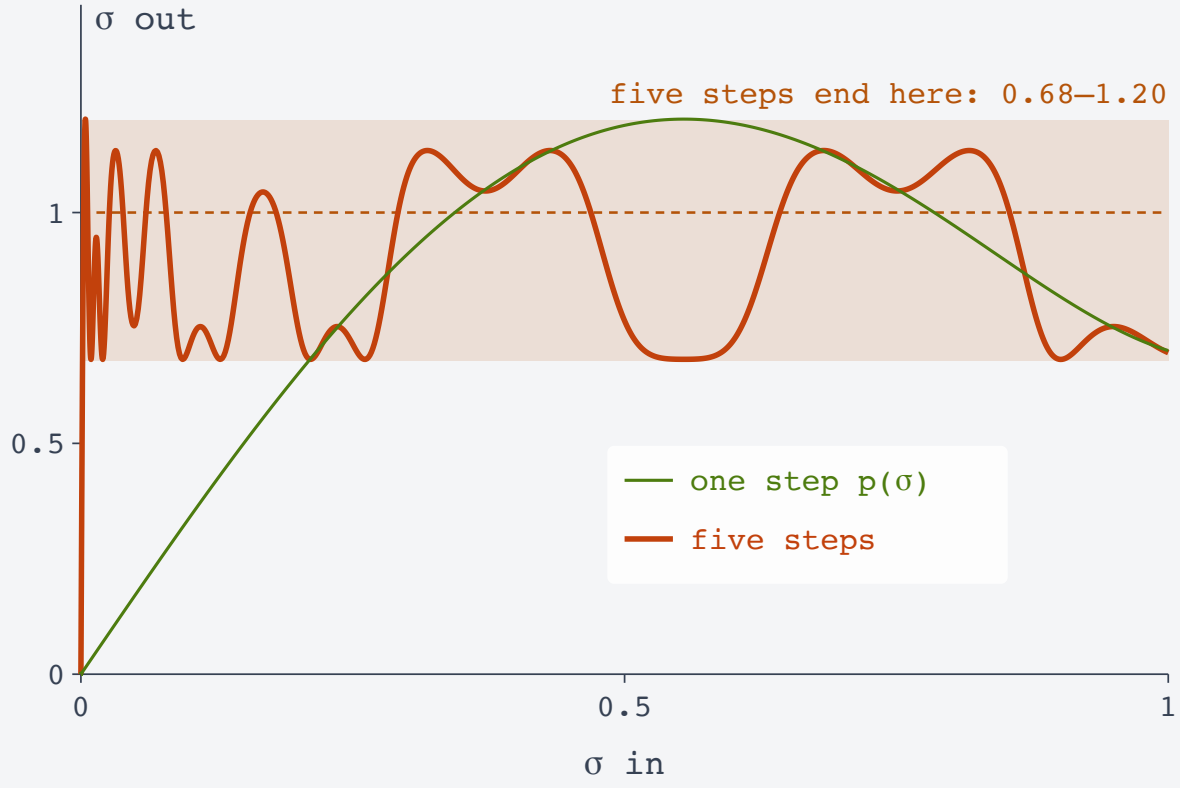
The norm view derives Muon's update, but it may not be why Muon trains well. Shumaylov et al. replaced the unit singular values of Muon's update with pseudo-random values between 0 and 1, or with powers of the singular values that reverse their order, so that neither update is steepest descent under any norm that depends only on the singular values. On a 124M-parameter GPT-2 both reached about Muon's validation loss, while SGD with only its largest singular values removed did not (Shumaylov et al., 2026). The runs are small and include no comparison with AdamW, so they bear on why Muon works, not on how much it gains.

Newton-Schulz makes the matrix sign practical

$$X_{k+1} = a\,X_k + b\,(X_k X_k^\top)\,X_k + c\,(X_k X_k^\top)^2\,X_k \tag{5}$$

- A full SVD every step: too expensive
- Newton–Schulz: matrix multiplications only, fast on GPUs
- Each step maps every σ to p(σ)
- Five steps: every σ above 0.002 ends near 1

p(σ) = 3.4445σ - 4.7750σ³ + 2.0315σ⁵



Read each curve at an input σ on the horizontal axis. Below about 0.0015 the five-step output stays under 0.68: near 0, five steps multiply σ by only about $a^5 \approx 485$, so σ = 0.001 ends near 0.47.

▼ Running the iteration on one matrix

Computing an SVD for every layer at every step would cost too much. Muon approximates the matrix sign with a Newton–Schulz iteration instead, formula (5), using the coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$ for five iterations (Jordan et al., 2024). The coefficients are tuned for a fast, approximate orthogonalization, which is all the update needs.

Reshape the momentum of one eligible weight to a 2D matrix X. If it is tall, transpose it, so that $XX^\top$ is the smaller Gram matrix, and transpose the result back at the end. Transposing leaves the singular values unchanged. Then normalize,

$$X_0 = \frac{X}{\|X\|_F + \epsilon_{\text{NS}}}, \qquad \epsilon_{\text{NS}} = 10^{-7}.$$

Since the largest singular value is at most the Frobenius norm, every singular value of X_0 lies between 0 and 1. Then apply formula (5) five times.

Each iteration costs three matrix multiplications: the Gram matrix $A = X_k X_k^\top$, then $A^2 = AA$, then $(bA + cA^2)\, X_k$. Scaling by a and adding the terms are elementwise operations. Five iterations are therefore 15 matrix multiplications, unless an implementation fuses some of them. Because the iteration consists of matrix multiplications only, it runs on the GPU's tensor cores.

Formula (5) multiplies X_k only by powers of $X_k X_k^\top$, which share X_k's singular directions. So each step keeps U and V and maps every singular value σ to the same odd polynomial,

$$p(\sigma) = 3.4445\,\sigma - 4.7750\,\sigma^3 + 2.0315\,\sigma^5,$$

and five steps apply p five times. The plot shows both maps for inputs between 0 and 1: one step $p(\sigma)$, and five steps $p(p(p(p(p(\sigma)))))$. Nothing in it is random. Every singular value from about 0.002 up to 1 ends between 0.68 and 1.20 after five steps, inside the shaded band. Where in the band a value lands jumps back and forth as the input changes, which is the wiggle of the five-step curve, but it never leaves the band. Values much smaller than 0.002 stay below it: near zero one step multiplies σ by $a = 3.4445$, five steps by $a^5 \approx 485$. Muon does not need the singular values to equal one exactly. Values between 0.7 and 1.2 already give every direction a step of about the same size.

The blog runs the iteration in bf16 and reports that this is stable (Jordan et al., 2024). The course runs it in fp32: the course's hidden matrices are small, at most $512 \times 1{,}536$, and fp32 keeps the update independent of the training precision. The blog also estimates that Muon adds less than 1% to the FLOPs of typical language-model training (Jordan et al., 2024). Time is measured separately: in the distributed implementation of Moonlight, Liu et al.'s Muon-trained mixture-of-experts model and scaling study, the optimizer usually takes 1% to 3% of the time of the forward and backward pass (Liu et al., 2025).

Five Newton-Schulz steps flatten the spectrum Read on your own

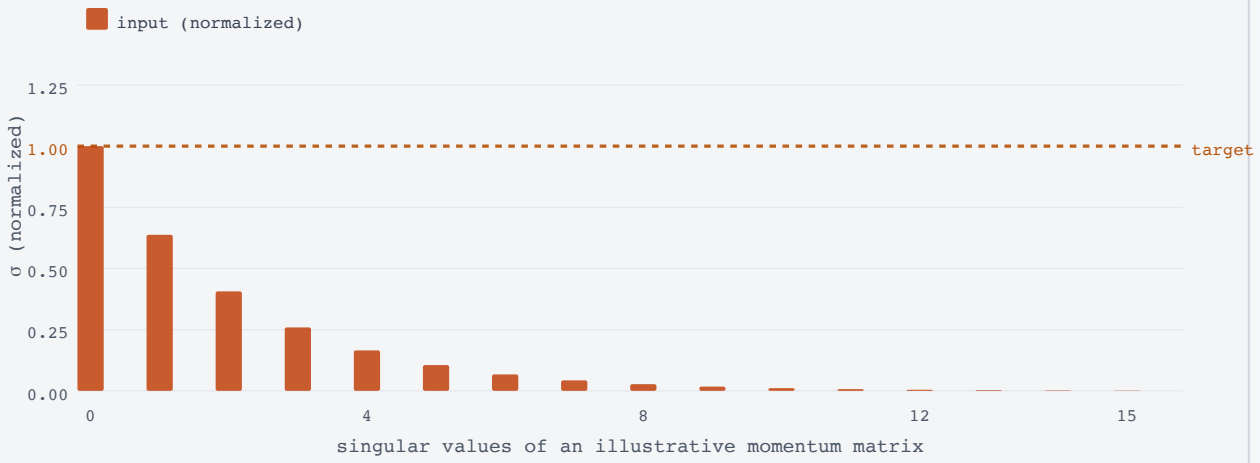
- Bars: an illustrative spectrum, largest σ = 1
- Each step applies one polynomial to every σ
- After five steps: a band around 1

ILLUSTRATIVE Polynomial: p(s) = 3.4445s - 4.775s³ + 2.0315s⁵

Iteration 0

Target: msign (all σ → 1)

σ within 30% of 1: 1/16



← Prev | Next → | Reset | 0 / 5 steps

σ within 30% of 1: 1/16 (6%)

Each bar is one singular value σ of an illustrative momentum matrix. The input spectrum, $\sigma_i = e^{-0.45i}$, is illustrative. The dashed line at σ = 1 is the target. Newton–Schulz pushes the values toward 1 without reaching it: after five steps they lie between 0.54 and 1.19, and 13 of 16 are within 30% of 1. The iteration p(s) = 3.4445s − 4.775s³ + 2.0315s⁵ is the quintic from the Muon blog post (Jordan, 2024).

▼ Why an approximate matrix sign is good enough

Claim. The iteration acts on each singular value separately: it maps σ to

$$p(\sigma) = a\sigma + b\sigma^3 + c\sigma^5 \tag{5a}$$

and leaves the singular directions unchanged.

Write the working matrix as $X_k = U\Sigma V^\top$. The powers of the Gram matrix keep the same U and V:

$$
\begin{aligned}
X_k X_k^\top &= U\Sigma V^\top V \Sigma U^\top = U\Sigma^2 U^\top && V^\top V = I \\
(X_k X_k^\top)\, X_k &= U\Sigma^2 U^\top U \Sigma V^\top = U\Sigma^3 V^\top && U^\top U = I \\
(X_k X_k^\top)^2 X_k &= U\Sigma^5 V^\top && \text{the same two identities.}
\end{aligned}
$$

Formula (5) therefore becomes

$$X_{k+1} = U\,(a\Sigma + b\Sigma^3 + c\Sigma^5)\,V^\top = U\,p(\Sigma)\,V^\top.$$

The widget applies the map (5a) to a list of singular values.

Three properties of this quintic explain what the iteration does. It is odd, so $p(0) = 0$: a singular value that is exactly zero stays zero at every iteration, and the iteration cannot create a missing direction. Its slope at zero is $p'(0) = a = 3.4445$, so a small singular value grows by about that factor per step. And $p(1) = 3.4445 - 4.7750 + 2.0315 = 0.701$, so 1 is not a fixed point. The tuned coefficients trade exactness for speed: they lift small values fast, and the values end up in a band around 1 instead of at 1.

In the widget, the bars are an illustrative spectrum of 16 values, scaled so that the largest is 1. The widget scales by the largest singular value, while the implementation divides by the Frobenius norm. This is a simplification for display. After one step the largest value drops to about 0.70, since $p(1) = 0.701$, while the next two, 0.64 and 0.41, overshoot past 1 to about 1.17 and 1.10.

A value that starts at 10^{-3} grows by about a per step while it is small, to roughly $3.4445^5 \times 10^{-3} \approx 0.48$ after five steps, so a much smaller nonzero value stays far from 1.

So five steps give approximately orthonormal rows only under conditions. The wide working matrix must have full row rank, and its nonzero singular values must start large enough for five steps to bring them near 1. For an originally tall matrix of full column rank, transposing back gives approximately orthonormal columns.

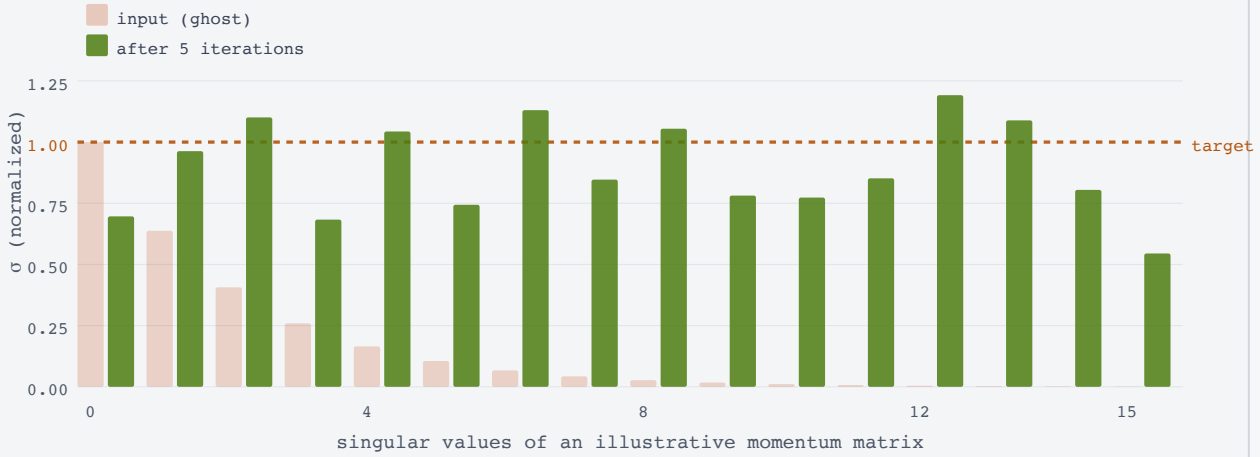
The iteration needs a matrix. The optimizer race earlier in the lecture moves a parameter with two coordinates, a vector, so Muon has nothing to orthogonalize there.

ILLUSTRATIVE Polynomial: $p(s) = 3.4445s - 4.775s^3 + 2.0315s^5$

Iteration 5

Target: msign (all σ → 1)

σ within 30% of 1: 13/16



← Prev | Next → | Reset | 5 / 5 steps

σ within 30% of 1: **13/16 (81%)**

Each bar is one singular value σ of an illustrative momentum matrix. The input spectrum, $\sigma_i = e^{-0.45i}$, is illustrative. The dashed line at σ = 1 is the target. Newton–Schulz pushes the values toward 1 without reaching it: after five steps they lie between 0.54 and 1.19, and 13 of 16 are within 30% of 1. Ghost bars show the normalized input; lime bars show the values after 5 Newton–Schulz steps. The iteration $p(s) = 3.4445s - 4.775s^3 + 2.0315s^5$ is the quintic from the Muon blog post (Jordan, 2024).

Iteration 5: 13 of the 16 values are now within 30% of 1, against 1 of 16 in the input, and all lie between 0.54 and 1.19. They keep oscillating around 1, so the largest input value ends at about 0.70. The smallest grew from about 0.001 to 0.54.

Muon targets matrices; AdamW handles the rest

```
def newton_schulz5(G, steps=5):            # approximate msign, matmuls only
    a, b, c = (3.4445, -4.7750, 2.0315)
    X = G.bfloat16() / (G.norm() + 1e-7)   # into [0, 1]; the course stays fp32
    for _ in range(steps):
        A = X @ X.T
        X = a * X + (b * A + c * A @ A) @ X
    return X

def muon_update(grad, buf, beta=0.95):     # one hidden weight matrix
    buf.mul_(beta).add_(grad)              # momentum buffer
    g = grad + beta * buf                  # Nesterov-style blend
    return newton_schulz5(g.reshape(len(g), -1)).view_as(g)

W -= lr * muon_update(W.grad, buf)         # LR-scaled step
```

$$0.2\sqrt{\max(\text{fan_in}, \text{fan_out})}$$

- Hidden 2D matrices: Muon
- Embeddings, head, norms, scalars: AdamW
- One momentum buffer, no v: half of AdamW's state
- Update rescaled to AdamW's root mean square (RMS) (Liu et al., 2025)

▼ From the blog's code to the course's optimizer

The code is Muon's core, as the blog states it (Jordan et al., 2024). It is the algorithm you implement this week. Read it as a trace of five operations, in order:

1. `buf.mul_(beta).add_(grad)` updates the momentum buffer, $M_t = \beta M_{t-1} + G_t$.
2. `grad + beta * buf` forms the Nesterov-style blend of the gradient and the buffer, $G_t + \beta M_t$.
3. `reshape(len(g), -1)` makes the update a 2D matrix.
4. `newton_schulz5(...)` applies the matrix-sign approximation in bf16, giving $O_t \approx \text{msign}(G_t + \beta M_t)$.
5. `W -= lr * ...` performs the parameter update, scaled by the learning rate.

Which parameters take Muon. The blog applies Muon to the hidden 2D weight matrices. The embeddings, the output head (the unembedding), vectors and scalars stay with **AdamW** (Jordan et al., 2024). The two groups also differ in state. A Muon matrix keeps one momentum buffer and no second moment v_t, while an AdamW parameter keeps the two moments m_t and

v_t. In fp32 that is 4 B of optimizer state per parameter for a Muon matrix against 8 B for an AdamW parameter, so Muon halves the state of the parameters it takes.

The update's scale. The orthogonalized update has its singular values near 1, so its size is set by the matrix's shape, and its root mean square (RMS), the square root of the mean of its squared entries, no longer matches AdamW's. Moonlight rescales each Muon update by $0.2\sqrt{\max(\text{fan_in}, \text{fan_out})}$, where `fan_in` and `fan_out` are the weight's input and output widths, its n columns and m rows. This matches the update's RMS to AdamW's, so that the learning rate and weight decay tuned for AdamW carry over to Muon (Liu et al., 2025). The match is a heuristic for transferring hyperparameters, and it needs checking on the target model.

What the course adds. The code on the slide leaves out the tall-matrix transpose, which the blog's published implementation has. Your `Muon` keeps the transpose and adds three things to the blog's method: Moonlight's rescale, decoupled weight decay as in AdamW, and Newton–Schulz in fp32. Both groups share one learning rate and one weight decay. The course groups parameters by their role, not their shape. The token embedding and the output head take AdamW although both are 2D. Every other 2D parameter takes Muon, and that includes the bottleneck matrices of latent attention (MLA) from Week 4. The remaining parameters, such as the norm weights, take AdamW.

Four practices keep Muon stable at scale

- Scale each update by its matrix shape
- Keep weight decay: weights grow otherwise
- Treat each head and each expert as its own matrix
- Watch the max logit, and bound it

▼ What labs changed to train Muon stably at scale

Four practices recur in the reports of labs that train large models with Muon.

Scale each update by the matrix's shape. Moonlight proves, as its Lemma 1, that the Muon update O_t of a full-rank $m \times n$ matrix has an RMS of $\sqrt{1/\max(m,n)}$ (Liu et al., 2025). The lemma follows because a full-rank O_t has $\min(m,n)$ singular values, all equal to one, and its squared Frobenius norm is the sum of their squares:

$$\mathrm{RMS}(O_t) = \frac{\|O_t\|_F}{\sqrt{mn}} = \sqrt{\frac{\min(m,n)}{mn}} = \sqrt{\frac{1}{\max(m,n)}}.$$

The size of the orthogonalized update therefore depends on the matrix's shape alone, and Moonlight names two ways this goes wrong. For a large matrix, such as a dense MLP matrix, $\max(m,n)$ is large and the update too small, which limits what the model can learn. For a small matrix, such as one KV head of GQA or MLA treated as a parameter of its own, the update is too large, and training becomes unstable (Liu et al., 2025). Multiplying each update by $\sqrt{\max(m,n)}$ cancels the shape, and a factor γ_{RMS} then sets one common update RMS for every matrix. The step becomes $\eta\,\gamma_{\mathrm{RMS}}\sqrt{\max(m,n)}\,O_t$. With $\gamma_{\mathrm{RMS}} = 0.2$ this is the rescale $0.2\sqrt{\max(\mathrm{fan_in}, \mathrm{fan_out})}$ of the course's optimizer. Moonlight picked 0.2 because AdamW's update RMS is usually 0.2 to 0.4 (Liu et al., 2025). DeepSeek V4 applies the same rule with $\gamma_{\mathrm{RMS}} = 0.18$, so that it can reuse its AdamW learning rate (DeepSeek-AI, 2026a). Laguna M.1 reports what can happen without the rescale. In preliminary runs, Muon without Moonlight's rescale and with standard weight decay "caused expert collapse beginning at around 450B tokens into training and propagating layer-by-layer through the first three MoE layers until the training diverged" (Poolside Team et al., 2026).

Keep weight decay. Jordan's original Muon has no weight decay. Moonlight found that Muon's advantage over AdamW shrank as it trained larger models on more tokens. The RMS of the weights and of the layer outputs kept growing, past the range that bf16 represents with high precision, which Moonlight says might hurt the model's performance. On an 800M-parameter model trained on 100B tokens, Muon with AdamW's decoupled weight decay reached a lower validation loss than both Muon without it and AdamW (Liu et al., 2025).

Decide what counts as one matrix. Muon equalizes the singular values of whatever matrix it receives, so the choice of matrix decides which directions share one equalized step. Jordan et al. report that Muon trains transformers better when it is applied to the Q, K and V weights separately than to one fused QKV matrix whose outputs are split (Jordan et al., 2024). Kimi K3 goes further. It splits the momentum of the Q, K and V projections by head and orthogonalizes each head's block on its own, because with the full matrix the heads with larger gradients dominate the shared update and the smaller ones are not normalized enough. K3 reports more balanced learning across heads and better training stability at larger scales (Kimi Team, 2026). DeepSeek-V4.1-Flash also splits its query and key weights by head and reports that head-wise Muon "outperforms vanilla Muon" (DeepSeek-AI, 2026b). GLM-5 applies the same split, "Muon Split", to the up-projections of MLA's queries, keys and values, and with it MLA matched grouped-query attention, which it had trailed (GLM Team, 2026). DeepSeek V4 optimizes the matrices of each MoE expert independently (DeepSeek-AI, 2026a). Splitting makes each matrix smaller, the situation of Moonlight's second failure case, where without the shape rescale a small matrix gets too large an update (Liu et al., 2025). The rescale gives every matrix's update the same RMS.

Watch the largest attention logit, and bound it. K2's 53B test run lost control of this quantity. K2 bounds it with QK-Clip.

Where labs differ. Labs agree on the large matrices and differ at the edges. DeepSeek V4 keeps the embedding, the prediction head, the norm weights, the static biases and gating factors of its manifold-constrained hyper-connection (mHC) modules on AdamW, and the remaining matrices, the MoE router included, take Muon (DeepSeek-AI, 2026a). Qwen3.8-Flash-Next keeps its router on AdamW, because Muon made early training fluctuate and destabilized the router (Qwen Team, 2026). DeepSeek V4.1-Flash moves even the embeddings and the prediction head off AdamW, to a momentum update that needs no second moment (DeepSeek-AI, 2026b).

Iterations, precision and sharding also matter Read on your own

- Five steps for Moonlight, 8 to 10 in newer runs (Liu et al., 2025) (Qwen Team, 2026) (DeepSeek-AI, 2026a)
- bf16 usually works, but rare outliers can break it
- Distributed Muon gathers each full matrix
- MiMo, Mistral switch to Muon after AdamW pretraining (Xiaomi LLM-Core Team, 2026) (Mistral AI, 2026)

▼ **Iteration count, precision, sharding and fine-tuning in reported Muon practice**

How many iterations. Moonlight observed that ten Newton–Schulz iterations orthogonalize more accurately than five but do not train better, so it kept five for efficiency (Liu et al., 2025). Newer reports run more than five. Qwen3.8-Flash-Next runs eight Newton–Schulz steps with a different set of coefficients, because eight "reduces both the magnitude and frequency of gradient-norm spikes in our stress test" (Qwen Team, 2026). DeepSeek V4 runs ten in two stages. The first eight use Jordan's coefficients $(3.4445, -4.7750, 2.0315)$ and bring the singular values close to 1 quickly. The last two use $(a, b, c) = (2, -1.5, 0.5)$, which in V4's words "stabilize the singular values precisely at 1" (DeepSeek-AI, 2026a). For these coefficients the map (5a) has $p(1) = 2 - 1.5 + 0.5 = 1$ and slope $p'(1) = 2 - 4.5 + 2.5 = 0$ at 1. So 1 is a fixed point, and values near it close in on it fast. Jordan's quintic makes the opposite trade: it grows small values fast and has $p(1) = 0.701$.

Which precision. DeepSeek V4, like the blog, observes that the iterations stay stable with bf16 matrix multiplications (DeepSeek-AI, 2026a). Step 3.5 Flash reports an exception in one run: with its own faster-converging iteration it occasionally saw loss spikes the run did not recover from. StepFun's simulations indicate that this iteration in bf16 can rarely produce extreme outliers through error accumulated in additions, and running only the iteration in fp16 stopped the spikes (Huang et al., 2026).

Sharding across devices. Newton–Schulz needs a weight's full gradient matrix, while a distributed optimizer gives each device only a piece of it. Moonlight's Distributed Muon gathers each matrix before the iteration, the implementation behind its 1% to 3% of forward-backward time (Liu et al., 2025). DeepSeek V4 instead assigns whole matrices to devices (DeepSeek-AI, 2026a). Week 6 builds distributed training.

Fine-tuning. In Moonlight's runs, Muon fine-tuning after Muon pretraining performed best, and with mismatched optimizers Muon showed no significant advantage over AdamW (Liu et al., 2025). Kimi K2 uses Muon in post-training and recommends it for fine-tuning K2, citing that finding (Kimi Team, 2025). Laguna keeps Muon in all its training stages, supervised fine-tuning and reinforcement learning included (Poolside Team et al., 2026). Two 2026 models pretrain with AdamW and switch later. MiMo-V2.6 moves its hidden matrices to a Muon variant at mid-training, because AdamW's efficiency fell as the batch of its reinforcement-learning stage grew, and it reports no loss spike after the switch

(Xiaomi LLM-Core Team, 2026). Mistral Large 4 runs its reinforcement-learning stage with Muon and does not say why (Mistral AI, 2026). Week 11 covers fine-tuning.

24

Half of AdamW's compute?

A paper reports this for its new optimizer.
What would you check before you switch?

How strong is the evidence for Muon?

- Speedrun: GPT-2 small on FineWeb, fewest steps to loss 3.28
- Muon: mean of 20 runs, AdamW: one undertuned run
- Moonlight: matched AdamW with about half its compute
- Wen et al.: speedup 1.4× at 0.1B, 1.1× at 1.2B

Keller Jordan's modded-nanoGPT optimizer speedrun: validation loss versus wall-clock time



Benchmark rule	Muon	AdamW
loss < 3.28 and (3.28 − mean)$\sqrt{n}$ ≥ 0.004 · score: training steps	3,375 steps · 3.27893 mean loss · 495.817 s mean · n=20 · clears the record rule	5,625 steps · 3.27903 loss · 833.211 s · n=1 · reaches loss below 3.28 but fails the significance test

Muon's line averages validation loss and elapsed time over repeated runs at each checkpoint. AdamW's line is one recorded run. Both come from Keller Jordan's track-3 logs.

▼ Six checks for a claimed speedup, applied to Muon

A paper that reports half of AdamW's compute for its new optimizer makes a claim about a comparison, so check the comparison. The standard is a well-tuned AdamW baseline: a challenger must beat it across useful scales and checkpoints, including the challenger's own overhead. Six checks follow from it.

- **Matched budget.** Compare at the same training-token, compute or wall-clock budget, and include the challenger's overhead. AlgoPerf fixes such a common budget (Dahl et al., 2023).
- **Tuning.** Give AdamW and the challenger comparable search effort and the same rule for picking checkpoints. The recurring failure mode is unfair baselines: a new optimizer, tuned hard, is compared against an untuned AdamW baseline, and under matched tuning the claimed speedup often shrinks or vanishes. Schmidt et al.'s benchmark shows why the tuning of the baseline matters (Schmidt et al., 2021).
- **Stability.** Compare full-run failures, the variation across repeats, and the behavior beyond an early favorable checkpoint.
- **Memory.** Measure each optimizer's peak allocated memory on your system under the same implementation, including any temporary workspace the optimizer needs.

- **Downstream quality.** Compare held-out and downstream quality at the matched endpoint as well as the training loss.
- **Replication.** Count the runs behind each endpoint, and ask whether the gap exceeds the run-to-run noise.

Scale belongs with the budget: a gain at one model size may not hold at yours.

The plot shows two record logs from Keller Jordan's modded-nanoGPT track 3, the optimizer speedrun on the slide (Jordan et al., 2026). In this track every entry trains the same model on the same data: a GPT with 12 layers and width 768, the shape of GPT-2 small, on the FineWeb dataset, with 1,024-token sequences and 524,288 tokens per step. Entrants may change only the optimizer, its hyperparameters and schedules, and the initialization. Track 3 minimizes the number of training steps, subject to a validation loss below 3.28 and a significance rule that accounts for replication. The rule computes

$$\text{score} = (3.28 - \text{mean endpoint validation loss}) \times \sqrt{\text{number of runs}},$$

and an endpoint is valid only if its loss is below 3.28 and its score is at least 0.004. Muon's record is 3,375 steps (495.817 s) at a mean loss of 3.27893 over 20 runs, and it clears both conditions. The AdamW record, which its maintainer flags as undertuned, is 5,625 steps (833.211 s) at 3.27903 from a single run, so its loss is below 3.28 but its score falls short of 0.004. With the rule's numbers the two scores are

$$\begin{aligned} \text{Muon:} \quad & (3.28 - 3.27893) \times \sqrt{20} \approx 0.0048, \\ \text{AdamW:} \quad & (3.28 - 3.27903) \times \sqrt{1} \approx 0.0010. \end{aligned}$$

The number of runs enters the score, but it does not normalize time. The plot's seconds only mark where the endpoints lie. The plot shows where two fixed-budget runs ended. It does not compare matched tuning, and it is not a general speedup.

Moonlight reports about 2× computational efficiency: in its own compute-optimal scaling study, Muon matched AdamW with about 52% of AdamW's training FLOPs (Liu et al., 2025). Wen et al. retuned several optimizers at four model sizes. Under matched tuning, the matrix-based optimizers gained 1.4× over a well-tuned AdamW at 0.1B, shrinking to 1.1× at 1.2B (Wen et al., 2026). Keep the two numbers apart and do not average them. The measured advantage depends on scale, tuning and the comparison budget. MAI-Base-2 switched its pretraining from AdamW to Muon, citing "more stable training dynamics and large efficiency gains over AdamW", and gives no numbers for either (Microsoft AI, 2026).

MuonClip at Scale

26

K2 traced its logit instability to Muon

- 53B test run: logits pass 1,000
- Saturated softmax weakens attention gradients
- K2: logit explosion more frequent with Muon than AdamW (Kimi Team, 2025)
- K2 skips qk-norm (normalizes q, k): MLA never forms keys (Kimi Team, 2025)

▼ K2 links logit growth to Muon's full-rank updates

K2's 53B test run from the opening trained with plain Muon, and its largest attention logits passed 1,000 (Kimi Team, 2025). The quantity behind that number is the **max logit**. For one attention head h, the logits are the scaled dot products $q_i \cdot k_j / \sqrt{D_h}$ of each query q_i with each key k_j it attends to, where D_h is the head dimension. The max logit $S^h_{\max}$ is the largest of these softmax inputs over every sequence in the batch.

A large max logit matters because of what it does to the softmax. When one score leads the others by a wide margin, the softmax puts nearly all of the query's weight on that one key, and the attention distribution becomes almost one-hot: it is **saturated**. A saturated softmax passes back little gradient, as Week 2 derived from the softmax Jacobian $\partial s_i / \partial z_j = s_i(\delta_{ij} - s_j)$: when the largest weight s_a of a row is close to 1, every entry of the row's Jacobian has magnitude at most $1 - s_a$, so the scores receive almost no gradient. This makes a feedback loop: larger logits saturate the softmax further and weaken the gradients that could correct them.

The K2 report states that this explosion "occurs more frequently with Muon but less with AdamW in our experiments" (Kimi Team, 2025). The observation comes from the team's runs only.

The report's Appendix E gives the team's hypothesis for the difference (Kimi Team, 2025). It starts from a bound on one logit. Write the attention layer's inputs x_i and x_j as rows, so that $q_i = x_i W_q$ and $k_j = x_j W_k$ (Weeks 2 and 3 wrote W_Q and W_K), and let $\|W\|_2$ be a projection's spectral norm, its largest singular value. Then

$$\begin{aligned} |q_i \cdot k_j| &\le \|q_i\|_2 \, \|k_j\|_2 && \text{Cauchy–Schwarz} \\ &\le \|x_i\|_2 \, \|x_j\|_2 \, \|W_q\|_2 \, \|W_k\|_2 && \|xW\|_2 \le \|x\|_2 \, \|W\|_2. \end{aligned}$$

RMSNorm keeps the input norms from growing, so a logit that keeps growing needs a growing $\|W_q\|_2$ or $\|W_k\|_2$. Muon's update comes out of msign, so all its singular values are equal and its effective rank, the number of singular directions that carry real weight, is full, where a typical Adam update has a few dominant ones. The team hypothesizes that full-rank updates

tend to line up with a high-rank weight's own singular directions, so that its singular values grow step after step. The logit depends on the weights only through the product $W_q W_k^\top$, since $q_i \cdot k_j = x_i W_q W_k^\top x_j^\top$, and that product compounds growth in either matrix.

A normalization aimed directly at growing attention logits is qk-norm, which normalizes each head's queries and keys before their dot product. K2 did not use it. Its stated reason is that qk-norm "is not applicable to multi-head latent attention (MLA), because its Key matrices are not fully materialized during inference" (Kimi Team, 2025). K2 uses MLA, and in the inference form Week 4 described for DeepSeek V2, the key up-projection is merged into the query side, so queries are scored directly against the cached latent. The per-head keys are never formed, so a normalization of each key has no key to act on. K2 added QK-Clip instead, which rescales the query and key projection weights of the offending heads and leaves Muon's update of every other parameter as it is.

The course model is a separate case. It runs MLA in the unabsorbed form of Week 4, which builds the per-head content keys, and it has qk-norm as an option.

QK-Clip rescales only the offending head

$$\gamma_h = \min\left(1,\ \frac{\tau}{S^h_{\max}}\right) \qquad W_q^h \leftarrow \sqrt{\gamma_h}\, W_q^h, \quad W_k^h \leftarrow \sqrt{\gamma_h}\, W_k^h \tag{6}$$

- Measured every step: each head's max logit S
- Chosen: threshold τ = 100; above it, scale W_q, W_k by √γ
- K2 run: 12.7% of heads clipped, then none (Kimi Team, 2025)
- MuonClip: Muon, weight decay, RMS matching, QK-Clip (Kimi Team, 2025)

▼ QK-Clip scales one head's logits by γ_h

QK-Clip treats the max logit as a monitored quantity with a hard budget. At every step K2 reads each head's max logit $S^h_{\max}$ from that step's forward pass and compares it with a threshold τ. The max logit is measured, while τ is a hyperparameter that the team chooses, and K2 sets it to 100. For a head above the threshold, formula (6) sets $\gamma_h = \min(1, \tau / S^h_{\max})$ and multiplies the head's query and key projection weights, W_q^h and W_k^h, each by $\sqrt{\gamma_h}$ (Kimi Team, 2025).

Claim. Scaling both projections by $\sqrt{\gamma_h}$ scales every logit of the head by γ_h. Each query $q_i = x_i W_q^h$ is linear in W_q^h, so it is scaled by $\sqrt{\gamma_h}$, and so is each key. Each logit is one query-key dot product, so

$$\begin{aligned} (\sqrt{\gamma_h}\, q_i) \cdot (\sqrt{\gamma_h}\, k_j) &= \sqrt{\gamma_h}\,\sqrt{\gamma_h}\,(q_i \cdot k_j) \quad \text{bilinearity} \\ &= \gamma_h\,(q_i \cdot k_j). \end{aligned} \tag{6a}$$

The factor $1/\sqrt{D_h}$ does not change this. Scaling each projection by γ_h would apply the ratio twice and scale the logits by γ_h^2. A head at or below the threshold has $\gamma_h = 1$ and is left alone. K2 clips each head on its own, to change training as little as possible, because only a few heads showed exploding logits (Kimi Team, 2025).

A worked example. Suppose a head's max logit is $S^h_{\max} = 225$. Then

$$\begin{aligned} \gamma_h &= \frac{100}{225} = \frac{4}{9} && \text{formula (6), } \tau = 100 \\ \sqrt{\gamma_h} &= \frac{2}{3} && \text{the factor on each projection} \\ 225\,\gamma_h &= 100 && \text{every logit scales by } \gamma_h \text{, by (6a).} \end{aligned}$$

With fixed inputs and no other change to the weights, the max logit would become 100.

The forward pass computes and saves $S^h_{\max}$ before the parameter update, and the clip leaves the current step's forward and backward computation unchanged (Kimi Team, 2025). Muon then updates the weights, and only after that does QK-Clip read the saved max logit and rescale the selected weights. It does not rescale Muon's momentum buffer. The next forward pass sees weights that carry both that step's Muon update and the clip, so its max logit has to be measured again, and it need not be 100.

K2's attention is MLA, in which each query and key has a content part and a rotary part (Week 4), and K2 clips only what belongs to one head. The content parts q^C and k^C each receive $\sqrt{\gamma_h}$ and the rotary query q^R the full γ_h. The rotary key k^R is shared by all heads, so K2 leaves it untouched, and one head's clip never reaches another (Kimi Team, 2025). Both parts of the logit are then scaled by γ_h:

$$
\begin{aligned}
(\sqrt{\gamma_h}\, q^C) \cdot (\sqrt{\gamma_h}\, k^C) &= \gamma_h\, (q^C \cdot k^C) \quad (6a) \\
(\gamma_h\, q^R) \cdot k^R &= \gamma_h\, (q^R \cdot k^R) \quad k^R \text{ untouched.}
\end{aligned}
$$

K2's report names the whole optimizer after the clip: "We integrate Muon with weight decay, consistent RMS matching, and QK-Clip into a single optimizer, which we refer to as MuonClip" (Kimi Team, 2025). Weight decay and RMS matching are the first two practices of the Muon tab.

How much did the clip do in Kimi K2's main run, with $\tau = 100$? In the first 70,000 steps, 12.7% of the attention heads triggered QK-Clip at least once. After that, every head had brought its max logit below 100, and the clip no longer acted (Kimi Team, 2025). This is one run's record. The signal to keep is the per-head max logit.

QK-Clip in action

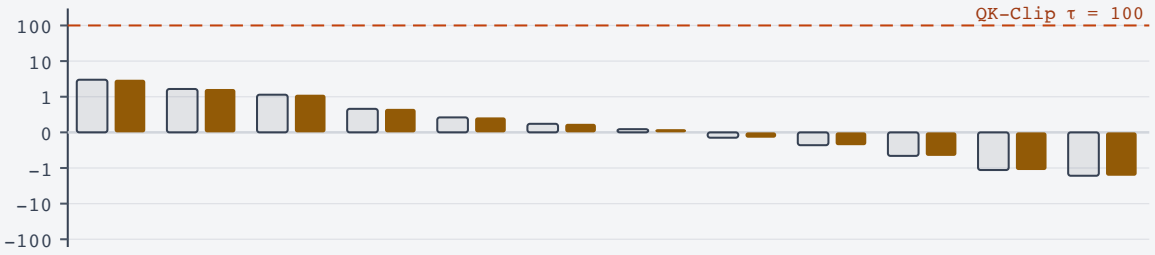
Read on your own

- Max logit capped at τ = 100
- The widget clips the current logits
- K2 rescales the weights after the optimizer step

ILLUSTRATIVE Max attention logit: 3.0

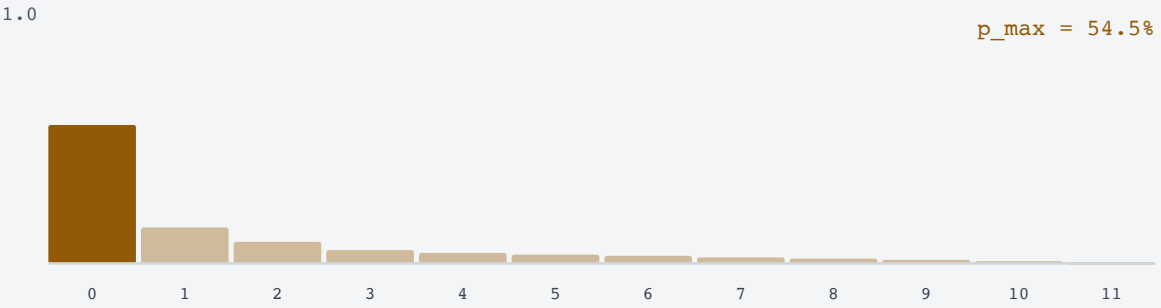
1.5 (healthy) 300 (exploding)

(a) attention logits: outline = before clamps, solid = after clamps



max logit (pre → post)	softmax entropy	top probability
3.0 → 3.0	**1.615 nats**	**54.5%**

(b) softmax distribution over 12 tokens



QK^T → softmax → loss

QK-Clip acts on the weights

Move the slider to watch the softmax saturate. Toggle QK-Clip and read panel (a): the solid post-clamp bars cap the max logit at 100, even while the softmax below stays saturated. The real QK-Clip rescales the query and key weights after the optimizer step, so the next forward pass sees the smaller logits; the widget applies the same rescaling to the current logits.

▼ Reading the QK-Clip widget

The widget holds one synthetic attention pattern: a query scored against twelve keys, with fixed relative sizes in which the largest score leads clearly and the second largest is 0.55 times the largest. The slider scales the whole pattern, its largest logit on a logarithmic track from 1.5 to 300, and panel (a) shows the logits before the controls as outline bars and after them as

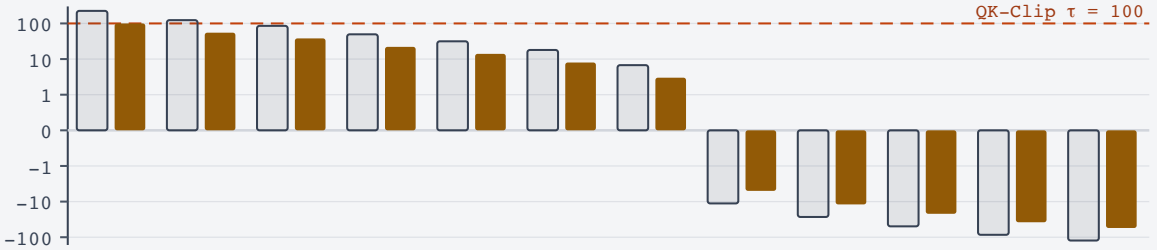
solid bars. Its axis is fixed, linear between -1 and 1 and logarithmic beyond, so from ± 1 outward each tick is ten times the one before it. A dashed line marks $\tau = 100$.

With QK-Clip off, the top probability is about 55% at the starting value of 3, and the other eleven keys share the rest. It passes 96% at a largest logit of 8 and is above 99% at 12, so this pattern saturates long before its logits reach the hundreds. With QK-Clip on and the largest raw logit, $S_{\max}$, above 100, the solid bars are the outline bars scaled by $100/S_{\max}$, so the largest stays at 100, while the outline bars keep growing. On the logarithmic axis, that scaling makes every solid bar shorter than its outline by the same length. The top probability stays at 100.0% and the entropy at 0.000 nats. In this pattern a logit of 100 still saturates the softmax: the largest score leads the second by 45, and $e^{-45} \approx 3 \times 10^{-20}$. The cap stops the logit from growing further. It does not, by itself, give the head a spread-out distribution again.

ILLUSTRATIVE Max attention logit: 225

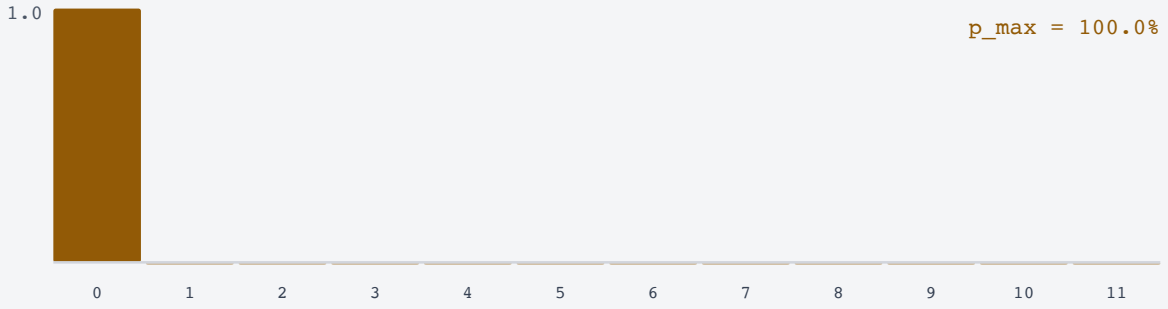
1.5 (healthy) 300 (exploding)

(a) attention logits: outline = before clamps, solid = after clamps



max logit (pre → post)	softmax entropy	top probability
225.0 → 100.0	0.000 nats	100.0%

(b) softmax distribution over 12 tokens



QK^T → softmax → loss

QK-Clip acts on the weights

Move the slider to watch the softmax saturate. Toggle QK-Clip and read panel (a): the solid post-clamp bars cap the max logit at 100, even while the softmax below stays saturated. The real QK-Clip rescales the query and key weights after the optimizer step, so the next forward pass sees the smaller logits; the widget applies the same rescaling to the current logits.

A raw max logit of 225, the worked example for formula (6), with QK-Clip on. Each solid bar is its outline bar times 100/225 = 4/9, so the largest logit drops to 100 and the second to 55. The top probability still reads 100.0% and the entropy 0.000 nats, because a lead of 45 over the second key is enough to saturate the softmax.

Stability comes from the whole training system Read on your own

- K2: QK-Clip on projection weights (Kimi Team, 2025)
- DeepSeek V4: RMSNorm on queries, KV entries (DeepSeek-AI, 2026a)
- GLM-5: per-head Muon, no clipping needed (GLM Team, 2026)
- Kolibri: adapted QK-Clip spiked where qk-norm held (Aleph Alpha, 2026)

Stability is a system property

▼ Stability has five possible sources

Three labs that train with Muon keep their attention logits in range in three ways. K2 rescales the query and key projection weights of any head whose max logit passes τ, and reports zero loss spikes in its main run (Kimi Team, 2025). DeepSeek V4 applies an RMSNorm, a form of qk-norm, to each query head and to its compressed KV entries just before attention, reports that this prevents the logits from exploding, and uses no QK-Clip (DeepSeek-AI, 2026a). GLM-5 orthogonalizes Muon's updates per attention head, its "Muon Split", and reports stable logits without any clipping (GLM Team, 2026). The three models also differ in architecture, data and scale, so the reports do not rank the three mechanisms. They show that how a run keeps its logits in range is a choice of the whole training system, not a property of Muon.

Later reports keep disagreeing. Kimi K3 keeps "the weight-clipping mechanism introduced in Kimi K2" (Kimi Team, 2026), next to the per-head Muon that it and DeepSeek-V4.1-Flash use (see "Four practices keep Muon stable at scale"). Qwen3.8-Flash-Next was trained "without relying on explicit clipping methods such as qk-clip ... or SwiGLU-clip" and credits its gated residual (Qwen Team, 2026). Kolibri compared two of these mechanisms on a grouped-query-attention proxy, one run per configuration: with qk-norm, training stayed stable, and the adapted QK-Clip still showed spikes (Aleph Alpha, 2026) (see "Labs stress-test stability on proxy models"). K2's own reason for QK-Clip, that MLA's keys are never materialized at inference, does not apply to grouped-query attention.

Sort the possible causes of a stable run into five places.

- **Optimizer:** Muon or AdamW decides how the gradient history becomes a parameter update.
- **Architecture:** the attention representation decides whether per-head weight clipping, qk-norm or another placement is coherent.
- **Loss:** an auxiliary penalty can discourage an extreme output without changing the attention computation directly.
- **Data:** corrupt, duplicated, shifted or unusually difficult batches can start a different path to instability.

- **Precision:** the exponent range, the accumulation type and local scaling decide whether finite mathematical values survive the implementation.

For your own runs, log each symptom on the tensor path where it forms, record which tensor an intervention changes and at which point of the step, and judge the optimizer, architecture, loss, data and precision choices together. A smooth loss trace alone cannot tell you which of them kept a run stable.

Schedules

30

Warmup protects the unstable first steps

- Early $\hat{v}$: built from a few gradients, so noisy
- Near initialization: high curvature, so full steps overshoot
- Warmup: raise the LR linearly from near zero
- K2: 500 steps, DeepSeek V3: 2,000 steps

▼ Why the first steps need a small learning rate

Two conditions make the first steps of training fragile. Adam divides each coordinate's step by the square root of its second-moment estimate $\hat{v}_t$, and at the start $\hat{v}_t$ is built from a handful of gradients, so the preconditioner rests on an immature, noisy statistic. The curvature of the loss is also high near initialization, so a full-size step can overshoot, and it does so in the worst part of the loss landscape. Warmup ramps the learning rate up from near zero, which keeps the early steps small while the optimizer state becomes meaningful. The course's schedule ramps it linearly over its warmup steps.

Run	Warmup
Kimi K2	500 steps
DeepSeek V3	2,000 steps
OLMo 2	2,000 steps
MAI-Thinking-1	~12B tokens

The table shows how four labs size their warmup. Kimi K2 warms up for 500 steps (Kimi Team, 2025), DeepSeek V3 for 2,000 steps (DeepSeek-AI, 2024b), OLMo 2 for 2,000 steps (OLMo Team, 2025), and MAI-Thinking-1 for about 12B tokens (Microsoft AI Team, 2026). The units differ: three runs report steps and one reports tokens. A step count converts to tokens only through the global batch size, and the batch size can itself change during a run.

Cosine and multi-step schedules spend the budget differently Read on your own

- Cosine: decay to 10% of peak
- The horizon is fixed in advance
- DeepSeek LLM: LR ×0.316 at 80% and 90% of tokens (DeepSeek-AI, 2024a)
- The flat phase can be reused later

▼ Cosine fixes the run's length in advance

The default schedule of the GPT-3 era is a warmup followed by **cosine decay** to 10% of the peak learning rate. The rate falls along half a cosine from the peak to that floor, which it reaches continuously at the last token of the run. The total token budget is therefore built into every step. Stop the run early and the rate has not yet come down. Train for longer and there is no clean point from which to continue.

DeepSeek LLM uses a **multi-step** schedule instead (DeepSeek-AI, 2024a). It reaches its peak rate η_{peak} after 2,000 warmup steps and holds it until 80% of the training tokens. There the rate is multiplied by 0.316, and at 90% it is multiplied by 0.316 again, so after warmup

$$\eta_t = \begin{cases} \eta_{\text{peak}} & \text{up to } 80\% \text{ of the tokens,} \\ 0.316\,\eta_{\text{peak}} & \text{from } 80\% \text{ to } 90\%, \\ 0.316^2\,\eta_{\text{peak}} = 0.099856\,\eta_{\text{peak}} & \text{from } 90\% \text{ on.} \end{cases}$$

The second cliff therefore ends at about $0.1\,\eta_{\text{peak}}$, the same 10% of peak that the cosine default reaches. The report's stated reason for the long flat phase is reuse: when training continues on new data, the run can pick up from a checkpoint of the constant phase. That is the continual-pretraining motivation, and the warmup-stable-decay schedule builds on the same reuse argument.

Compare cosine and multi-step schedules Read on your own

- Multi-step: two cliffs down to 10% of peak
- MAI-Thinking-1: cosine to the same 10%
- DeepSeek V3: constant phase, then cosine to 3.3% of peak

Schedule: Multi-step (DeepSeek V1) | Constant + cosine (DeepSeek V3) | Cosine (MAI-Thinking-1)

Peak → ×0.316 at 80% → ×0.316 at 90% (fraction of run)



DeepSeek LLM: peak learning rate after 2,000 warmup steps, ×0.316 at 80% of tokens and ×0.316 again at 90%, ending at 10% of peak. Its 2,000 warmup steps, about 1% of the run for the 7B model, are the near-vertical rise from the bottom edge at the start.

▼ Reading the three presets in the schedule builder

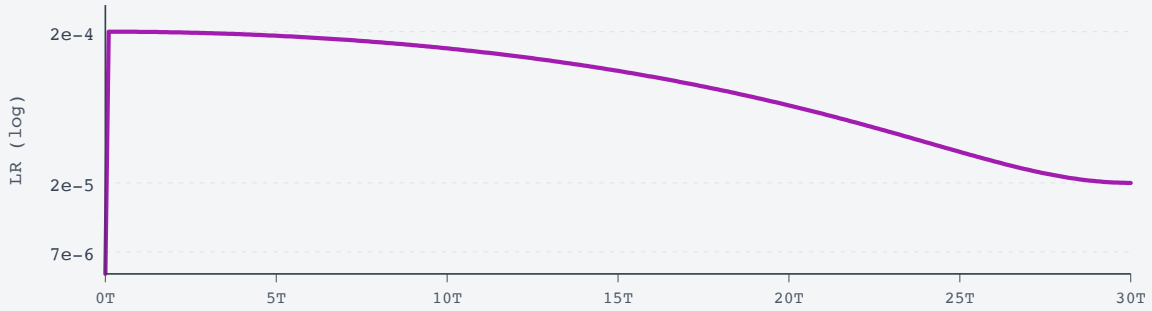
The builder plots each preset on its own logarithmic learning-rate axis: the multi-step preset in fractions of its peak over percent of the run, because its shape holds for any token count, and the two cosine presets in absolute numbers over tokens.

Compare the floors first. MAI-Thinking-1's cosine starts to decay at the end of warmup and ends at 10% of its peak (Microsoft AI Team, 2026), the same floor that the multi-step schedule reaches with its two cliffs. These two schedules differ only in how they spend the run on the way down. DeepSeek V3's schedule ends lower, at 3.3% of its peak (DeepSeek-AI, 2024b), and its long constant phase makes its shape closer to the multi-step schedule's than to MAI's.

The multi-step schedule holds its peak for 80% of the run and comes down in two steps. DeepSeek V3's holds it for about 68% of the run and then comes down continuously. MAI's leaves the peak at once and spends the whole run on the way down.

Schedule: Multi-step (DeepSeek V1) | Constant + cosine (DeepSeek V3) | Cosine (MAI-Thinking-1)

Peak 2e-4 → cosine to 2e-5 over 30T tokens, no constant phase



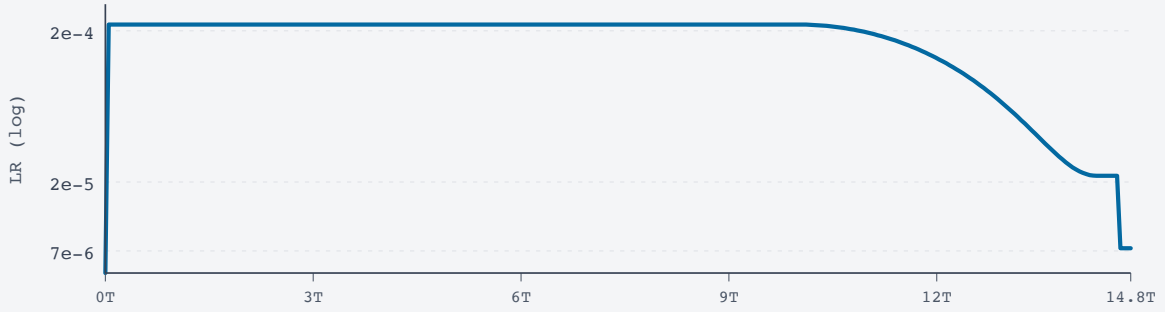
MAI-Thinking-1's pretraining run decays by cosine from 2e-4 to 2e-5 over 30T tokens, with no constant phase. Its mid-training phases are not drawn.

Its warmup, about 12B of 30T tokens, is the near-vertical rise from the bottom edge at the start.

MAI-Thinking-1: the rate leaves its peak of 2e-4 right after warmup and falls by cosine to 2e-5 over 30T tokens. It ends at 10% of peak, the floor the multi-step schedule reaches with two cliffs.

Schedule: Multi-step (DeepSeek V1) | Constant + cosine (DeepSeek V3) | Cosine (MAI-Thinking-1)

2.2e-4 constant to 10T → cosine to 2.2e-5 → 7.3e-6 for the last 167B (14.8T tokens)



DeepSeek V3 holds 2.2e-4 to 10T tokens, decays by cosine to 2.2e-5 at 14.3T, holds that for 333B tokens, then drops to 7.3e-6 for the last 167B. It ends at 3.3% of peak.

Its 2,000 warmup steps, about 25B of 14.8T tokens, are the near-vertical rise from the bottom edge at the start.

DeepSeek V3: the rate holds 2.2e-4 to 10T tokens, about 68% of the run, falls by cosine to 2.2e-5 and ends at 7.3e-6 for the last 167B tokens, 3.3% of peak. Its long constant phase makes it look more like the multi-step schedule than like MAI's cosine.

WSD separates training from checkpoint decay

- Cosine: the decay is tied to a fixed run length
- Warmup-stable-decay (WSD): named by MiniCPM (Hu et al., 2024)
- Stable: constant LR, every checkpoint can start a decay
- Decay: branch a leg when you need one
- K2: 2e-4 for 10T tokens, then decay over 5.5T (Kimi Team, 2025)

▼ WSD starts the decay only when a finished model is needed

A cosine schedule ties its decay to a run length fixed in advance, so a longer run means starting over. **Warmup-stable-decay (WSD)** takes the reuse argument to its end. MiniCPM named the schedule (Hu et al., 2024). **Warmup** ramps the learning rate to its working value. **Stable** trains at that constant rate, and every checkpoint saved in this phase can later start a decay. **Decay** lowers the rate, and it is run as a branch: when you need a finished checkpoint, you take a stable checkpoint and decay a copy of it, while the original stable run continues at its constant rate. The stable run commits to no end point, and each decay leg sets its own horizon.

Kimi K2's main run used this schedule. After a 500-step warmup it trained at a constant 2e-4 for 10T tokens, then decayed along a cosine from 2e-4 to 2e-5 over 5.5T tokens, which ends the 15.5T-token main run (Kimi Team, 2025). A later long-context extension anneals the rate from 2e-5 to 7e-6 (Kimi Team, 2025). That anneal is a separate phase, and its 7e-6 is not the endpoint of the main run.

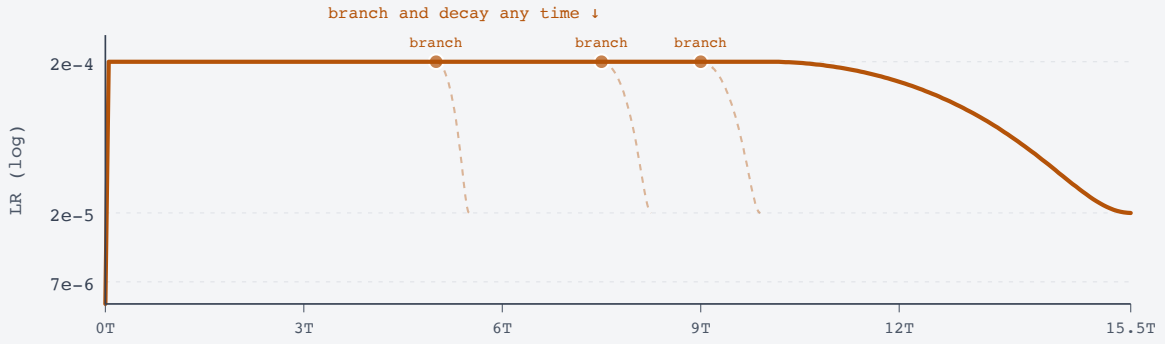
The course's schedule, `wsd_with_warmup`, has the same three phases with a different decay. It warms up linearly over `warmup_steps`, holds the full rate up to and including step `decay_start_frac * max_steps`, and then falls linearly to `min_lr_frac` times the peak at `max_steps`. The Week 5 configs set `min_lr_frac: 0`, so the course decays linearly to zero, where K2's decay is a cosine to 2e-5.

Branch a decay leg without ending the stable run

- Solid: K2's reported 15.5T run
- Dashed: illustrative decay legs, each 10% of its stable span (Hu et al., 2024)
- Open question: does skipping decay help fine-tuning?
- Yano et al.: one lab, 1B and 8B, SFT only (Yano et al., 2026)

Schedule: Multi-step (DeepSeek V1) | Constant + cosine (DeepSeek V3) | WSD (Kimi K2)

ILLUSTRATIVE Main run: 2e-4 stable → 2e-5 at 15.5T; later long-context anneal: 7e-6 (separate)



Kimi K2's main run holds 2e-4 for 10T tokens, then decays by cosine to 2e-5 at 15.5T. A later long-context anneal to 7e-6 is a separate phase and is not drawn. The dashed traces are illustrative WSD branches: each decays over 10% of its stable-token budget to 10% of its starting learning rate. Kimi K3 later chose cosine over WSD: after separate hyperparameter searches for each schedule, cosine consistently reached a lower final loss (Kimi Team, 2026).

Its 500 warmup steps, about 34B of 15.5T tokens, are the near-vertical rise from the bottom edge at the start.

▼ One stable run yields several finished checkpoints

The builder opens on K2's schedule. The solid curve is K2's reported 15.5T-token main run, which ends at 2e-5. The dashed curves are illustrative, and K2 reports none of them. Each starts from a stable checkpoint, at 5T, 7.5T and 9T tokens, and decays over 10% of the stable tokens before it, so the legs end at 5.5T, 8.25T and 9.9T (Hu et al., 2024): three finished checkpoints from one stable run that keeps training.

WSD's case is the reuse of the stable prefix, not a lower final loss. MiniCPM's small models matched or beat cosine at the same step (Hu et al., 2024). Kimi K3, after separate hyperparameter searches for each schedule, found cosine's final loss consistently lower and chose it (Kimi Team, 2026). The two results come from different scales and protocols.

Every `checkpoint_interval` steps, the course trainer writes `checkpoint-step-{step}.pt` with the model's weights, the optimizer's state, the step count and the number of training windows drawn so far. A decay leg resumes from such a file and continues the data where the checkpoint stopped. The stable run that wrote the file is untouched by the leg.

One question about decay is open. Could leaving out the learning-rate decay make a model better suited to supervised fine-tuning, even when its pretraining loss is worse? Yano et al. report this result in an ICLR 2026 paper from a single lab, for 1B and 8B models, evaluated with supervised fine-tuning (SFT) only (Yano et al., 2026). That is enough to raise the question, and not enough to recommend skipping decay.

A second lab, Aleph Alpha, has since reported the same reversal at a larger scale. Maskey et al. took one mixture-of-experts run with 30B total and 3B active parameters to 7.5T tokens and compared three endings: a constant learning rate, a cooldown to 10% of the peak learning rate over the last 800B tokens, and a merge of 20 checkpoints from the constant phase (Maskey et al., 2026). The cooldown beat the constant rate on every pretraining signal, with a validation loss of 1.648 against 1.734, though the merge scored highest on the pretraining aggregate (0.460 against 0.440). After identical mid-training, long-context training and SFT, the cooldown scored worst, with an aggregate score of 0.247 against 0.360 for the constant rate and 0.363 for the merge (Maskey et al., 2026). The authors name their limit as "one 30B MoE family with one seed per setup". Kolibri and Ling 2.6 now end pretraining without a decay and merge checkpoints instead, a "Warmup-Stable-Merge" schedule (Aleph Alpha, 2026; Ling Team, 2026). Nemotron 3 Ultra merges stable-phase checkpoints "weighted to emulate our learning rate decay schedule" for evaluation (NVIDIA, 2026).

Batch size changes through training too Read on your own

(7) $$B \to kB \quad \Rightarrow \quad \eta \to \sqrt{k}\,\eta \qquad (\text{Adam})$$

- Bigger batch: less wall-clock time, less learning per token
- Critical batch size (CBS): where returns start to diminish
- Stochastic gradient descent (SGD): scale the LR linearly
- Adam: scale the LR by the square root

▼ The critical batch size marks diminishing returns

A bigger batch buys **wall-clock time**, because data parallelism processes more samples in each step. Past some point it costs **token efficiency**: the same tokens teach the model less per token. The **critical batch size (CBS)** marks that point of diminishing returns. Below it, adding samples to a step cuts the number of serial optimization steps needed to reach a training target, at roughly the same token cost. Above it, a larger batch consumes more tokens per step without a matching reduction in the steps required. The boundary moves with the training state and with the optimizer.

In symbols, write B for the global batch in tokens and $N_{\text{steps}}(B)$ for the number of optimizer steps a run with that batch needs to reach a fixed loss target, so that the run consumes $B\,N_{\text{steps}}(B)$ tokens. The CBS, B_{crit}, separates two cases of an empirical relationship:

$$\begin{aligned} N_{\text{steps}}(2B) &\approx \tfrac{1}{2}\,N_{\text{steps}}(B) \quad && \text{for } B \text{ well below } B_{\text{crit}} \\ N_{\text{steps}}(2B) &\approx N_{\text{steps}}(B) && \text{for } B \text{ well above } B_{\text{crit}}. \end{aligned}$$

Below the CBS, doubling the batch halves the serial steps at about the same token count $B\,N_{\text{steps}}(B)$. Above it, doubling saves few steps and about doubles the tokens.

When the batch grows, the learning rate has to follow. For stochastic gradient descent (SGD) the usual rule is linear: k times the batch, k times the rate. For Adam it is **square-root scaling**, formula (7):

$$\begin{aligned} \text{SGD:} \quad & B \to kB \quad \Rightarrow \quad \eta \to k\,\eta \\ \text{Adam:} \quad & B \to kB \quad \Rightarrow \quad \eta \to \sqrt{k}\,\eta \quad \text{formula (7).} \end{aligned}$$

The rule in formula (7) is a heuristic and a starting point for a local measurement. For example, if the batch doubles from an Adam rate of $\eta = 2 \times 10^{-4}$, the heuristic proposes

$$\sqrt{k}\,\eta = \sqrt{2} \times 2 \times 10^{-4} \quad k = 2$$
$$\approx 2.828 \times 10^{-4}$$

as a candidate rate for the larger-batch branch.

Why not estimate the CBS from gradient noise? The gradient-noise-scale approach estimates it from the variance of mini-batch gradients relative to their mean (McCandlish et al., 2018). Merrill et al. report that this estimate is unreliable under Adam, whose preconditioner keeps changing the geometry of the step (Merrill et al., 2025). They measure instead. Fork one checkpoint into short runs at several batch sizes, change the learning rate with the batch, and compare the loss at the same token horizon. An untuned large-batch branch would make the larger batch look worse than it is (Merrill et al., 2025). In their OLMo 1B and 7B measurements the CBS starts near zero, grows rapidly, and then plateaus around 4,096 documents (Merrill et al., 2025).

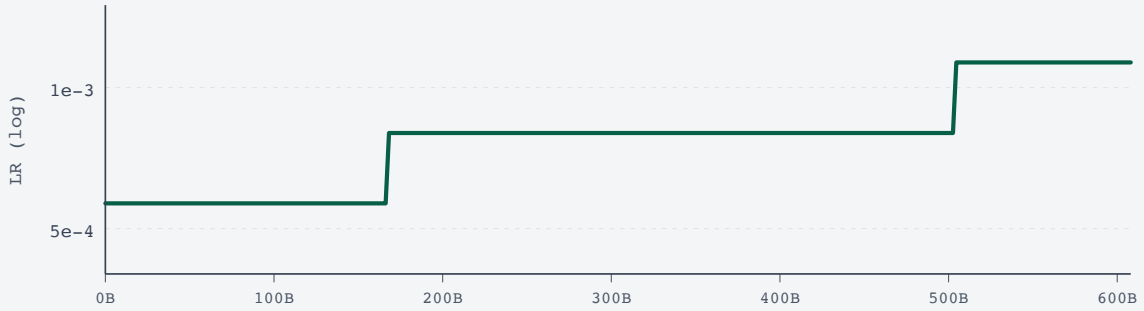
A batch of 4,096 documents at a nominal 4,096 tokens each is $4{,}096 \times 4{,}096 = 16{,}777{,}216$ tokens, about 16.8M, only if every sequence is full. With padding, packing or variable lengths, count the effective tokens. Measure the boundary for the current checkpoint and optimizer, and record the chosen batch, learning rate and token count together.

Couple the batch ramp to the learning rate Read on your own

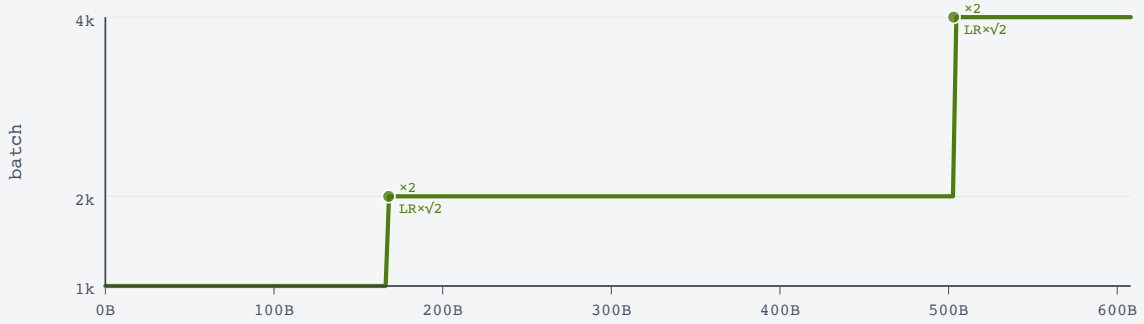
- Both tracks share one token axis
- Under Adam, k times the batch means $\sqrt{k}$ times the LR
- AI2: each batch doubling raises the LR by $\sqrt{2}$

Schedule: Multi-step (DeepSeek V1) | Constant + cosine (DeepSeek V3) | WSD (Kimi K2) | Cosine (MAI-Thinking-1) | Custom

OLMo 1B base LR: ×√2 at each batch doubling (168B, 503B tokens)



Batch size | This run's batch | AI2 batch warmup (OLMo 1B)



OLMo 1B run with batch-size warmup (Merrill et al., 2025): each doubling of the batch multiplies the base learning rate by $\sqrt{2}$. The cosine schedule that this base rate scales is not drawn, and neither is the 50B-token linear anneal to zero after 608B tokens.

Batch track: the batch starts at 1,024 documents of 4,096 tokens and doubles when the measured critical batch size reaches twice the batch, at 168B and 503B tokens.

▼ Reading a batch ramp beside the learning rate

The rule to apply is formula (7): under Adam, when the global batch grows by a factor k, multiply the learning rate by $\sqrt{k}$. One, two and three doublings of the batch give

$$\sqrt{2} \approx 1.414, \qquad \sqrt{4} = 2, \qquad \sqrt{8} = 2\sqrt{2} \approx 2.83.$$

The builder puts the learning rate and the batch size on one token axis, so you can read both at the same point of the run. It opens on the batch-size warmup of AI2's OLMo 1B run (Merrill et al., 2025). Where the batch track steps up, the learning-rate track steps up with it by a factor of $\sqrt{2}$, from about 5.66e-4 to 8e-4 and then to about 1.13e-3. After two doublings the batch is four times its starting size and the base rate twice its starting value, which is formula (7) with $k = 4$.

Under the "This run's batch" option, DeepSeek V3's ramp covers about the first 3% of its 14.8T-token axis, a short rise at the left edge of the batch track (DeepSeek-AI, 2024b). Compare the two runs where their batches grow: AI2's learning-rate track rises with each doubling, and V3's does not.

37

How frontier runs scale the batch Read on your own

- DeepSeek LLM: fit batch and LR to compute (DeepSeek-AI, 2024a)
- DeepSeek V3: 3,072 to 15,360 sequences over the first 469B tokens (DeepSeek-AI, 2024b)
- AI2: double the batch when the measured CBS allows (Merrill et al., 2025)

▼ Three batch strategies and what each one shows

DeepSeek LLM fits a law to a compute budget. After tuning proxy runs it reported $B_{\text{opt}} = 0.2920\, C^{0.3271}$ and $\eta_{\text{opt}} = 0.3118\, C^{-0.1250}$, which choose the batch size and the learning rate together (DeepSeek-AI, 2024a). Here C is the training compute in FLOPs and B_{opt} is the batch size in tokens. These are empirical fits, and the coefficients have meaning only in those units. The report fit them on runs from about 1e17 to 2e19 FLOPs and checked the result at 1e20 FLOPs. Keep the units and coefficients as reported, and do not extend the fit beyond the measured range.

DeepSeek V3 ramps the batch by training progress. Its batch grows from 3,072 to 15,360 sequences over the first 469B tokens and stays at 15,360 for the rest of training, so the run uses smaller, efficient batches early and more parallelism later (DeepSeek-AI, 2024b). The report does not test the ramp on its own.

AI2 warms the batch up by the measured CBS. It doubles the batch whenever the measured CBS reaches twice the current batch and scales the learning rate by $\sqrt{2}$ at each doubling. In a branched comparison at a fixed token budget, this matched the loss of a fixed-batch run with 43% fewer serial steps (Merrill et al., 2025). Batch-size warmup is the practical use of the CBS curve: small batches early, while the CBS is tiny, and large batches later, once it has grown.

Batch warmup: 2026 reports disagree Read on your own

- Qwen3.8, Muon: no gain, 18.8% more steps (Qwen Team, 2026)
- DeepSeek V4.1: fixed batch of 100.6M tokens, no instability (DeepSeek-AI, 2026b)
- Mistral Large 4, AdamW: ramps 1× to 4×, for throughput (Mistral AI, 2026)
- A.X K2, AdamW: ramp against early gradient spikes (Baek et al., 2026)

▼ **Four 2026 reports split on batch warmup**

Batch-size warmup starts with a small batch and grows it, on the argument that the critical batch size is small early in training. Four labs split on it in 2026. Qwen3.8-Flash-Next retested the ramp under Muon on a mixture-of-experts model with 10.8B total and 0.89B active parameters, trained on 4T tokens. The ramp grew the batch from 6.3M to 25.2M tokens by 524B tokens, once with the constant-batch run's peak learning rate and once with a lower one. Both ramps ended within run-to-run variance of the constant batch, but slightly worse. They needed 18.8% more optimizer steps and changed no stability measure, and Qwen concludes that the warmup is no longer necessary (Qwen Team, 2026). Its reasons are the optimizer and the experts: under Muon, a batch below the predicted optimum costs more than one above it, Muon keeps its data efficiency at larger batches where AdamW's degrades, and a large batch gives every expert enough tokens (Qwen Team, 2026). DeepSeek ramped V4-Flash's batch to 75.5M tokens (DeepSeek-AI, 2026a). It then trained V4.1-Flash on 45T tokens at a fixed batch of 100.6M tokens "with no instability", without saying why it changed (DeepSeek-AI, 2026b).

Mistral Large 4, trained with AdamW, ramps its batch from 1× to 4× its predicted batch size. It argues that a large batch early in training wastes compute and tokens, because a gradient estimated from a fraction of the batch suffices, and that the ramp also acts as a regularizer, more so the more often the data repeats. A ramp from 0.25× to 2× performed best in its experiments, and it chose 1× to 4× for wall-clock time, because larger batches use the hardware more efficiently (Mistral AI, 2026). A.X K2, also trained with AdamW, ramps its batch over roughly its first 295B tokens "to mitigate gradient spikes" (Baek et al., 2026). Neither report publishes numbers for its ramp. The one measured test ran under Muon, so treat the optimizer as one possible reason for the split, not as a settled rule.

Reading a Run

39

At 70% of the run, the LR decays.

Does the loss now fall faster or slower?

Answer A2

Which signal leads depends on the failure

- Log the gradient norm before clipping
- A healthy norm drifts slowly within a band
- Which signal leads differs from failure to failure

▼ **Log the gradient norm before clipping**

After the loss, the pre-clip gradient norm is usually the most informative single trace. Global-norm clipping takes the accumulated gradient g and a threshold τ_{clip}, computes the clip coefficient

$$\alpha = \min\left(1,\ \frac{\tau_{\text{clip}}}{\|g\|_2 + \epsilon_{\text{clip}}}\right),$$

where the small constant ϵ_{clip} guards against division by zero, and replaces g with αg. Log the pre-clip norm $\|g\|_2$ at every optimizer step, after gradient accumulation and after any loss scale has been divided back out, and before the gradient is multiplied by α.

The order matters because clipping erases the magnitude. With $\tau_{\text{clip}} = 1$, pre-clip norms of 2 and 20 give coefficients of about $1/2$ and $1/20$, and both post-clip norms come out near 1. The post-clip series cannot tell a moderate excursion from a severe one. It confirms that the clip was enforced, and it hides how large a step the optimizer was asked to take.

In a healthy run the pre-clip norm is elevated and noisy during warmup, while the parameters are still far from any basin. Afterwards it settles into a slowly drifting band, which the dashboard shades as the "healthy band". Learn the band's level and its variance from stretches where nothing is wrong. That band is the recent baseline, and a spike score on the gradient norm compares each new value with it.

When a run fails, which signal moves first depends on the path the failure takes. No trace leads in every failure, so read lead and lag for each failure, from each signal's timing against its own baseline. Nemotron 3 Ultra's report gives two cases, both observed in one run. Before its first divergence, the loss of its second multi-token-prediction head spiked before the training loss did. Before its second, which the report never explained, the residual-stream norms of early layers began rising at about 7.5T tokens and spiked at about 11T, and the largest expert overload in its mixture-of-experts layers (MaxVio) climbed to about 12 by 12T (NVIDIA, 2026). The report calls that overload "correlated with training instability", not causal. The exercise asks which signal crossed first in its two runs at a high learning rate, the hot pair, and you answer from your own traces.

Read loss, gradients, learning rate, and throughput together

Read on your own

- First: the logged LR and phase
- Throughput: stalls point to data or kernels
- Max logit against its threshold
- Then loss and pre-clip norm together

▼ One reading order for a run's dashboard

Read a run's dashboard in a fixed order instead of scanning for the most dramatic curve. First align every trace and event log on optimizer-step IDs, and mark the earliest deviation in any trace, not merely the largest. Then go through the run in this order:

1. **Schedule.** Check the logged learning rate and phase first: warmup, stable or decay. A skipped warmup or a misplaced decay is visible at once. Compare the logged values with the intended schedule and with the state stored in the checkpoint, including the global batch size and the data-mixture phase.
2. **Throughput and data.** Tokens per second shows a dataloader stall or a dropped kernel that raises no error. At the same interval, inspect the effective token count, the batch statistics, the data cursor and any mixture or sampler boundary.
3. **Max logit.** Compare its trajectory with QK-Clip's threshold, $\tau = 100$.
4. **Loss and pre-clip gradient norm together.** In the loss, find the first change of slope or variance, or an outlier, and do not assign a learning stage from the shape alone. Ask whether the norm left its band before, at or after the loss event. At the same steps, check the precision events (finite-value checks, skipped steps and loss-scale changes) and the clip coefficient.
5. **Checkpoints.** Compare the last known-good checkpoint with the first affected state: model, optimizer, schedule, loss-scaler state and data cursor.
6. **Replay.** Restart from the known-good checkpoint with the captured data interval and configuration. First reproduce the symptom, then vary one suspected factor while holding the alternatives fixed.

One trace alone rarely identifies the cause: a shared timestamp shows an association, not a cause. A causal claim needs the replay and a controlled intervention that changes the proposed mechanism while leaving the competing explanations in place. All of this needs the run's own baseline: its healthy band, its batch and phase state, and its checkpoint history.

Loss Spikes & Stability

Loss spikes have several trajectories and causes

- Shapes: fast recovery, slow recovery or divergence
- Causes: bad batches, logit growth, fp16 underflow, initialization
- OLMo drops documents with 32+ repeated n-grams (Ettinger et al., 2025)
- The trace alone does not name a cause

▼ A spike's shape does not name its cause

A **loss spike** is a sharp jump of the training loss above its recent band. What follows the jump is the spike's trajectory, and it comes in three shapes: the loss can recover fast, recover slowly, or diverge. Classify the trajectory you observe before you propose a cause. A diverging trajectory does not return to a finite, usable band. A run can also settle into a new band, a shifted trajectory: it continues, but it may keep a lasting penalty in loss or quality. Over a short window, slow recovery and a shift can look alike.

The reported causes fall into four groups. The first is pathological data batches. OLMo removes documents with 32 or more repeated n-grams specifically to stop spikes (Ettinger et al., 2025). A bad example can also come from a bug in the data pipeline. MAI-Base-2 clips activation gradients at selected places and uses the clips to check its data: one clipped token led back to a code example whose license header had been cut off mid-sentence, a preprocessing bug (Microsoft AI, 2026). The report ties it to a clipped gradient, not to a spike. The second group is attention-logit explosion, in which growing scores saturate the softmax, as in the MuonClip at Scale tab. The third is fp16 numerics: the format's limited exponent range can underflow gradients. The fourth is bad initialization.

Hardware can also cause spikes. Laguna M.1's cross-replica hash checks found "a single machine producing silently corrupted arithmetic, surfacing as anomalous global gradient norms peaking at $\sim 10^6$" (Poolside Team et al., 2026). Altenbernd et al. inject such faults into LLaMA models of 60M to 1.3B parameters and observe NaNs, short spikes in the loss, the gradient norm and the attention logits, and persistent divergence of the parameters (Altenbernd et al., 2026).

The same shape can arise from the data, the optimizer state, the numerical range, the initialization or an interaction among them, and one trigger can give different shapes at different parameter states. A spike can depend jointly on the current parameters, the optimizer state and the data batch, and replaying or skipping data changes more than one of these at once. So the trace alone does not name a cause.

The spike score makes detection operational

$$\text{spike score} = \frac{1}{T-128}\sum_{t=129}^{T} \mathbf{1}\left[\, n_t \geq \mu(n_{t-128:t-1}) + 6\,\sigma_{\text{pop}}(n_{t-128:t-1}) \,\right] \tag{8}$$

- Preceding 128 norms: population mean and standard deviation (Merrill et al., 2026)
- Flag: at least 6σ above the mean
- Score: share of eligible steps flagged
- Course trainer: a z-score of each step's loss

▼ **Formula (8) scores the share of 6σ gradient-norm excursions**

A spike score turns "this run spikes a lot" into one number that compares runs. Formula (8) defines it on the pre-clip gradient norm. Let $n_t = \|g_t\|_2$ be that norm at optimizer step t, for $t = 1, \ldots, T$, where g_t is the gradient vector. For each eligible step $t = 129, \ldots, T$, the baseline is the 128 completed norms before it, $n_{t-128}, \ldots, n_{t-1}$, so the current value is not in its own baseline. Their mean μ and population standard deviation σ_{pop} both divide by 128, not by 127:

$$\mu = \frac{1}{128}\sum_{j=1}^{128} n_{t-j},$$

$$\sigma_{\text{pop}} = \sqrt{\frac{1}{128}\sum_{j=1}^{128}\left(n_{t-j} - \mu\right)^2}.$$

Step t is flagged when $n_t \geq \mu + 6\,\sigma_{\text{pop}}$. No step is scored until the 128-step history is full, so the denominator is the $T - 128$ eligible steps, not T. The score is the fraction of eligible steps that are flagged.

An illustrative calculation, not a reported training trace: for a window with mean 1.0 and population standard deviation 0.075, the threshold is

$$\mu + 6\,\sigma_{\text{pop}} = 1.0 + 6 \times 0.075 = 1.45.$$

A current value of 1.4 stays below it. A value of 1.45 is flagged, because the comparison is inclusive. If $T = 160$ and exactly two eligible values are flagged, the score is $2/(160 - 128) = 6.25\%$.

The rule's numbers come from the OLMo Hybrid report. It defines its spike score as the percentage of values in a time series that are at least six standard deviations away from a rolling average of the last 128 values, and applies it to the gradient norm (Merrill et al., 2026). The report does not say whether the current value is in its own window, which standard deviation it uses, or whether values below the average count, so formula (8) is our reading: the 128 preceding values, the population standard deviation, values above the mean only. OLMo 2 used a closely related statistic with a threshold of seven standard deviations (OLMo Team, 2025).

Other reports count spikes differently. Qwen3.8-Flash-Next flags a step whose loss exceeds its 201-step rolling median by more than 0.1 (Qwen Team, 2026). K2-V2, a model unrelated to Kimi K2, flags a loss more than five robust standard deviations, measured by the median absolute deviation, above its rolling median (K2 Team, 2025). Compare spike rates only under one definition.

The trainer you write this week logs a different score. At every step it computes the z-score of that step's loss, $(\mathcal{L}_t - \mu)/\sigma_{\text{pop}}$, where μ and σ_{pop} are now the mean and the population standard deviation of the preceding 128 losses, never including the step it scores. It logs no value until the window is full. The exercise flags a crossing when this per-step score reaches 6.

Rollback, skip, and resume after a spike

- Roll back to a clean checkpoint
- Skip batches (PaLM) or change the recipe
- DeepSeek V4: rollback alone, spikes came back (DeepSeek-AI, 2026a)

▼ Recovery must restore one coherent state

When a spike hit PaLM's training, the team rolled back to a checkpoint from before the spike, skipped 200 to 500 data batches, and resumed (Chowdhery et al., 2023). This is the standard playbook for a long run. Treat it as one logged change of the whole training state.

1. **Roll back.** Restore one known-good checkpoint whole: the parameters, the optimizer moments and any other optimizer state, the precision scaler, the learning-rate scheduler with its token counters, the RNG and sampler state, and the data cursor. Check that every restored field comes from the same checkpoint boundary.
2. **Skip.** Declare and log the exact range of skipped batch or sample IDs, then advance only the data cursor and the sampler through it. No hidden optimizer step happens while skipping: the optimizer moments, the scaler's growth or backoff, the scheduler and the token counters that drive the schedule stay where they are.
3. **Resume.** Run forward and backward from the first unskipped batch. Take the next optimizer and scheduler step only after that batch completes successfully, and record the first resumed batch and its optimizer-step ID, so that a replay uses the same boundary.
4. **Log.** Keep the source checkpoint's ID, the detector's inputs, the skipped range, the first resumed batch and step, the configuration hash, and whether a replay reproduced the symptom.

Newer reports pair the rollback with a change, or automate the decision to take it. DeepSeek V4 traced its spikes to outliers in its mixture-of-experts layers. Rollbacks restored the training state only for a while, because they did not stop those spikes from recurring (DeepSeek-AI, 2026a). An automatic detector there now triggers a short rollback and switches on a routing change for a period before normal training resumes (DeepSeek-AI, 2026a). Nemotron 3 Ultra rolled back twice: once it restored an fp32 gradient reduction, and once it started the learning-rate decay early (NVIDIA, 2026). K2-V2 rolls back automatically only for wide spikes, where the loss stays above a floor for a whole window, and lets narrow ones, which often self-correct, run on (K2 Team, 2025). Skipping data can help only when a batch was the trigger. In the DeepSeek and Nemotron cases the trigger was in the recipe, so the rollback came with a change to it.

The playbook changes the parameters and optimizer state, and it also moves the data position. A run that recovers is protected, but the recovery does not show which of those changes

removed the trigger. To attribute the spike to a batch, to the optimizer state or to a numerical mechanism, compare replays or controlled interventions.

Match each control to its intervention point

- QK-Clip: projection weights, after the optimizer step
- qk-norm: Q and K, before the dot product
- Softcapping: squash the logits below a cap c

$$\text{logits} \leftarrow c \cdot \tanh\left(\frac{\text{logits}}{c}\right) \tag{9}$$

- z-loss: output logits, through the loss

$$\mathcal{L}_z = \lambda_z \log^2 Z, \qquad Z = \sum_i e^{z_i} \tag{10}$$

- SwiGLU clamp: MLP activations
- Gradient clipping: the global gradient norm

▼ Where and when each control acts

Stability controls do not form one linear pipeline. They act on four connected tensor paths: the attention logits, the output logits, the MLP activations and the parameter gradients. For each control, ask which tensor it changes and at which point of the step.

QK-Clip acts on the attention projection weights after the optimizer step, so its rescaling first reaches the next forward pass, as the MuonClip at Scale tab showed (Kimi Team, 2025).

qk-norm normalizes the queries and the keys separately before their dot product forms a score. It originated in ViT-22B (Dehghani et al., 2023). OLMo 2 ablated its effect on logit growth and divergence directly (OLMo Team, 2025), and later reports include it in OLMo 3, Gemma 3, DeepSeek V4 and gpt-oss (OpenAI, 2025). In the course model, qk-norm is two RMSNorms over the head dimension, one for the queries and one for the keys, each applied to every head after the projection has been reshaped into heads and before RoPE rotates the vectors. In latent attention it normalizes the content halves only, so the rotary halves stay unnormalized. In the course's Week 5 reference run (one seed, eager PyTorch, one H100 node), the hot-learning-rate arm with qk-norm processed 362,354 tokens/s against 488,314 without it, about 26% slower. We attribute the gap to the two RMSNorms running as separate, unfused kernels, and we did not measure how much compiling the model recovers.

Softcapping transforms logits with formula (9),

$$\text{logits} \leftarrow c \tanh\left(\frac{\text{logits}}{c}\right),$$

which passes small values almost unchanged and bends large ones smoothly toward the cap c. An attention-logit softcap acts after the dot product and before the attention softmax. A final-logit softcap is the analogous operation after the language-model head and before the output softmax. Gemma 2 used a cap of 50 on attention logits and 30 on final logits (Gemma Team, 2024), and Gemma 3 reports replacing softcapping with qk-norm (Gemma Team, 2025). The final-logit cap has come back next to qk-norm: the released configurations of Gemma 4 and Muse Spark Open set final-logit caps of 30 and 20 and no attention-logit cap, and neither says whether its cap was active in training (Google DeepMind, 2026) (Meta Superintelligence Labs, 2026). With $c = 50$, a raw attention logit of 100 becomes

$$50 \tanh(100/50) = 50 \tanh 2 \approx 48.2.$$

z-loss acts on the output logits through the training objective. With output logits z_i and the softmax normalizer $Z = \sum_i e^{z_i}$, formula (10) adds the penalty

$$\mathcal{L}_z = \lambda_z (\log Z)^2,$$

averaged over predicted positions, to the loss before the backward pass. It changes the gradient that reaches the logits. It does not clamp the logits, and it does not act on attention probabilities. PaLM and OLMo 2 report a weight $\lambda_z = 10^{-4}$ (Chowdhery et al., 2023) (OLMo Team, 2025), and OLMo 3 reports $\lambda_z = 10^{-5}$ (Ettinger et al., 2025).

A **SwiGLU clamp** acts on MLP intermediates. SwiGLU is the gated MLP from Week 2: a gate branch $\mathrm{Swish}(xW_{gate})$ multiplies a linear branch xW_{up} elementwise before W_{down} maps back to the model width. DeepSeek V4 clamps the linear branch to $[-10, 10]$ and caps the gate at 10, before their product (DeepSeek-AI, 2026a).

Gradient clipping bounds the global norm of the parameter gradients before the optimizer reads them. The technique goes back to training recurrent networks (Pascanu et al., 2013). As the final backstop it bounds a step without locating the source.

ViT-22B and Pascanu et al. introduced qk-norm and gradient clipping, but where a control comes from does not tell you how well it works. OLMo 2 and Kolibri each ablate qk-norm in their own setup, and Kolibri also compares it with an adapted QK-Clip (OLMo Team, 2025) (Aleph Alpha, 2026).

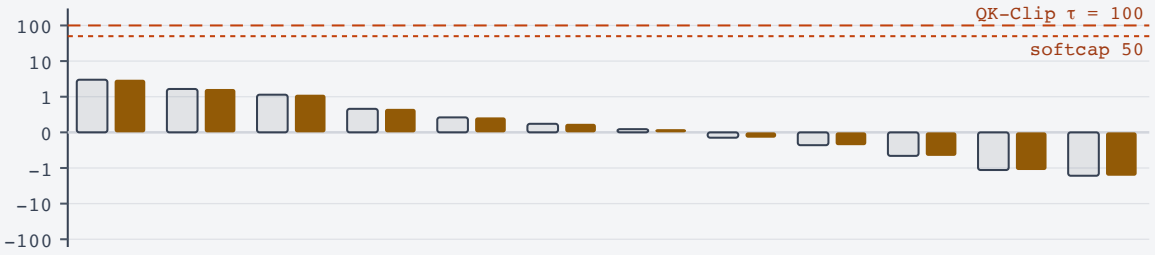
Compare stability controls where they act Read on your own

- Raise the max logit, toggle a control
- Watch which tensor changes

ILLUSTRATIVE Max attention logit: 3.0

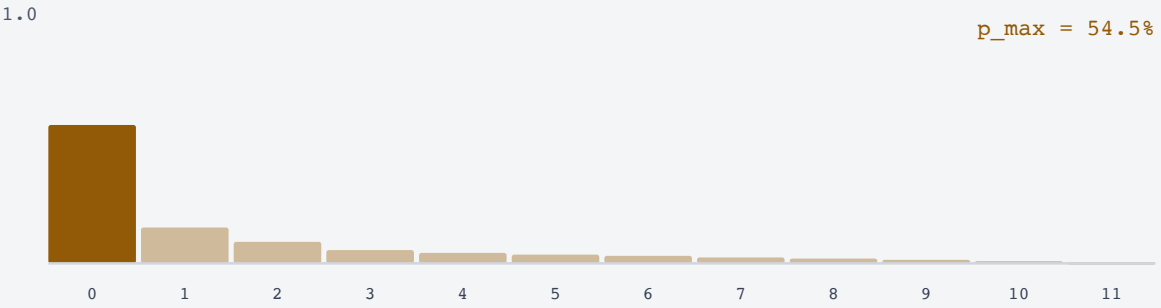
1.5 (healthy) 300 (exploding)

(a) attention logits: outline = before clamps, solid = after clamps



max logit (pre → post)	softmax entropy	top probability
3.0 → 3.0	1.615 nats	54.5%

(b) softmax distribution over 12 tokens



QK^T → softmax → loss

qk-norm
acts on Q and K before QK^T

softcap
acts on the logits

z-loss
acts on the loss

QK-Clip
acts on the weights

Move the slider, then toggle the controls. They act at different points: on the queries and keys before the logits form (qk-norm), on the logits (softcap), on the weights (QK-Clip), or on the loss (z-loss). In this widget, qk-norm ignores the slider: normalized queries and keys bound the logit, so it stays at about 1.4 whatever the raw scale.

▼ Reading the stability controls widget

The widget uses the synthetic attention pattern of the QK-Clip widget: one query scored against twelve keys, with the slider setting the largest raw logit on a logarithmic track from 1.5 to 300. It simulates four controls: qk-norm, the softcap, QK-Clip and z-loss. The SwiGLU clamp

and gradient clipping act on tensors this widget does not draw, the MLP intermediates and the parameter gradients, so they stay in the prose. The widget maps where each control acts.

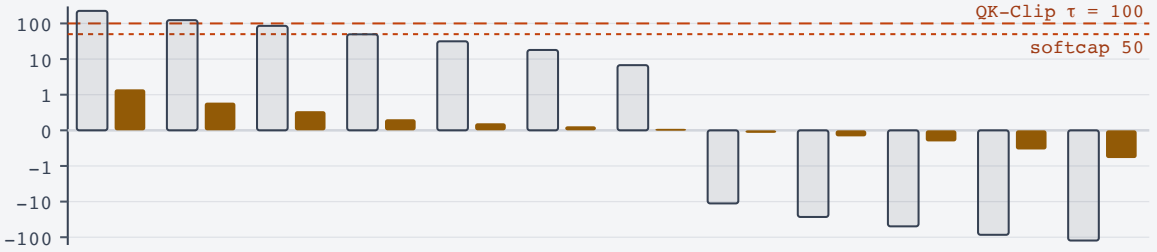
Move the slider past 100 and switch on one control at a time. The axis and the dashed line at $\tau = 100$ are those of the QK-Clip widget, and a second dashed line marks the softcap at 50.

- **qk-norm** forms the logits from normalized queries and keys.
- **The softcap** applies formula (9) with $c = 50$. Small logits pass almost unchanged, and large ones bend toward 50.
- **z-loss** leaves the bars unchanged, and its readout shows $\log Z$ and the penalty $10^{-4}\,(\log Z)^2$. The control changes the loss, and through it the gradient, not the logits of the current step.

ILLUSTRATIVE Max attention logit: 225

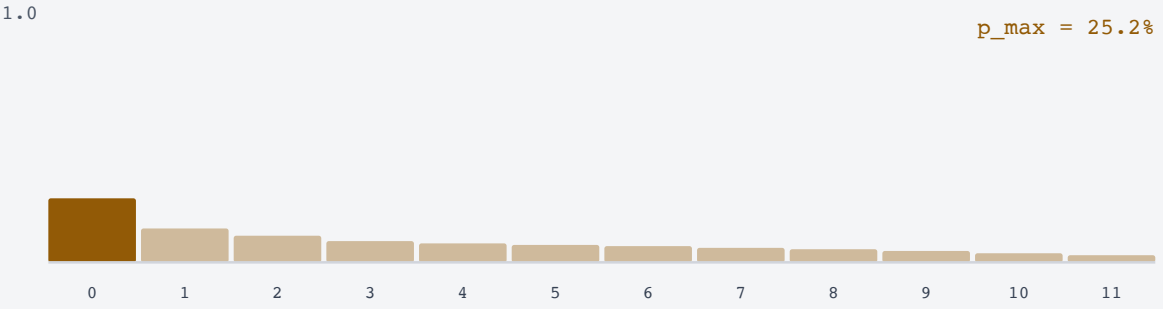
1.5 (healthy) 300 (exploding)

(a) attention logits: outline = before clamps, solid = after clamps



max logit (pre → post)	softmax entropy	top probability
225.0 → 1.4	2.291 nats	25.2%

(b) softmax distribution over 12 tokens



QK^T → softmax → loss

qk-norm
acts on Q and K before QK^T

softcap
acts on the logits

z-loss
acts on the loss

QK-Clip
acts on the weights

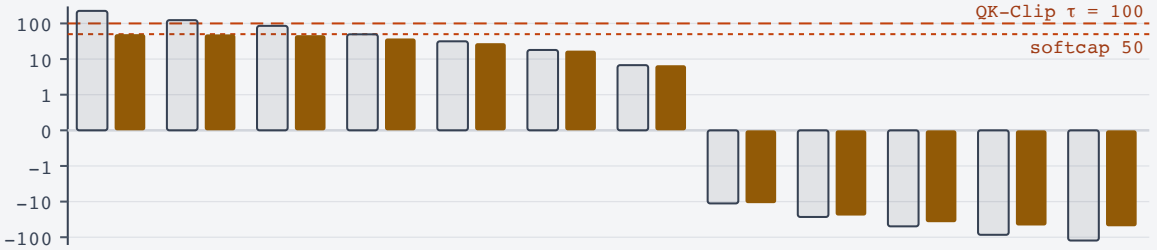
Move the slider, then toggle the controls. They act at different points: on the queries and keys before the logits form (qk-norm), on the logits (softcap), on the weights (QK-Clip), or on the loss (z-loss). In this widget, qk-norm ignores the slider: normalized queries and keys bound the logit, so it stays at about 1.4 whatever the raw scale.

qk-norm at a raw max logit of 225: the logits now come from normalized queries and keys, so the slider's scale drops out. The largest logit is 1.4 at every slider value, and the softmax spreads its weight again, with a top probability of 25.2%.

ILLUSTRATIVE Max attention logit: 225

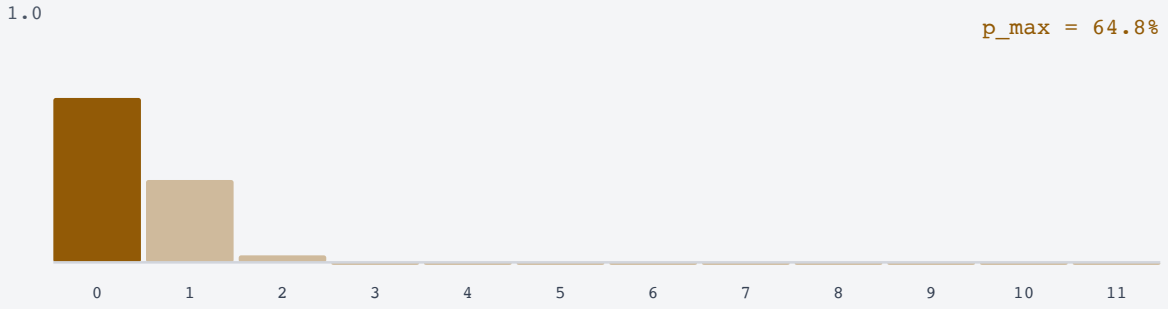
1.5 (healthy) 300 (exploding)

(a) attention logits: outline = before clamps, solid = after clamps



max logit (pre → post)	softmax entropy	top probability
225.0 → 50.0	0.746 nats	64.8%

(b) softmax distribution over 12 tokens



QK^T → softmax → loss

qk-norm
acts on Q and K before QK^T

softcap
acts on the logits

z-loss
acts on the loss

QK-Clip
acts on the weights

Move the slider, then toggle the controls. They act at different points: on the queries and keys before the logits form (qk-norm), on the logits (softcap), on the weights (QK-Clip), or on the loss (z-loss). In this widget, qk-norm ignores the slider: normalized queries and keys bound the logit, so it stays at about 1.4 whatever the raw scale.

The softcap at a raw max logit of 225: the solid bars bend toward the dashed line at 50, and the largest logit becomes 50.0. The second, 124 before the cap, becomes 49.3, so the gap between them shrinks from 101 to 0.7 and the top probability falls to 64.8%.

Labs stress-test stability on proxy models Read on your own

- Qwen3.8: hold the LR at 2–4× its optimum
- Count spikes, clip crossings, largest activations
- Qwen3.8: gated norm, 32 to 3.2 spikes per 10k steps (Qwen Team, 2026)
- Kolibri: QK-Clip spiked where qk-norm held (Aleph Alpha, 2026)

▼ **Proxy models expose instabilities before a run scales up**

Wortsman et al. showed that two instabilities of large runs, attention-logit growth and output-logit divergence, also appear in small models trained at high learning rates (Wortsman et al., 2024). Qwen3.8-Flash-Next uses that proxy to test a recipe before scaling it. It holds a mixture-of-experts model with 25B total and 3B active parameters at a constant 2× or 4× its optimal learning rate, without the usual decay. It counts loss spikes, the steps whose loss lies more than 0.1 above a 201-step rolling median. It also tracks the 99.9th percentile of the pre-clip gradient norm, how often the norm crosses the clip threshold of 0.5, and each block's largest activation (Qwen Team, 2026). At 4×, AdamW on the previous Qwen3.5 structure spiked 183 times per 10,000 steps, while Muon on the same structure never crossed the clip threshold in either of its two runs. In a pair of runs at 3× that differ only in a gated norm, a learned gate on the RMSNorm output, AdamW spiked 32.0 times per 10,000 steps without the gate and 3.2 times with it (Qwen Team, 2026).

Kolibri compared logit controls at its baseline learning rates, on a proxy with 7.8B total and 0.6B active parameters and grouped-query attention, one run per configuration over the first 53B tokens. With qk-norm, training stayed stable across the learning rates and batch sizes of its sweeps. Without qk-norm, the model diverged early and did not recover. A QK-Clip adapted to grouped-query attention, at $\tau = 100$ in place of qk-norm, still showed gradient-norm spikes. Without qk-norm, training was stable at a tenth of the learning rate, and its loss then stayed above that of the model with qk-norm throughout the run (Aleph Alpha, 2026). Your hot pair is a small test of the same kind.

Norm placement and training hygiene are stability levers Read on your own

(11) $$h = x + \mathrm{RMSNorm}(\mathrm{Attn}(x)) \qquad h_{\text{out}} = h + \mathrm{RMSNorm}(\mathrm{MLP}(h))$$

- OLMo 2: normalize each sublayer's output (OLMo Team, 2025)
- Reordering plus qk-norm: spike score from 0.108 to 0.069 (OLMo Team, 2025)
- Gemma: normalize before and after each sublayer (Gemma Team, 2024)
- OLMo 2: no weight decay on embeddings, ε = 1e-8 (OLMo Team, 2025)

▼ How to read the residual path

The position of a norm is a stability lever. OLMo 2 moves the norm from the input of each sublayer to its output (OLMo Team, 2025), which formula (11) writes as

$$h = x + \mathrm{RMSNorm}(\mathrm{Attn}(x)),$$
$$h_{\text{out}} = h + \mathrm{RMSNorm}(\mathrm{MLP}(h)).$$

Read it from the inside out: the first line normalizes the sublayer output $\mathrm{Attn}(x)$ before adding it to x, so the residual stream itself is not normalized, and the sum is h. The second line feeds that h into the MLP, normalizes the MLP's output and adds the result back to the same h. Read it as $\mathrm{MLP}(h)$, never as $\mathrm{MLP}(x)$.

OLMo 2 applied this reordering and qk-norm together, and the gradient-norm spike score fell from 0.108 to 0.069 against the pre-norm baseline (OLMo Team, 2025). The comparison supports the combined configuration in OLMo 2's setup. It does not separate the effect of the reordering from that of qk-norm, and it gives no general rule for where a norm belongs. The Gemma family instead normalizes both sides of each sublayer, before and after it, a sandwich norm (Gemma Team, 2024). Both are architecture-specific choices, and in both the residual stream still gives an additive path through the stack. A.X K2 replaced its predecessor's norms on both sides of each sublayer with Gated Norm, a learned gate on the RMSNorm output (Qwen3.8-Flash-Next measures such a gate in "Labs stress-test stability on proxy models"), at a cost of about 5% of training throughput. In an ablation on a mixture-of-experts model with 20B total and 3B active parameters, a further norm after the MLP added nothing once the gate was in place (Baek et al., 2026). Use the exact residual ordering a model names, and compare whole configurations instead of moving one norm on the strength of one report.

Two smaller choices also belong to OLMo 2's recipe. It excludes the embeddings from weight decay, and it sets AdamW's $\epsilon = 10^{-8}$ (OLMo Team, 2025). The course trainer differs on the

first: one `weight_decay` applies to every parameter group, embeddings and norm weights included.

Initialization sets how the residual stream grows Read on your own

- Week 2 set each layer's starting scale
- Each block adds to the residual stream, which grows (Xiong et al., 2020)
- GPT-2: residual projections scaled by 1/√N (Radford et al., 2019)
- OLMo 2: plain N(0, 0.02), fewer loss spikes (OLMo Team, 2025)

▼ **Initialization counters the residual stream's growth with depth**

Week 2's slide "Scaling keeps the softmax out of saturation" set each layer's starting scale at the course width $D = 512$. PyTorch's default for `nn.Linear` draws uniformly with variance $1/(3\,\mathrm{fan_in})$, a standard deviation of about 0.026. GPT-2 and nanoGPT draw most weights from a normal distribution with a fixed standard deviation of 0.02. That argument looked at one layer. Depth raises a second question, because every block adds its outputs to the residual stream.

How the stream grows

In a pre-norm block, the stream x passes through two additions:

$$x \leftarrow x + \mathrm{Attn}(\mathrm{RMSNorm}(x)),$$
$$x \leftarrow x + \mathrm{MLP}(\mathrm{RMSNorm}(x)).$$

Let x_0 be the stream entering the first block, f_i the i-th branch output added to it, and $x_i = x_{i-1} + f_i$ for $i = 1, \ldots, N$, where N is the number of residual additions, two per block.

- **A1. Uncorrelated additions.** Each addition is roughly uncorrelated with the stream it joins, $\mathbb{E}\langle x_{i-1}, f_i\rangle \approx 0$.
- **A2. Equal additions.** Each addition has about the same expected squared norm $\mathbb{E}\|f\|_2^2$, because the norm fixes the scale of every branch's input. Nothing fixes the scale of the stream.

Claim. Under A1 and A2, the stream's squared norm grows about linearly with the number of blocks, and its RMS about like the square root:

$$\begin{aligned}\mathbb{E}\|x_i\|_2^2 &= \mathbb{E}\|x_{i-1}\|_2^2 + 2\,\mathbb{E}\langle x_{i-1}, f_i\rangle + \mathbb{E}\|f_i\|_2^2 && \text{expand } \|x_{i-1} + f_i\|_2^2 \\ &\approx \mathbb{E}\|x_{i-1}\|_2^2 + \mathbb{E}\|f\|_2^2 && \text{by A1 and A2,}\end{aligned}$$

and after all N additions

$$\mathbb{E}\|x_N\|_2^2 \approx \mathbb{E}\|x_0\|_2^2 + N\,\mathbb{E}\|f\|_2^2.$$

Every addition contributes the same amount, so the squared norm rises in proportion to depth. Xiong et al. derive bounds on this growth in a simplified setting: uniform single-head attention, Xavier-initialized square weights, Gaussian inputs and a ReLU MLP. There, for the pre-norm Transformer, the expected squared norm of the hidden state x_ℓ after ℓ layers satisfies

$$(1 + \ell/2)\, D \le \mathbb{E}\|x_\ell\|_2^2 \le (1 + 3\ell/2)\, D,$$

where D is the model width (their Lemma 2) (Xiong et al., 2020). Xiong et al. count layers, not additions, so their x_ℓ follows ℓ blocks and is $x_{2\ell}$ in the indexing above.

GPT-2 scales with depth, OLMo 2 does not

GPT-2 counters the growth at initialization. In its own words: "A modified initialization which accounts for the accumulation on the residual path with model depth is used. We scale the weights of residual layers at initialization by a factor of $1/\sqrt{N}$ where N is the number of residual layers" (Radford et al., 2019).

Claim. Under A1 and A2, scaling a branch's output projection by $1/\sqrt{N}$ scales its variance by $1/N$, so N such additions add about as much as one unscaled addition:

$$\mathbb{E}\|x_N\|_2^2 \approx \mathbb{E}\|x_0\|_2^2 + N \cdot \frac{\mathbb{E}\|f\|_2^2}{N} = \mathbb{E}\|x_0\|_2^2 + \mathbb{E}\|f\|_2^2.$$

nanoGPT reads the rule as scaling the output projections of attention and of the MLP, the matrices that write into the stream, and counts two residual additions per block. With L blocks (nanoGPT's `n_layer`), $N = 2L$, and those weights get a standard deviation of $0.02/\sqrt{2L}$ (Karpathy, 2025).

OLMo 2 went the other way. It replaced its predecessor's depth-scaled initialization with one normal distribution, mean 0 and standard deviation 0.02, $\mathcal{N}(0, 0.02^2)$, for every parameter (OLMo Team, 2025). In a baseline run made prone to spikes, mainly by shortening warmup, the new initialization had no loss spikes, and the gradient-norm spike score fell from 0.40 to 0.03, in OLMo 2's setup only. It converged slightly more slowly (OLMo Team, 2025).

Two refinements concern the tails and the attention output. OLMoE found that a normal truncated at three standard deviations trained more stably and performed better than an untruncated one, a difference that became clear only near 450B tokens, where the untruncated run began to diverge (Muennighoff et al., 2025). OLMo 2's recipe also truncates (OLMo Team, 2025). MAI-Base-1, the base model of MAI-Thinking-1, follows GPT-2's scaling and also initializes the attention output to zero, through zero gains of the RMSNorm on that output. Its reason: a near-uniform softmax at initialization averages the visible tokens, which can imbalance the routing of the following mixture-of-experts layers, and MAI reports that the zero initialization cut that early imbalance (Microsoft AI Team, 2026).

The course model keeps the simple choices. Every linear layer keeps PyTorch's default initialization, except the output head: it is tied to the token embedding, which is drawn from

$\mathcal{N}(0, 0.02^2)$, so the head shares those weights. Every RMSNorm gain starts at 1, and no projection is scaled with depth.

50

Extreme values need different explanations Read on your own

- Massive activations: few huge, fixed-position values (Sun et al., 2024)
- Attention sinks: tokens drawing a large share of attention
- Logit growth: pre-softmax scores rising
- Reported together, but separate mechanisms

▼ Keep the observations separate

Three observations about extreme values sound alike and are often mentioned together. Describe each by the tensor it concerns before you propose a connection.

- **Hidden-state outliers** are single unusually large values in an intermediate hidden representation.
- **Massive activations** are a specific, named pattern of such values. Sun et al. report very few activations far larger than all others, for example 100,000 times larger, in a few fixed feature dimensions and at particular tokens such as the starting token (Sun et al., 2024). Not every hidden-state outlier is a massive activation.
- **Attention logits** are the pre-softmax query-key scores. Their growth during training is a numerical and optimization symptom, which can saturate the softmax or accompany divergence.
- **Attention sinks**, from Week 4, are tokens that receive a disproportionate share of attention after the softmax. They can support useful computation and matter for streaming, so the name alone does not mean a failure.

Sun et al. connect two of these themselves. In their account, these largely input-independent values act as bias terms the model needs, by concentrating attention on the tokens that carry them. When they trained GPT-2 with explicit attention biases added to self-attention, the massive activations did not appear (Sun et al., 2024). The paper makes no claim about training stability.

The observations can overlap in one run without being one mechanism. A link between them stays a hypothesis until measurements aligned on the same tensors, or a controlled intervention, support it. Activation outliers also matter for FP8 training (see "FP8 needs local scaling and high-precision islands").

Mixed Precision

51

A bf16 weight equals 1.0.

The optimizer adds 0.001.
What does bf16 store?

Answer A3

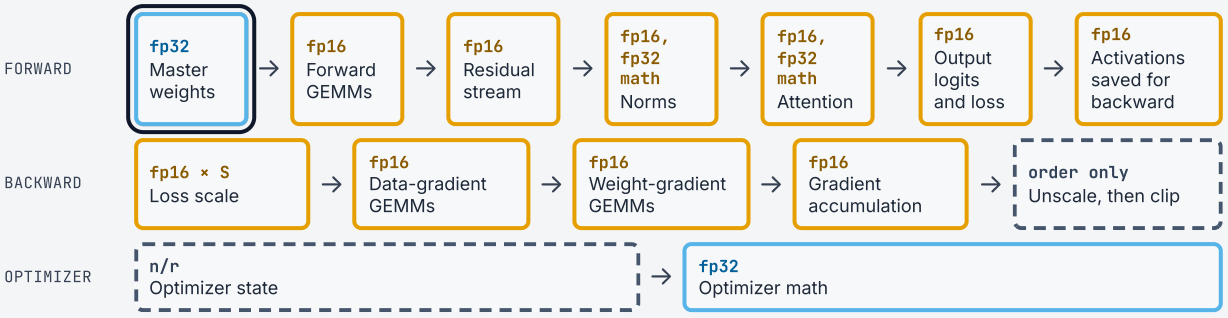
Mixed precision keeps selected operations in fp32

- Why not fp32: bf16 matmuls ~15× faster on an H100
- Matrix multiplications: bf16
- Master weights, optimizer state, gradient accumulation: fp32

Per tensor: what range, what precision?

Where each number format is used in one training step

fp16 + loss scaling (2018) | FP8: DeepSeek V3 / V4 | FP8: MAI-Thinking-1 | bf16: course trainer



Master weights, fp32. An update smaller than 2^{-24} becomes zero in fp16, and an update 2,048 times smaller than its weight is shifted out when the two are added. (§3.1)

Source: Micikevicius et al., 2018

fp32 | bf16 | fp16 | FP8 E4M3 | FP8 E5M2 | not reported or not used

Pick a recipe, then a box. Each box names the number format the recipe uses at that point of the step, and the line below gives the report's reason and its section. A dashed box means the report does not say, or the recipe does not use that step. A GEMM is a general matrix multiplication. The course trainer's formats come from its code under bf16 autocast.

▼ Each tensor gets the range and precision it needs

Why not train in fp32 throughout? Speed and memory. On an H100 SXM, the dense Tensor Core peak is 989.5 TFLOP/s in bf16 and 1,979 TFLOP/s in FP8, against 494.7 TFLOP/s in TF32 and 67 TFLOP/s for plain fp32 on the ordinary cores (NVIDIA, 2022). A 16-bit format also halves the bytes every weight and activation takes in memory and moves through HBM and between GPUs, which matters as much for the operations that Week 4 found bandwidth-bound. So the bulk of the arithmetic runs in bf16 or FP8, and fp32 stays only where a tensor needs its precision or range: the small updates of "Small updates vanish in a bf16 weight", and the sums and statistics below.

"Training in bf16" usually names the format of the dominant matrix multiplications, not one dtype for every operation. A mixed-precision recipe decides tensor by tensor: what range of values does this tensor need, and how much precision?

The formats answer the two questions with their bit fields, written sign/exponent/fraction. Exponent bits set the range, and fraction bits set the precision. The table lists positive finite values. The FP8 formats and their bit allocations come from (Micikevicius et al., 2022).

format	bits	maximum finite	minimum normal	minimum subnormal
fp32	1/8/23	3.4028235e38	1.1754944e-38	1.4012985e-45
fp16	1/5/10	65,504	6.1035156e-5	5.9604645e-8
bf16	1/8/7	3.3895314e38	1.1754944e-38	9.1835496e-41
E4M3 finite-only (E4M3FN)	1/4/3	448	0.015625	0.001953125
E5M2	1/5/2	57,344	6.1035156e-5	1.5258789e-5

The E4M3 row is the finite-only variant commonly named **E4M3FN**, whose maximum finite value is 448. Below the minimum normal, a format continues with **subnormal** values, evenly spaced down to the minimum subnormal. Under round-to-nearest, ties-to-even (RNE), a magnitude below half the minimum subnormal, about 3e-8 for fp16, rounds to zero.

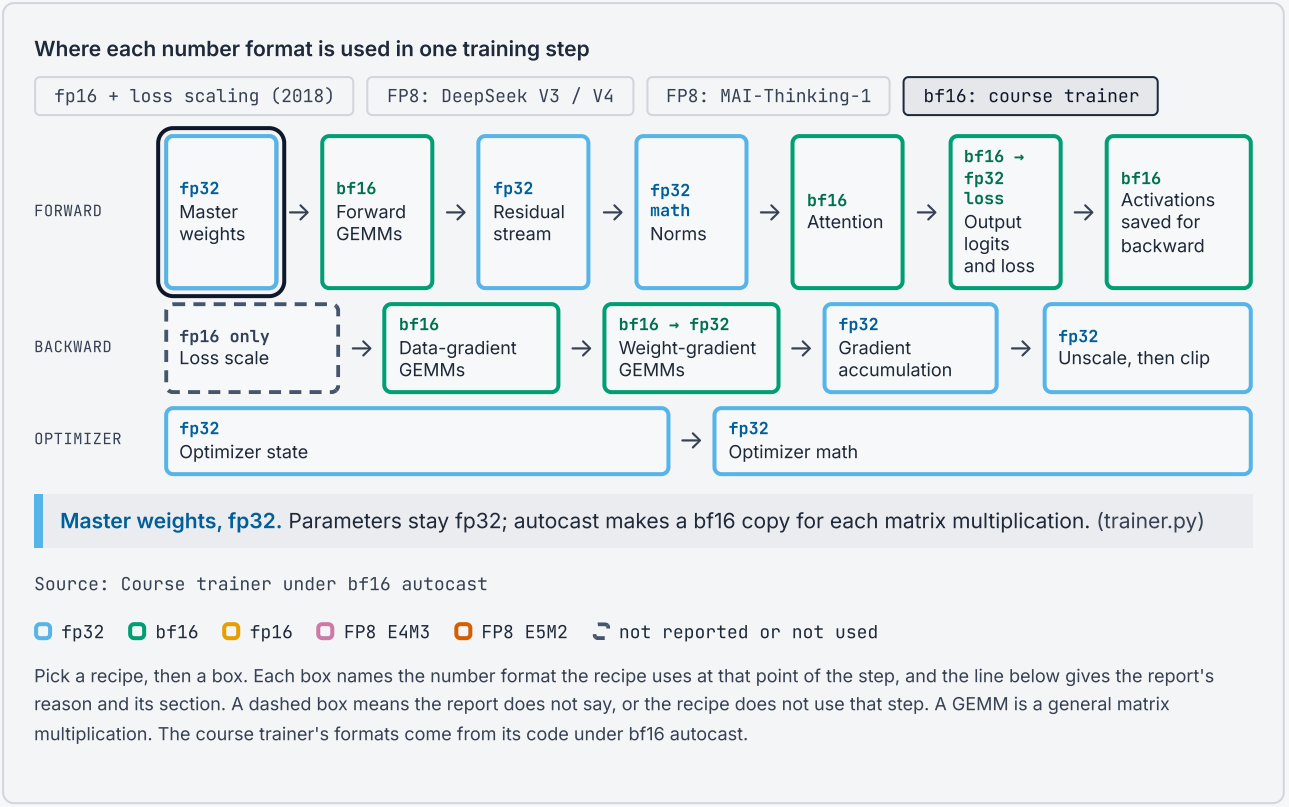
The matrix multiplications hold most of the arithmetic and commonly run in bf16, while the master weights and the optimizer state stay in fp32 to preserve small updates. **Parameter-gradient accumulation** sums the gradient contributions of several microbatches in fp32, so the stored gradient is not rounded again at every addition. Accumulation inside a matrix multiplication, where the GEMM sums its partial products in higher precision, is a separate choice and a separate summation problem.

The 16 B per parameter account of "Optimizer state is real memory" holds no accumulation buffer. A separate fp32 parameter-gradient accumulation buffer adds 4 B per parameter and must be counted explicitly.

Production recipes keep more than the optimizer in fp32. MAI-Thinking-1 lists its **fp32 islands**: the pre-softmax activations (attention scores, router logits and the final output logits), the residual stream end to end, the embedding weights, the norm weights, the router weights, and all optimizer state and AdamW computation (Microsoft AI Team, 2026). The router logits and weights belong to the mixture-of-experts routing of Week 13. Other models keep different tensors in fp32.

Besides its fp32 parameters, the course trainer makes three choices. The forward pass and the loss run under `torch.autocast`, which casts the operands of each matrix multiplication to bf16 in the course's bf16 runs. The cross-entropy is computed on logits cast to fp32. `RMSNorm` computes in fp32 and casts its result back to its input's dtype, which in the course model matters for qk-norm. The residual stream stays in fp32, because it starts as the fp32 token embedding and each block adds a bf16 branch output to it, so the block norms read and return fp32. The queries and keys, in contrast, come out of a bf16 projection. Without the cast back, their product with the fp32 norm weight would be fp32, and RoPE and the other operations before attention would run in fp32.

The widget lays out one training step in three lanes, forward, backward and optimizer, and colors each box by the format a recipe uses there, opening on the fp16 recipe of 2018 (Micikevicius et al., 2018). The master weights are fp32 in all four recipes, while the forward GEMMs move from fp16 to FP8 and, in the course trainer, to bf16. The optimizer state is where the two FP8 recipes part. DeepSeek V3 stores AdamW's first and second moments in bf16 and keeps the master weights and the gradients accumulated across the batch in fp32 (DeepSeek-AI, 2024b). MAI-Thinking-1 keeps its momentum buffers and all AdamW computation in fp32 (Microsoft AI Team, 2026).



The course trainer under bf16 autocast. Every matrix multiplication, forward and backward, runs in bf16, while the master weights, the residual stream, gradient accumulation, clipping and the optimizer state stay in fp32. The loss-scale box is dashed, because the trainer scales the loss only in fp16 runs.

Two 2026 instabilities came from bf16 gradient sums

Read on your own

- Nemotron 3 Ultra: bf16 output-layer gradient accumulation (NVIDIA, 2026)
- Multi-token prediction (MTP) gradients lost, MTP-2 loss spiked first
- Laguna M.1: bf16 output-head all-reduce, drifting logits (Poolside Team et al., 2026)
- Fix: fp32 sums again, Nemotron also rolled back

▼ A small gradient contribution can vanish in a bf16 sum

Two 2026 reports trace a training instability to a gradient sum kept in bf16. Nemotron 3 Ultra is a mixture-of-experts model with 550B total and 55B active parameters, trained on 20T tokens. NVIDIA attributes its first divergence, at about 8T tokens, to a change in the output layer. As a throughput optimization, the local gradient accumulation of that layer moved from fp32 to bf16, so that the data-parallel gradient reductions could travel in bf16. The model has two multi-token prediction (MTP) blocks, auxiliary blocks that predict tokens further ahead and add their gradients to the same output layer, with a loss weight of 0.05 each. Week 8 covers MTP. With bf16's 7 fraction bits, the report says, the MTP blocks' contribution to that layer's weight gradient was "essentially lost" (NVIDIA, 2026). The loss of the second MTP block, MTP-2, spiked before the training and validation losses did. Rolling back to an earlier checkpoint and returning to the fp32 gradient reduction stabilized the run again (NVIDIA, 2026).

Laguna M.1 is a mixture-of-experts model with 225.8B total and 23.4B active parameters. It used no z-loss, and its RMSNorm does not subtract the mean, so its output logits could grow freely, and they drifted strongly upward. As the activations grew, the bf16 all-reduce of the gradient with respect to the input of the output head, the language-model head, became "the dominant source of numerical error". That error flowed back into the rest of the model and destabilized training. Poolside forced that all-reduce to fp32 (Poolside Team et al., 2026).

Nemotron's second divergence, at about 16T tokens, has no known cause. The run was pretrained in NVFP4, a 4-bit format, and switching all tensors to bf16 did not remove the divergence. Starting the learning-rate decay early, right after a rollback, mitigated it (NVIDIA, 2026). Each account is one run, reported by its own lab.

bf16 usually avoids fp16's range-driven loss scaling

- Same 16 bits: fp16 favors precision, bf16 range
- fp16: tiny gradients round to zero
- Scale the loss, then unscale before clipping
- Dynamic scaler: double after 2,000 finite steps
- Overflow: skip the step, halve the scale

▼ **Loss scaling lifts small gradients into fp16's range**

fp16 and bf16 split the same 16 bits differently. fp16 spends more of them on the fraction (10 bits against 7), so it is more precise. bf16 spends more on the exponent (8 bits against 5), so it has fp32's range. fp16's narrow range creates an **underflow cliff**: a gradient below fp16's round-to-zero boundary of about 2.98e-8 is stored as zero, and its contribution to the update is lost.

The classic fix is **loss scaling**: multiply the loss $\mathcal{L}$ by a scale s before the backward pass (Micikevicius et al., 2018). The backward pass then starts from s instead of 1, so by the chain rule every gradient it computes is multiplied by s too,

$$\nabla_\theta(s\,\mathcal{L}) = s\,\nabla_\theta\mathcal{L},$$

which shifts small gradients up into fp16's representable range. Before the gradients are used, they are divided by s again. A gradient of 10^{-9} stored directly in fp16 rounds to zero.

Worked example. With $s = 2^{16} = 65{,}536$:

$$\begin{aligned} 10^{-9} \times 2^{16} &= 6.5536 \times 10^{-5} && \text{scale} \\ &\to 1100 \times 2^{-24} \approx 6.5565 \times 10^{-5} && \text{round to fp16} \\ \left(1100 \times 2^{-24}\right)/2^{16} &\approx 1.0004 \times 10^{-9} && \text{unscale in fp32.} \end{aligned}$$

The scaled value lies just above fp16's minimum normal, 6.1035×10^{-5}. In that part of the normal range, 10 fraction bits give a spacing of 2^{-24}, so fp16 cannot store the value exactly. Scaling keeps the gradient from rounding to zero. The round trip is close but not exact.

Micikevicius et al. used a constant scale, between 8 and 32K for their networks, chosen so that the largest scaled gradient stays below fp16's maximum of 65,504. They unscale right after the backward pass, before clipping, and skip a step whose weight gradients overflow (Micikevicius et al., 2018).

A dynamic scaler adjusts s during training, and the course scaler is one. Its state is the current s and a count of consecutive finite steps, and it starts at $s = 2^{16}$. On each step it scales the loss, runs the backward pass, unscales the gradients by $1/s$ and checks that their norm is finite. **Unscale before clipping**, so that the clipping threshold keeps its units. The scale then follows one rule:

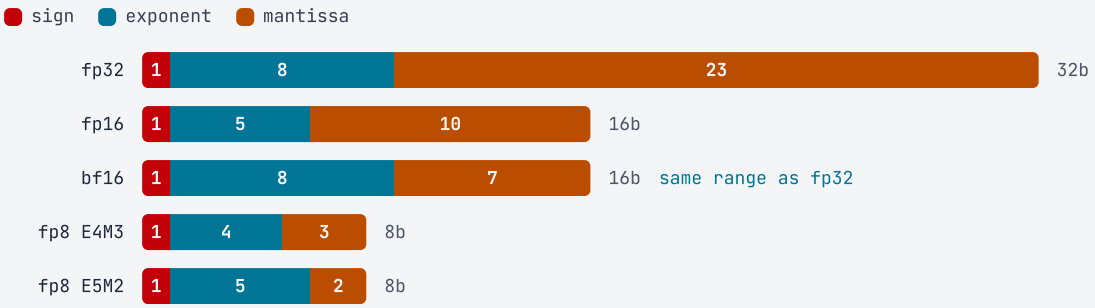
$$s \leftarrow \begin{cases} s/2 & \text{on an inf or a NaN: the step is skipped} \\ 2s & \text{after 2,000 finite steps in a row} \\ s & \text{otherwise.} \end{cases}$$

On the finite path the trainer clips, takes the optimizer and scheduler steps, and counts the step. On an overflow it skips the optimizer and scheduler steps, halves s, restarts the count and logs the skipped step with its scale. The weights, the optimizer moments and the step counts stay as they were. A skipped step consumes no step of the budget but still counts its batch as drawn. After 32 consecutive skips the trainer raises an error that names the scale. With `loss_scale: static` the scale never moves, and the configuration accepts a loss scale only for fp16.

bf16 keeps fp32's 8-bit exponent field, so it usually avoids the range-driven loss scaling that fp16 needs (Kalamkar et al., 2019). It is not infallible: values can still overflow, and numerically sensitive operations such as softmax and normalization can still need higher precision. Monitor for non-finite values in either format.

Watch loss scaling move gradients into range

- Raise the scale: less mass underflows
- bf16 moves the underflow threshold far off the axis

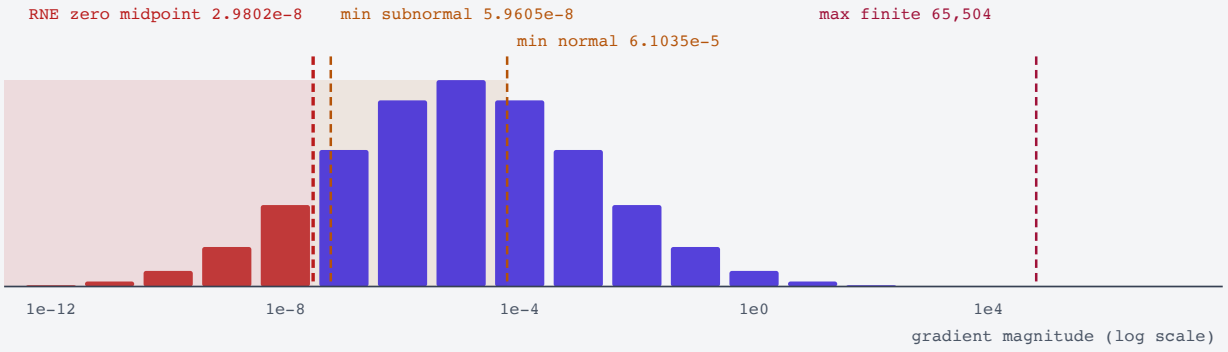


ILLUSTRATIVE Gradient magnitudes vs. the representable range

fp16 bf16 loss scale $\times 2^{0} = \times 1$

12.6% of gradient mass falls at or below the round-to-zero midpoint

0.0% of gradient mass lies above the max finite value



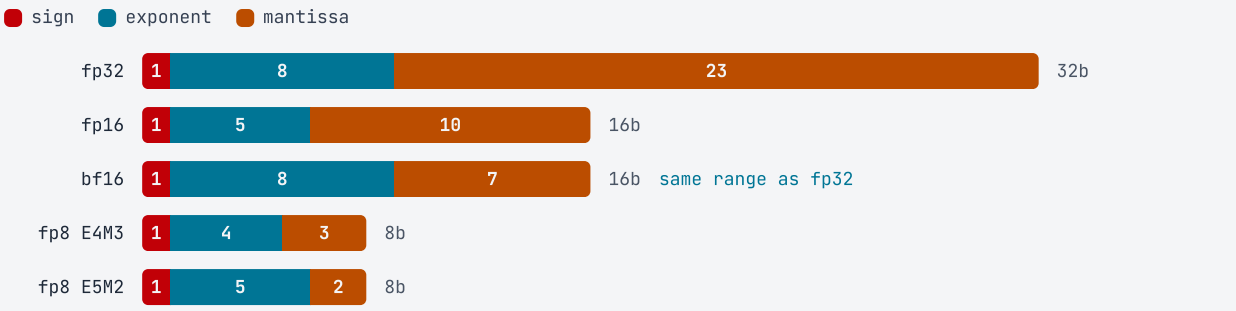
fp16 minimum subnormal 5.9605e-8; fp16 minimum normal 6.1035e-5.
bf16 minimum subnormal 9.1835e-41; bf16 minimum normal 1.1755e-38.
Round-to-nearest, ties-to-even zero midpoint: fp16 2.9802e-8; bf16 4.5918e-41. The exact midpoint tie rounds to zero; magnitudes above it can round to the minimum subnormal.
Hardware flush-to-zero modes may instead discard the full subnormal range. Loss scaling shifts gradients before backward and unscales them before clipping and the optimizer step.

▼ Reading the loss-scaling panel

The widget has two parts. On the left, the bit layouts of the five formats are drawn to scale, so you can see that bf16 shares fp32's exponent width. On the right, a synthetic histogram of gradient magnitudes, a bell shape in log space peaking near 10^{-5}, spans 10^{-12} to 10^{2} on a logarithmic axis. Dashed lines mark the chosen format's thresholds: at the low end the RNE round-to-zero midpoint, the minimum subnormal and the minimum normal, and for fp16 at the high end its maximum finite value, 65,504. Bars at or below the midpoint turn red, bars above the maximum take the overflow color, and two readouts give each group's share of the mass.

The loss-scale slider runs from $s = 2^0$ to $s = 2^{16}$. Each doubling shifts every bar right by $\log_{10} 2 \approx 0.3$ decades, since $\log_{10}(2x) = \log_{10} x + \log_{10} 2$, while the thresholds stay where they are. The scale moves the gradients relative to fixed thresholds, so the red share falls as the scale grows. In fp16 about 12.6% of the illustrative mass rounds to zero at a scale of 1, and none from 2^{15} on. The same shift moves the top of the histogram toward fp16's maximum: up to 2^9 no mass lies above 65,504, from 2^{10} on the overflow readout is no longer zero, and at 2^{16} it reads about 1.9%.

The format toggle does the opposite. Switching to bf16 leaves the histogram where it was and moves the thresholds. bf16's round-to-zero midpoint, about 4.6e-41, lies more than 26 decades left of the axis, and its maximum, about 3.4e38, lies far to the right. No illustrated gradient underflows or overflows at any scale. The panel models these two range limits only.

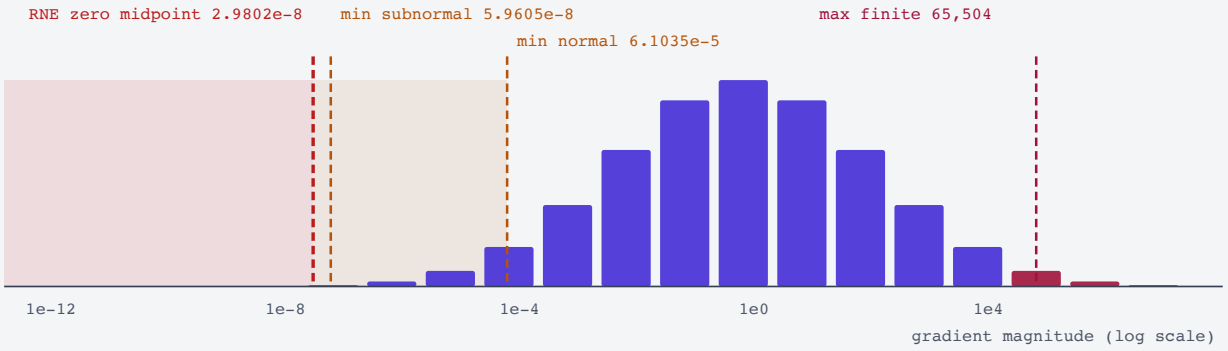


ILLUSTRATIVE Gradient magnitudes vs. the representable range

fp16 bf16 loss scale $\times 2^{16} = \times 65{,}536$

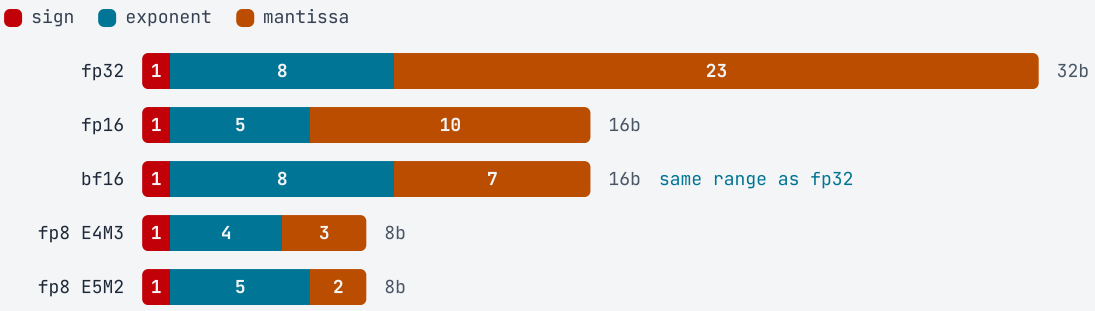
0.0% of gradient mass falls at or below the round-to-zero midpoint

1.9% of gradient mass lies above the max finite value



fp16 minimum subnormal 5.9605e-8; fp16 minimum normal 6.1035e-5.
bf16 minimum subnormal 9.1835e-41; bf16 minimum normal 1.1755e-38.
Round-to-nearest, ties-to-even zero midpoint: fp16 2.9802e-8; bf16 4.5918e-41. The exact midpoint tie rounds to zero; magnitudes above it can round to the minimum subnormal.
Hardware flush-to-zero modes may instead discard the full subnormal range. Loss scaling shifts gradients before backward and unscales them before clipping and the optimizer step.

fp16 at a loss scale of 2^{16} = 65,536, the course scaler's starting value. Every bar lies about 4.8 decades further right than in the opening state, and the thresholds stay where they were. No mass rounds to zero now, but 1.9% lies above fp16's maximum of 65,504 and would overflow.



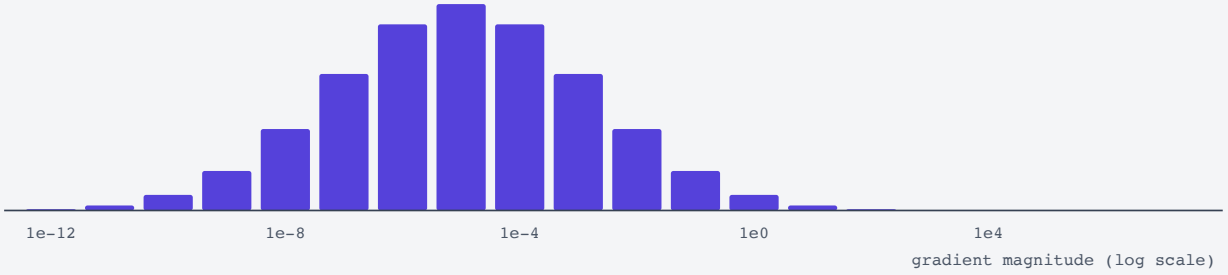
ILLUSTRATIVE Gradient magnitudes vs. the representable range

fp16 bf16 loss scale $\times 2^0 = \times 1$

0.0% of gradient mass falls at or below the round-to-zero midpoint

0.0% of gradient mass lies above the max finite value

bf16 thresholds lie far off this axis: minimum normal 1.2e-38, maximum 3.4e38



fp16 minimum subnormal 5.9605e-8; fp16 minimum normal 6.1035e-5.
bf16 minimum subnormal 9.1835e-41; bf16 minimum normal 1.1755e-38.
Round-to-nearest, ties-to-even zero midpoint: fp16 2.9802e-8; bf16 4.5918e-41. The exact midpoint tie rounds to zero; magnitudes above it can round to the minimum subnormal.
Hardware flush-to-zero modes may instead discard the full subnormal range. Loss scaling shifts gradients before backward and unscales them before clipping and the optimizer step.

bf16 at a loss scale of 1: the histogram has not moved, and both readouts show 0.0%. Where fp16 lost 12.6% of the mass at this scale, bf16 loses none, because its round-to-zero midpoint, about 4.6e-41, and its maximum, about 3.4e38, lie far off both ends of the axis.

FP8 needs local scaling and high-precision islands

- DeepSeek V3: FP8 matrix multiplications, forward and backward (DeepSeek-AI, 2024b)
- Scales per 1×128 activation tile and 128×128 weight block
- Partial sums promoted to fp32 every 128 elements
- Embedding, head, norms and attention stay in bf16 or fp32
- MAI-Base-2, A.X K2: microscaling FP8 (MXFP8), 32-value blocks (Microsoft AI, 2026) (Baek et al., 2026)
- Local scales absorb outliers

▼ Why FP8 wants per-tile scales

FP8 has 8 bits and a narrow range: E4M3's positive finite values span from about 0.002 to 448, so every tensor must be scaled into it. DeepSeek V3 runs the forward and backward GEMMs in FP8 and manages the precision with local scales (DeepSeek-AI, 2024b). Activations are scaled per 1×128 **tile**, one token by 128 channels, and weights per 128×128 **block**, 128 input by 128 output channels. The tensor cores accumulate a GEMM's partial products in limited precision, so after every 128 elements, an interval specific to that report, the partial results are promoted to fp32 and summed there. The embedding, the output head, the mixture-of-experts gating, the normalization and the attention operators stay in bf16 or fp32 (DeepSeek-AI, 2024b). These are higher-precision islands one level down.

DeepSeek V3 explains why local scales help. Low-precision training "is often limited by the presence of outliers in activations, weights, and gradients" (DeepSeek-AI, 2024b), a point it supports by citing Sun et al. among others (Sun et al., 2024). The standard practice scales a whole tensor so that its largest absolute value maps to FP8's largest value, and this "makes low-precision training highly sensitive to activation outliers" (DeepSeek-AI, 2024b).

Use one divisor convention for a tile or block b with scale s_b: quantize a value x to the code $q = x/s_b$ and dequantize with $\hat{x} = s_b\, q$. The scale maps the block's largest magnitude (amax) onto E4M3's maximum finite value:

$$s_b = \frac{\max_{x \in b} |x|}{448}.$$

Worked example. A tile whose amax is 896 gets $s_b = 896/448 = 2$. An ordinary value $x = 6$ becomes the code $q = 6/2 = 3$, and dequantization gives back $\hat{x} = 2 \times 3 = 6$.

Claim. A value's code depends on the largest magnitude in its own block only:

$$|q| = \frac{|x|}{s_b} = 448\, \frac{|x|}{\max_{x' \in b} |x'|}.$$

With one scale per tensor, b is the whole tensor. One outlier then sets the scale for every value in the tensor and pushes the ordinary codes toward the bottom of the range. With 1×128 tiles, an outlier sets the scale of its own tile only.

Some libraries call the reciprocal $1/s_b$ the "scale" and compute $q = x \times (1/s_b)$, so read an API's quantization and dequantization equations before you port a checkpoint.

Outliers can also appear late in a run. Fishman et al. trained Llama 2 7B in FP8 on up to 2 trillion tokens and found that outliers at the input of the MLP's last linear layer, the SwiGLU product, "emerge only after processing approximately 200 billion tokens" (Fishman et al., 2025). With the quantization of that one layer's input disabled, FP8 training converged on the long runs where it had diverged (Fishman et al., 2025).

MAI-Thinking-1 varies the recipe (Microsoft AI Team, 2026). It uses **E4M3** for the forward GEMMs and the wider-range **E5M2** for the GEMMs that compute data gradients, and it keeps the weight-gradient GEMMs in bf16 with fp32 accumulation. It takes delayed scales from a 1,024-step history of each tensor's maximum and rounds stochastically wherever gradients are cast down.

Microsoft AI has since moved on from that recipe. Its MAI-Base-2 report lists tensor-wise delayed scaling with E4M3 and E5M2 for MAI-Base-1, the base model of MAI-Thinking-1, and "MXFP8 E4M3" for MAI-Base-2 (Microsoft AI, 2026). A.X K2 trains in MXFP8, microscaling FP8, a format in which each block of 32 consecutive values shares one scale. It uses E4M3 in both passes and keeps its main parameters and gradients in fp32. Its reason is the same: block-wise scaling "better accommodates outliers in activations and gradients" (Baek et al., 2026).

Local FP8 scaling and loss scaling both manage range, at different scopes: loss scaling applies one factor s to every gradient, FP8 scaling one factor s_b to each tile or block of a GEMM operand. Neither DeepSeek V3 nor MAI-Thinking-1 compares FP8 with bf16 in speed or quality under matched conditions. Treat FP8 as a recipe made tensor by tensor: check each scale's orientation, each accumulation boundary and each higher-precision island.

DeepSeek V4 also reports FP8 storage for selected dimensions of its KV cache and FP4 quantization of its expert weights (DeepSeek-AI, 2026a). Those are later choices for storage and inference, separate from V3's recipe for the training GEMMs.

Synthesis

57

The opening claim, decoded

K2: 15.5T tokens, zero loss spike

- Optimizer: MuonClip (Muon, weight decay, RMS matching, QK-Clip)
- Schedule: WSD, 10T tokens stable, 5.5T decay
- Monitoring: per-head max logit, every step
- Control: QK-Clip at $\tau = 100$
- Precision: bf16 weights, fp32 gradient accumulation

▼ K2's claim, read choice by choice

The K2 report states that Kimi K2 was pre-trained on 15.5 trillion tokens with zero loss spike (Kimi Team, 2025).

Optimizer. MuonClip, K2's name for Muon with weight decay, RMS matching and QK-Clip in one optimizer (see "QK-Clip rescales only the offending head") (Kimi Team, 2025). Weight decay and the shape rescale to a common update RMS keep Muon stable at scale, and QK-Clip bounds each head's attention logits.

Schedule. WSD: after a 500-step warmup, 2e-4 for 10T tokens, then a cosine decay to 2e-5 over 5.5T, at a constant global batch of 67M tokens (Kimi Team, 2025).

Monitoring. QK-Clip needs each head's max logit at every step, so K2 measures it throughout the run, and a claim of zero loss spikes can only be checked against such traces. The course trainer logs `max_attn_logit`, the largest masked logit over all heads and layers, on one step axis with the loss and the other signals. Its function `max_attention_logit` computes it in fp32 with autocast off, on the first sequence of each logged step only, so the value samples the batch, whereas K2's per-head clip covers every sequence.

Control. QK-Clip at $\tau = 100$, which acted only early, on 12.7% of the heads in the first 70,000 steps (Kimi Team, 2025).

Precision. bf16 parameters with an fp32 gradient-accumulation buffer (Kimi Team, 2025).

What changed between the two runs? K2 attributes the difference to MuonClip: with plain Muon, the 53B test run's attention logits passed 1,000, and in the main run QK-Clip held each head's max logit $S^h_{\max}$ at the threshold τ or below (Kimi Team, 2025). The runs also differ in scale, 53B total parameters against 1T, and the report does not give the test run's batch size, schedule or precision, so it cannot show that the clip alone made the difference. It does show that the main run kept in range the quantity the test run lost control of.

The five choices act as one training system, and K2's recipe is one way of setting all five.

Four labs combine the same five choices differently

Read on your own

	DeepSeek V4	Kimi K2	OLMo 3	MAI-Thinking-1
Optimizer	Muon for most parameters; AdamW for selected groups	MuonClip (Muon with weight decay, RMS matching and QK-Clip)	AdamW, with OLMo 2's settings	AdamW (beta1 .95, beta2 .925; wd .1/.01/.005 by group)
LR schedule	2,000-step warmup, constant phase, cosine decay	WSD: 500-step warmup; 2e-4 for 10T; cosine decay to 2e-5 over 5.5T	pretraining: modified cosine (7B), truncated cosine (32B)	pretraining: ~12B-token warmup, cosine to 10% of peak
Batch schedule	Flash ramps to 75.5M; Pro reports a 94.4M-token maximum	constant 67M-token global batch	pretraining: 4.19M (7B) / 8.39M (32B) tokens	134M tokens in every phase
Stability	query/KV-entry RMSNorm; SwiGLU clamping	per-head QK-Clip, $\tau = 100$ in K2	qk-norm, z-loss 1e-5, output RMSNorm, gradient clipping 1.0	zero-initialized attention outputs; model-specific fp32 islands
Precision	selected bf16/fp32 training operations; bf16/FP8 KV storage; post-training FP4 quantization-aware training (QAT)	bf16 parameter storage; fp32 gradient-accumulation buffer	bf16 throughout base-model training	bf16 base; E4M3 forward, E5M2 data-gradient; delayed scaling; stochastic rounding

▼ Reading the four recipes by row and by column

Each column is the recipe one lab reports for its own model, and the column header's citation covers all five cells below it, numbers included: DeepSeek V4 (DeepSeek-AI, 2026a), Kimi K2

(Kimi Team, 2025), OLMo 3 (Ettinger et al., 2025) and MAI-Thinking-1 (Microsoft AI Team, 2026).

Read a row to see how the reports differ on one choice. Compare only what the four reports disclose for that row. The batch row mixes a ramp, a constant batch, size-specific batches and one value for all phases, so its four values cannot be compared directly. The stability row shows where each system intervenes. The precision row depends most on the stage of training: DeepSeek V4 separates selected training operations, the storage of its KV entries and quantization-aware training (QAT) after pretraining, while the other columns describe different operations or stages.

Read a column to see how one lab's choices work together in one system. Stay within the column, and do not splice cells from different labs into one recipe. A column shows what one lab combined, but it does not show that any single cell caused that lab's outcome.

- **DeepSeek V4:** Muon with AdamW for selected groups; 2,000 warmup steps, a constant phase and a cosine decay; a batch ramp to 75.5M tokens (Flash) and a 94.4M-token maximum (Pro); RMSNorm on queries and KV entries, and SwiGLU clamping; selected bf16 and fp32 operations, bf16 or FP8 KV storage, and FP4 QAT after pretraining.
- **Kimi K2:** MuonClip, WSD, a constant 67M-token batch, per-head QK-Clip, and bf16 parameters with fp32 gradient accumulation.
- **OLMo 3:** AdamW with OLMo 2's settings; size-specific cosine schedules; 4.19M- and 8.39M-token batches; qk-norm, z-loss at 1e-5, output RMSNorm and clipping at 1.0; bf16 throughout base-model training.
- **MAI-Thinking-1:** AdamW; about 12B warmup tokens, then cosine decay; a 134M-token batch in every phase; zero-initialized attention outputs and fp32 islands; E4M3 forward and E5M2 data-gradient FP8 with delayed scaling and stochastic rounding.

Where a report leaves a setting out, that does not mean the setting is absent. Any recipe taken from the table still has to be validated on the run you are planning.

Summary

59

Six ways to keep a long run stable

1. AdamW: the tuned baseline to beat
2. Muon: shape-scaled updates, weight decay, bounded logits
3. Schedules: warmup, WSD decay legs, a batch ramp
4. Monitoring: pre-clip norm, max logit, spike score
5. Controls: each at its own tensor path
6. Precision: bf16 compute, fp32 master weights, local FP8 scales

▼ The answer to today's question

A long training run stays stable through six choices made together and measured against a baseline. The six split the optimizer choice in two, AdamW as the baseline and Muon for the matrix parameters, and add the schedules, monitoring, controls and precision.

AdamW is the tuned baseline to beat. Its practical starting point is $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 0.1 and clipping at 1.0, and a new optimizer has to beat it under the same tuning budget.

Muon changes how matrix parameters are updated. In hybrid recipes such as DeepSeek V4's and nanochat's, Muon takes the hidden matrices and AdamW the rest (DeepSeek-AI, 2026a) (Karpathy, 2026). At scale it needs the shape rescale, weight decay and bounded attention logits, and how much it helps depends on the scale, the tuning and the budget of the comparison.

The learning-rate and batch schedules spend the budget together. Warmup protects the first steps, WSD lets a decay leg branch off whenever a finished checkpoint is needed, and a growing batch trades gradient noise against throughput.

Monitoring locates an anomaly before you intervene. Read the loss, the pre-clip gradient norm, the learning rate, the throughput and the max logit on one step axis, and summarize spikes with a spike score. Which signal moves first depends on the failure.

Each control acts at its own place on the tensor path. QK-Clip, qk-norm, softcapping, SwiGLU clamping, z-loss, gradient clipping, norm placement and data hygiene each act on a different tensor or at a different time in the step, and they can interact.

Mixed precision is chosen per tensor: bf16 matrix multiplications, fp32 master weights, accumulators and selected islands, dynamic loss scaling for fp16, and local scales for FP8.

Distributed training must partition the optimizer state and the gradients so that the full training state fits across devices. Week 6 turns optimizer state into distributed state, and asks how to move it without changing the optimization.

60

To-dos until next week



Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes
- Read Bonus 3 before the Week 6 tutorial

Deadlines

- Quiz 2: Friday 13 November
- Fourth exercise: Wednesday 18 November, 22:00

Week 6 turns optimizer state into distributed state, and asks how to move it without changing the optimization.

Reflection questions

1. You are choosing between AdamW and a Muon+AdamW hybrid for a bf16 pretraining run with fp32 optimizer state. For each eligible matrix parameter, how many optimizer-state bytes does each choice require, and which other memory categories must remain unchanged in your comparison?
2. A WSD run reaches the end of its stable phase, and you need checkpoints adapted to two different token budgets without repeating that phase. Which checkpoints and schedule states would you save, where would you branch the decay legs, and what comparison would reveal the trade-off against separate cosine runs?
3. Two models show growing attention logits: one can modify its attention computation, while the other can intervene only through optimizer-side weight updates or the training loss. For each model, where would you place a stability control, what signal would it act on, and what limitation would you record?
4. A pretraining run produces gradients near `1e-8` and occasional large activations. Compare bf16 and fp16 for this run: which range or precision property drives your format choice, when would dynamic loss scaling be required, and which operations or states would you keep in fp32?
5. Run A shows an isolated loss spike with no preceding rise in pre-clip gradient norm. Run B shows pre-clip gradient norm rising for hours while loss remains smooth. What evidence would you inspect next for each run, and which interventions would you test without claiming the trace proves a cause?
6. Your small-vocabulary course model uses AdamW with `beta2=0.999`, and its second-moment estimate reacts slowly to rare large gradients. What change to `beta2` would you test, what response do you predict, and which logged comparison would distinguish faster adaptation from added noise?

References

62

References: Adam and Muon foundations

- Bernstein, J., & Newhouse, L. (2024). Old Optimizer, New Norm: An Anthology. arXiv preprint arXiv:2409.20325.
- Jordan, K., et al. (2024). Muon: An optimizer for hidden layers in neural networks. Blog post, December 2024.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. International Conference on Learning Representations (ICLR).
- Kosson, A., Welborn, J., Liu, Y., Jaggi, M., & Chen, X. (2026). Weight Decay may matter more than μP for Learning Rate Transfer in Practice. *International Conference on Learning Representations (ICLR 2026)*.
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. International Conference on Learning Representations (ICLR 2019).
- Shumaylov, Z., Da Costa, N., Zaika, P., Mucsányi, B., Massucco, A., Gelberg, Y., Schönlieb, C.-B., Gal, Y., & Hennig, P. (2026). Muon is Not That Special: Random or Inverted Spectra Work Just as Well. arXiv preprint arXiv:2605.11181.
- Wang, X., & Aitchison, L. (2025). How to set AdamW's weight decay as you scale model and dataset size. *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), PMLR 267*.

63

References: Frontier optimizer reports

- DeepSeek-AI (2026a). DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv preprint arXiv:2606.19348.
- DeepSeek-AI (2026b). DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. arXiv preprint arXiv:2609.19969.
- GLM Team. (2026). GLM-5: From Vibe Coding to Agentic Engineering. Technical report.
- Huang, A., Li, A., Kong, A., et al. (2026). Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters. arXiv preprint arXiv:2602.10604.
- Kimi Team (2025). Kimi K2: Open Agentic Intelligence. arXiv preprint arXiv:2507.20534.
- Kimi Team (2026). Kimi K3: Open Frontier Intelligence. arXiv preprint arXiv:2607.24653.
- Liu, J., et al. (2025). Muon is Scalable for LLM Training. arXiv preprint arXiv:2502.16982.
- Mistral AI (2026). Mistral Large 4. Technical report, Mistral AI.
- Poolside Team, Abadji, J., et al. (2026). Laguna M.1/XS.2 Technical Report. arXiv preprint arXiv:2605.27605.
- Qwen Team (2026). On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability. Technical report, GitHub, 26 August 2026.
- Xiaomi LLM-Core Team (2026). MiMo-V2.6: Scaling Reinforcement Learning Towards Self-Improvement. arXiv preprint arXiv:2610.11959.

References: Optimizer benchmarking

- Dahl, G. E., et al. (2023). Benchmarking Neural Network Training Algorithms. arXiv preprint arXiv:2306.07179.
- Defazio, A., et al. (2024). The Road Less Scheduled. arXiv preprint arXiv:2405.15682.
- Jordan, K., et al. (2026). modded-nanoGPT track-3 optimization record logs: AdamW record and Muon record.
- Kasimbeg, P., et al. (2025). Accelerating Neural Network Training: An Analysis of the AlgoPerf Competition. International Conference on Learning Representations (ICLR 2025).
- Schmidt, R. M., Schneider, F., & Hennig, P. (2021). Descending through a Crowded Valley - Benchmarking Deep Learning Optimizers. Proceedings of the 38th International Conference on Machine Learning (ICML 2021).
- Wen, K., Hall, D., Ma, T., & Liang, P. (2026). Fantastic Pretraining Optimizers and Where to Find Them. International Conference on Learning Representations (ICLR 2026).

References: Learning-rate schedules

- DeepSeek-AI (2024a). DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. arXiv preprint arXiv:2401.02954.
- Hu, S., et al. (2024). MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies. arXiv preprint arXiv:2404.06395.
- Ling Team (2026). Ling and Ring 2.6 Technical Report: Efficient and Instant Agentic Intelligence at Trillion-Parameter Scale. arXiv preprint arXiv:2606.15079.

References: Batch size and post-trainability

- Maskey, S., Scholl, P., Knupp, J., Neitemeier, P., & Wirges, S. (2026). Good Pretraining, Bad SFT: Checkpoint Quality Across the Training Stack. arXiv preprint arXiv:2609.08966.
- McCandlish, S., Kaplan, J., Amodei, D., et al. (2018). An Empirical Model of Large-Batch Training. arXiv preprint arXiv:1812.06162.
- Merrill, W., Arora, S., Groeneveld, D., & Hajishirzi, H. (2025). Critical Batch Size Revisited: A Simple Empirical Approach to Large-Batch Language Model Training. Advances in Neural Information Processing Systems 38 (NeurIPS 2025).
- Yano, K., Kiyono, S., Kobayashi, S., Takase, S., & Suzuki, J. (2026). Pre-training LLM without Learning Rate Decay Enhances Supervised Fine-Tuning. International Conference on Learning Representations (ICLR 2026).

References: Stability foundations

- Altenbernd, A., Wiesner, P., & Kao, O. (2026). Exploring Silent Data Corruption as a Reliability Challenge in LLM Training. IEEE International Symposium on Cluster, Cloud, and Internet Computing (CCGrid 2026).
- Chowdhery, A., Narang, S., Devlin, J., et al. (2023). PaLM: Scaling Language Modeling with Pathways. Journal of Machine Learning Research, 24(240).
- Dehghani, M., et al. (2023). Scaling Vision Transformers to 22 Billion Parameters. Proceedings of the 40th International Conference on Machine Learning (ICML 2023).
- Muennighoff, N., Soldaini, L., Groeneveld, D., et al. (2025). OLMoE: Open Mixture-of-Experts Language Models. International Conference on Learning Representations (ICLR 2025).
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. Proceedings of the 30th International Conference on Machine Learning (ICML 2013).
- Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI.
- Sun, M., Chen, X., Kolter, J. Z., & Liu, Z. (2024). Massive Activations in Large Language Models. Conference on Language Modeling (COLM 2024).
- Wortsman, M., Liu, P. J., Xiao, L., et al. (2024). Small-scale proxies for large-scale Transformer training instabilities. International Conference on Learning Representations (ICLR 2024).
- Xiong, R., Yang, Y., He, D., et al. (2020). On Layer Normalization in the Transformer Architecture. ICML 2020.

References: Stability model reports

- Aleph Alpha (2026). Kolibri: A Sovereign European Model on the Pareto Frontier. arXiv preprint arXiv:2610.31945.
- Ettinger, A., Bertsch, A., Kuehl, B., et al. (2025). Olmo 3. arXiv preprint arXiv:2512.13961.
- Gemma Team (2024). Gemma 2: Improving Open Language Models at a Practical Size. arXiv preprint arXiv:2408.00118.
- Gemma Team (2025). Gemma 3 Technical Report. arXiv preprint arXiv:2503.19786.
- Google DeepMind (2026). Gemma 4 31B: model card and configuration. Hugging Face model repository.
- K2 Team (2025). K2-V2: A 360-Open, Reasoning-Enhanced LLM. arXiv preprint arXiv:2512.06201.
- Merrill, W., et al. (2026). Olmo Hybrid: From Theory to Practice and Back. arXiv preprint arXiv:2604.03444.
- Meta Superintelligence Labs (2026). Muse Spark Open 816B-A42B: model card and configuration. Hugging Face model repository.
- Microsoft AI (2026). Frontier Pre-training at Microsoft AI. Technical report, Microsoft AI.
- NVIDIA (2026). Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. arXiv preprint arXiv:2606.15007.
- OLMo Team (2025). 2 OLMo 2 Furious. arXiv preprint arXiv:2501.00656.
- OpenAI (2025). gpt-oss-120b & gpt-oss-20b Model Card. arXiv preprint arXiv:2508.10925.

References: Precision

- Baek, C., Arya, D., Kim, E., et al. (2026). A.X K2 Technical Report. arXiv preprint arXiv:2608.30181.
- DeepSeek-AI (2024b). DeepSeek-V3 Technical Report. arXiv preprint arXiv:2412.19437.
- Fishman, M., Chmiel, B., Banner, R., & Soudry, D. (2025). Scaling FP8 training to trillion-token LLMs. International Conference on Learning Representations (ICLR 2025).
- Kalamkar, D., Mudigere, D., Mellempudi, N., et al. (2019). A Study of BFLOAT16 for Deep Learning Training. arXiv preprint arXiv:1905.12322.
- Micikevicius, P., Narang, S., Alben, J., et al. (2018). Mixed Precision Training. International Conference on Learning Representations (ICLR).
- Micikevicius, P., Stosic, D., Burgess, N., et al. (2022). FP8 Formats for Deep Learning. arXiv preprint arXiv:2209.05433.
- NVIDIA (2022). NVIDIA H100 Tensor Core GPU. Product specifications.
- The Microsoft AI Team (2026). MAI-Thinking-1: Building a Hill-Climbing Machine. Technical report, Microsoft AI.

References: Codebases and playbooks

- Ben Allal, L., et al. (2025). The Smol Training Playbook: The Secrets to Building World-Class LLMs. Hugging Face.
- Karpathy, A. (2025). nanoGPT: model.py. GitHub repository.
- Karpathy, A. (2026). nanochat. GitHub repository.

Image credits

- Kircher, A. (1665). Alchemical furnace crowned with a cluster of retorts. *Mundus subterraneus*, Fornax spagyrica. Science History Institute. Public domain.
- Boyle, R. (1669). Boyle's air pump with its glass receiver on a stand. *A continuation of new experiments physico-mechanical*, The I. Plate. Library of Congress. Public domain.
- Schott, G. (1668). Schott's calculating box, rods set in a grid of numbers. *Organum mathematicum*, Icon. VIII. Library of Congress. Public domain.
- Papin, D. (1681). Papin's digesters, pressure vessels with screw clamps and a safety valve. *A new digester or engine for softning bones*, Frontispiece. Library of Congress. Public domain.
- Anonymous (1551). Hour lines of a sundial projected on a sheet. *Compaßt oder Sonnen Ur*. Rijksmuseum. CC0.
- Magalotti, L. (1667). Florentine thermometers of several forms and a hygrometer. *Saggi di naturali esperienze fatte nell'Accademia del Cimento*, Tav. VIII. Library of Congress. Public domain.
- Siemienowicz, K. (1650). Barrel set with a fan of rockets, a firework display on a plinth. *Artis magnae artilleriae pars prima*. Science History Institute. Public domain.
- Siemienowicz, K. (1650). Conversion wheel giving the weight of 100 Roman pounds in the pounds of other nations. *Artis magnae artilleriae pars prima*. Science History Institute. Public domain.
- Kircher, A. (1665). Combination table linking every alchemical substance with every other. *Mundus subterraneus*, Tabula combinatoria. Science History Institute. Public domain.
- Kircher, A. (1665). Furnaces, stills and retorts of the chemical laboratory. *Mundus subterraneus*. Science History Institute. Public domain.

Answers

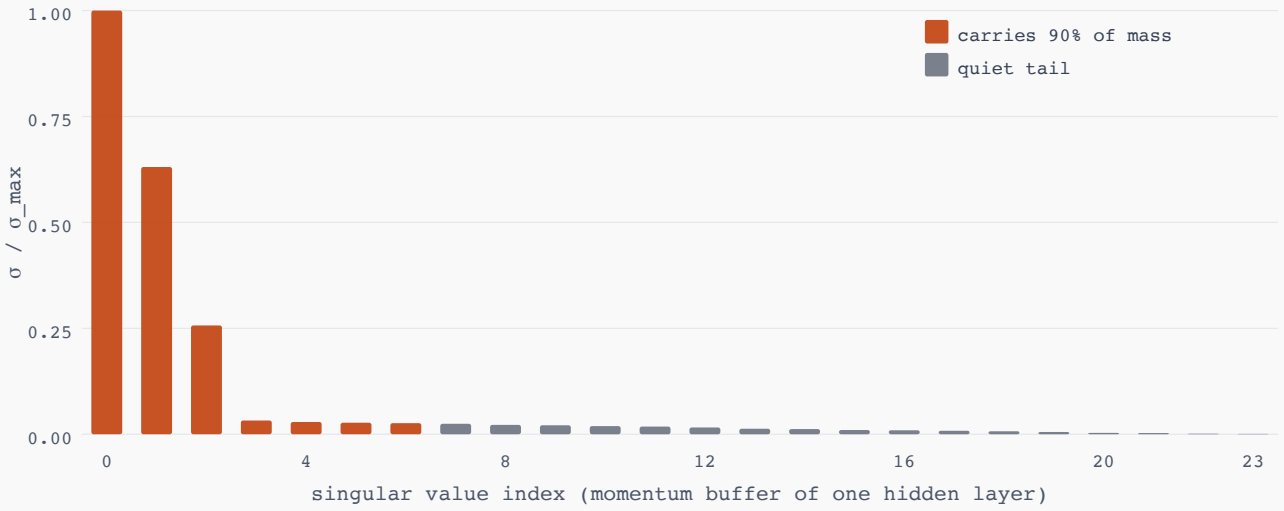
A1 · Momentum concentrates in a few directions

ILLUSTRATIVE Buffer: M_t = 0.95·M_(t-1) + G_t (50 steps, 24×24)

σ for 90% of mass: 7/24

Gradient: G_t = ρ·signal + (1-ρ)·noise

signal weight ρ = 0.70



signal weight ρ: 0.70 (share of each gradient that repeats step after step)

0 = fresh noise every step 1 = same rank-3 signal

Each bar is one singular value σ of the simulated momentum buffer M_t = 0.95·M_(t−1) + G_t after 50 steps for a 24×24 hidden-layer matrix, normalized to σ_max = 1. Each gradient mixes a persistent rank-3 signal (weight ρ) with fresh noise (weight 1−ρ). Orange bars carry 90% of the spectral mass; gray bars are the quiet tail. High ρ (a strong shared signal) concentrates the spectrum: the momentum buffer is effectively low-rank.

- Here 7 of 24 singular values carry 90%: effectively low-rank
- Pure noise needs 16, strong signal only 3

▼ A simulated momentum matrix is effectively low-rank

The question asks about a simulated momentum matrix of size 24×24, and the answer is 7. Such a matrix has 24 singular values, one per rank-one direction $\sigma_i u_i v_i^\top$ of its singular value decomposition, and not one per entry: an $m \times n$ matrix has $\min(m, n)$ of them, however many entries it holds. AdamW treats the same matrix as 576 separate coordinates, while Muon works with these 24 directions. At the slider's starting value, 7 of the 24 directions carry 90% of the matrix's spectral mass. With pure noise the count is 16. With a gradient that is almost all signal, from $\rho = 0.8$ upward, it is 3, the rank of the signal.

Sort the singular values, $\sigma_1 \geq \cdots \geq \sigma_{24}$. The readout "$\sigma$ for 90% of mass" is the smallest count k_{90} with $\sigma_1 + \cdots + \sigma_{k_{90}}$ at least 90% of the total. A small k_{90} out of 24, a few directions carrying most of the matrix, is what **effectively low-rank** means here. It does not mean that the matrix has algebraic rank k_{90}: the other singular values are small, but they are not zero.

Each simulated gradient mixes two parts, $G_t = \rho\, S_t + (1-\rho)\, N_t$, and both parts are scaled to unit Frobenius norm first. The **signal** S_t stands for the part of the gradient that every minibatch shares, roughly the full-batch gradient. Here it is a sum of the same three fixed rank-one matrices at every step, with weights that vary slowly over the 50 steps. The **noise** N_t stands for how one minibatch's gradient differs from that: a new random 24×24 matrix at every step, independent of the earlier ones. The slider moves the signal weight ρ from 0 to 1 in steps of 0.05, and the prompt's setting is $\rho = 0.7$. Three things are simplified: a real full-batch gradient drifts as the weights change, its rank is not fixed at 3, and real minibatch noise is not equally large in every direction.

Why does a shared signal concentrate the spectrum? Unrolled from zero, the buffer $M_t = 0.95\, M_{t-1} + G_t$ weights the gradient from j steps back by 0.95^j:

$$M_{50} = \sum_{j=0}^{49} 0.95^j\, G_{50-j}.$$

Two assumptions come from the setup.

- **A1.** The signal points in the same three directions at every step. The weights on those three directions vary slowly, so treating them as equal makes the signal's growth of about 18.5 an approximation.
- **A2.** The noise terms are independent from step to step, so they partly cancel.

For independent terms of equal size the squared sizes add, so their weighted sum grows like the square root of the summed squared weights.

Claim. Under A1 and A2, the buffer's signal grows about six times as much as its noise over the 50 steps:

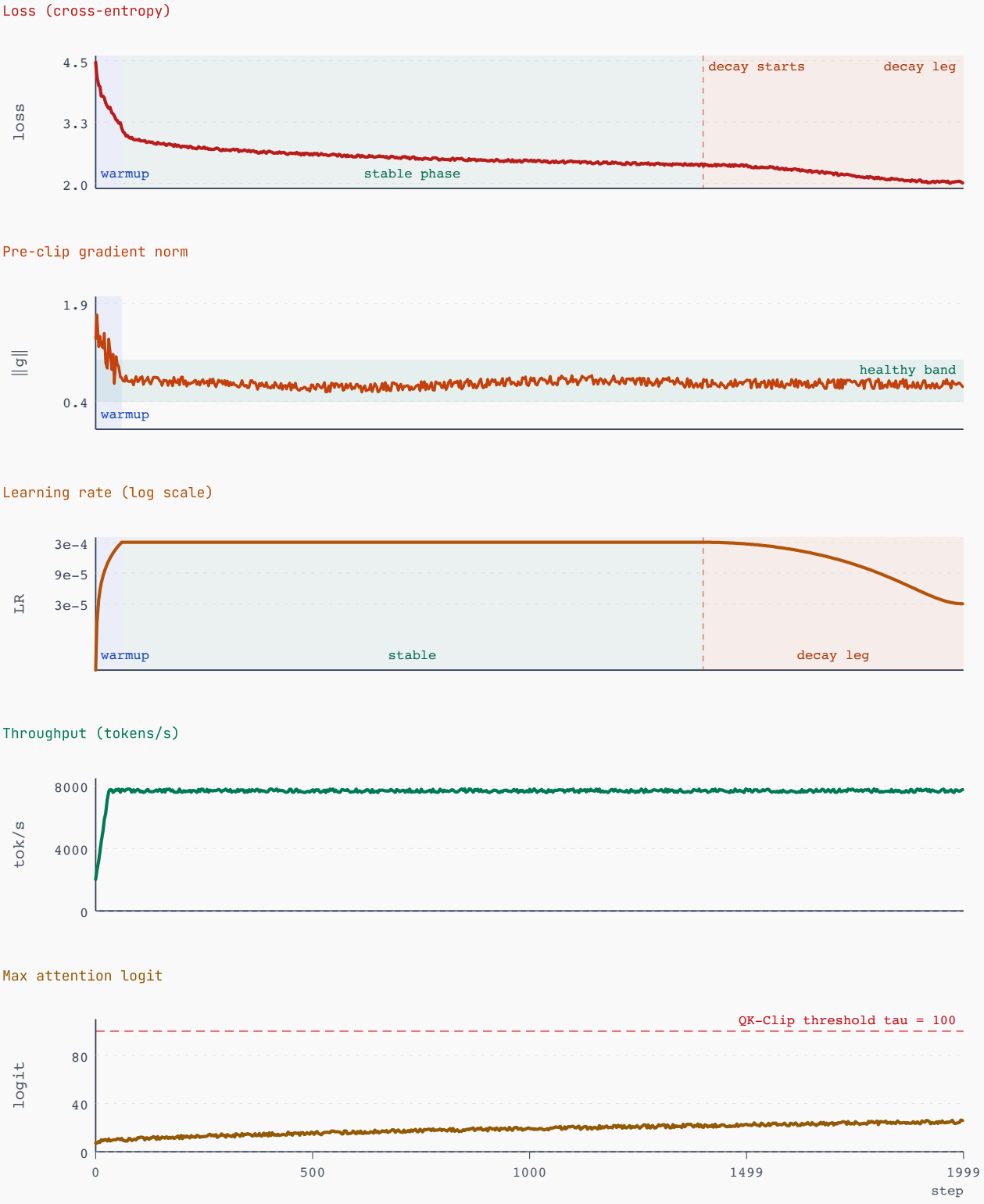
$$\begin{aligned} \text{signal:} \quad & \sum_{j=0}^{49} 0.95^j = \frac{1-0.95^{50}}{1-0.95} \approx 18.5 && \text{A1} \\ \text{noise:} \quad & \Big(\sum_{j=0}^{49} 0.95^{2j}\Big)^{1/2} = \sqrt{\frac{1-0.95^{100}}{1-0.95^2}} \approx 3.2 && \text{A2.} \end{aligned}$$

So the mass piles into the three signal directions. At $\rho = 0$ nothing adds up coherently. Even then the count is 16 and not 24, because the singular values of a random matrix are unequal too. The bars come from this simulation, not from a trained network.

Back to the question

A2 · Decaying the learning rate drops the loss

ILLUSTRATIVE Healthy WSD training run: dashboard traces



Synthetic traces of a healthy WSD run, drawn so that each phase is easy to see. They are not a recorded course run.

- Faster: the loss drops 0.35 against 0.12 over equal spans
- Phases: fast drop, slow middle, decay drop
- Loss across tokenizers: not comparable

▼ The loss falls faster once the learning rate decays

The loss falls faster. The widget is the dashboard of an illustrative healthy run under a warmup-stable-decay schedule: five panels on one shared axis of optimizer steps. The traces are synthetic, so neither their values nor their smoothness say anything about performance. The learning rate warms up over the first 3% of the steps, holds its peak until 70% of the run, and then follows a cosine down to 10% of the peak.

The loss curve has three phases: a fast drop at the start, a slow decline through the long stable middle, and a second drop once the decay leg begins. In the widget the loss falls by 0.35 over the last 30% of the run, against 0.12 over the 30% before the decay starts, from 40% to 70% of the run. These numbers come from the illustration, but the effect itself is documented. MiniCPM trained 0.036B models with WSD and reports that in the decay stage, as the learning rate begins to decrease, "the loss experiences a significant rapid decline" and reaches or goes below the loss of a cosine schedule that ends at the same step (Hu et al., 2024). This drop is why a decay leg gives a finished checkpoint.

Absolute loss values are not directly comparable across tokenizers. The loss is a cross-entropy per predicted token, and two tokenizers cut the same text into different units, so their losses measure different predictions.

Back to the question

A3 · Small updates vanish in a bf16 weight

$$\text{spacing at } 1.0 = 2^{-7} \approx 0.0078$$

- bf16 stores 1.0: the update rounds away
- Any update below half a spacing is lost
- Fix: an fp32 master copy, compute stays bf16

▼ Why a small update vanishes in a bf16 weight

bf16 stores 1.0. A floating-point number is a significand times a power of two. bf16 stores 7 fraction bits, and with the implicit leading 1 that gives $p = 8$ bits of significand precision.

Claim. Between 1 and 2, bf16's values are $2^{-7} \approx 0.0078$ apart. The power of two is fixed at 2^0 there, so for integers f from 0 to 127 the values are

$$\begin{aligned} x_f &= \left(1 + f \cdot 2^{-7}\right) \times 2^0 && \text{7 fraction bits} \\ x_{f+1} - x_f &= 2^{-7} = 2^{1-p} \approx 0.0078 && p = 8. \end{aligned}$$

The next bf16 value above 1.0 is therefore 1.0078125. The exact sum $1 + 0.001 = 1.001$ lies between the two, 0.001 above 1.0 and about 0.0068 below 1.0078125. Round-to-nearest stores the closer value, 1.0. Any update smaller than half a spacing, $2^{-8} \approx 0.0039$ at this weight, rounds away in the same way, however many steps add it. Between 2^e and 2^{e+1} the spacing is 2^{e-7}, so it doubles with every power of two the weight grows, and the larger a weight is compared with its updates, the more of them are lost.

The fix is a second copy of the weights in fp32, the **master weights** of the optimizer's memory account. fp32 has $p = 24$ bits of significand precision, so near 1.0 its values are $2^{-23} \approx 1.2 \times 10^{-7}$ apart and an update of 0.001 survives. The optimizer adds each update to the fp32 master copy, and the bf16 copy used for computation is cast from it. Small updates then accumulate in fp32 until together they move the weight past a bf16 spacing. Ten updates of 0.001 bring the master copy to $1 + 10 \times 0.001 \approx 1.010$, which casts to 1.0078125 in bf16. A weight stored only in bf16 still reads 1.0 after all ten. The course trainer keeps its parameters in fp32 and lowers the precision only inside the forward pass, so its parameters play the role of the master copy.

Back to the question